

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Магнитогорский государственный технический университет
им. Г.И. Носова»
(ФГБОУ ВО «МГТУ им. Г.И. Носова»)

Кафедра вычислительной техники и программирования

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

на тему:

**Проектирование и разработка информационной системы управления
волонтерами с элементами геймификации и аналитики**
по направлению подготовки 09.03.01 Информатика и вычислительная техника
профиль «Проектирование и разработка Web-приложений»

Исполнитель: Зозин Павел Алексеевич, студент 4
курса, группы Авб-22-2
Руководитель: Гладышева Мария Михайловна

Магнитогорск, 2026

ЗАДАНИЕ

на выполнение выпускной квалификационной работы

Обучающемуся Зозин Павел Алексеевич, группа Авб-22-2.

Тема работы: Проектирование и разработка информационной системы управления волонтерами с элементами геймификации и аналитики.

Исходные данные к работе: материалы технического задания, результаты преддипломной практики, программная реализация Пульс, проектная документация, нормативные требования к оформлению ВКР.

Содержание расчетно-пояснительной записки:

- 1) провести теоретико-информационный анализ управления волонтерской деятельностью, существующих решений и сформировать требования к системе;
- 2) спроектировать и реализовать Пульс, включая архитектуру, модели данных, алгоритмы, интерфейс и визуализацию итогов;
- 3) провести проверку работоспособности и оценить результаты эксплуатации системы на пользовательских сценариях.

Перечень графического материала: IDEF0-декомпозиция, дерево целей, информационные потоки, концептуальная модель, архитектура, модели базы данных, схема функционирования, блок-схемы алгоритмов, карта экранов и макет итоговой аналитики.

Дата выдачи задания: «22» апреля 2026 г.

Срок представления работы к защите: «___» июня 2026 г.

Руководитель ВКР _____ /М.М. Гладышева/

Обучающийся _____ /П.А. Зозин/

ОТЗЫВ РУКОВОДИТЕЛЯ ВКР

на выпускную квалификационную работу обучающегося группы Авб-22-2

Зозин Павел Алексеевич

на тему: «Проектирование и разработка информационной системы управления волонтерами с элементами геймификации и аналитики»

Данная страница включена в комплект пояснительной записки как место для официального отзыва руководителя. Текст отзыва, оценка, рекомендация к защите и подпись должны быть внесены руководителем на утвержденном кафедрой бланке; автоматическая генерация не подменяет официальный подписанный документ.

Актуальность и соответствие темы заданию:

Качество проектных решений и программной реализации:

Достоинства, замечания и рекомендации:

Заключение о возможности допуска к защите и оценка:

Руководитель ВКР _____ /М.М. Гладышева/ «__»
_____ 2026 г.

РЕФЕРАТ

Пояснительная записка содержит введение, 3 раздела, заключение, 33 нумерованные таблицы, 39 рисунков, список из 46 использованных источников и приложения. В работе представлены модели предметной области и данных, архитектура, алгоритмические схемы, макеты интерфейса, результаты сборки и программа опытной эксплуатации продукта.

Ключевые слова: ИНФОРМАЦИОННАЯ СИСТЕМА, ВОЛОНТЕРЫ, WEB-ПРИЛОЖЕНИЕ, ГЕЙМИФИКАЦИЯ, АНАЛИТИКА, GO, VUE.JS, POSTGRESQL, REST API, СЕРТИФИКАТЫ

Объектом разработки является процесс управления волонтерской деятельностью в организациях, которые проводят мероприятия, распределяют задачи, учитывают вклад участников и формируют отчетность.

Цель работы - разработка web-ориентированной информационной системы Пульс, позволяющей вести учет волонтеров, мероприятий, задач и подтвержденных часов, а также повышать вовлеченность участников за счет достижений, рейтингов и персональной статистики.

В ходе работы выполнен анализ предметной области, сформулированы функциональные и нефункциональные требования, спроектирована архитектура приложения, разработаны серверная часть на языке Go, клиентская часть на Vue.js и база данных PostgreSQL. Реализованы модули авторизации, организаций, мероприятий, задач, учета времени, уведомлений, базы знаний, геймификации, аналитики, сертификатов и системного администрирования.

Практическая значимость результата заключается в возможности использовать систему как основу для автоматизации деятельности малых и средних волонтерских организаций, студенческих объединений и социальных проектов.

СОДЕРЖАНИЕ

Оглавление обновляется автоматически

ВВЕДЕНИЕ

Развитие цифровых сервисов изменило подход к организации общественных, образовательных и социальных инициатив. Волонтерские объединения работают как устойчивые организационные структуры, которым требуется учет участников, планирование мероприятий, распределение задач, подтверждение часов и подготовка отчетов. При отсутствии единой информационной системы эти процессы распределяются между таблицами, мессенджерами, электронными формами и устными договоренностями.

Такая организация работы создает ряд практических проблем. Данные о волонтерах быстро устаревают, история участия хранится неполно, координатору сложно определить доступных исполнителей, а руководитель организации не получает оперативную картину по мероприятиям, задачам и фактически подтвержденному вкладу. Дополнительно возрастает нагрузка на сотрудников, которые вручную проверяют заявки, считают часы, готовят справки и сертификаты.

Актуальность темы подтверждается длительным развитием добровольческого движения и его цифровой инфраструктуры. По материалам Ассоциации Добро.рф, в 2014 году добровольцами становились 5% россиян [8], а действующая цифровая платформа сообщает более чем о 9 млн пользователей [9]. Таким образом, подтверждаемая данными динамика охватывает период 2014-2025 годов, то есть более десяти лет. Для прикладной системы это означает устойчивый запрос на проверяемый учет участия, часов и результатов добровольцев.

Количественные ориентиры актуальности приведены в таблице 1. Они фиксируют не прогноз, а опубликованные значения и нормативные этапы, по которым можно оценить масштаб предметной области и продолжительность ее развития.

Объектом разработки является процесс управления волонтерской деятельностью в организации. Предметом разработки является web-ориентированная информационная система Пульс, обеспечивающая учет

волонтеров, организаций, мероприятий, задач, времени, достижений, сертификатов и аналитических показателей.

Целью выпускной квалификационной работы является повышение эффективности управления волонтерской деятельностью за счет проектирования и разработки информационной системы Пульс с элементами геймификации и аналитики.

Для достижения цели необходимо решить следующие задачи:

Таблица 1 - Количественные и нормативные ориентиры актуальности темы

Период	Подтвержденный ориентир	Значение для разработки
1995-2025	Правовое регулирование добровольческой деятельности действует с принятия Федерального закона N 135-ФЗ; интервал - 30 лет	Добровольчество является устойчивой предметной областью, а не краткосрочным явлением
2014	5% россиян становились добровольцами по данным ВЦИОМ, опубликованным Ассоциацией Добро.рф [8]	Начальная точка долгосрочного наблюдения вовлеченности
2025	Цифровой контур платформы Dobro.ru указывает более 9 млн пользователей [9]	Масштаб цифрового взаимодействия требует учета событий, заявок и результатов

1) провести теоретико-информационный анализ управления волонтерской деятельностью, существующих решений и сформировать дерево целей и требования к Пульс;

2) спроектировать и реализовать информационную систему Пульс, включая архитектуру, модели данных, алгоритмы, интерфейс и визуализацию итогов;

3) провести проверку работоспособности и оценить результаты эксплуатации Пульс на ключевых пользовательских сценариях.

Методами выполнения работы являются системный анализ, IDEF0-декомпозиция процесса, моделирование информационных потоков, ER-моделирование и проектирование физической структуры PostgreSQL, проектирование REST API, разработка web-приложения, функциональная проверка сценариев и квантификация пользовательского интерфейса.

Средствами реализации являются язык Go и framework Gin для серверного API, PostgreSQL для реляционного хранения данных, Vue.js 3 и TypeScript для клиентской части, Vite для сборки, Docker-конфигурации для воспроизводимого развертывания. Фактическое наличие соответствующих модулей проверено по исходному коду проекта.

Результатом работы является web-ориентированная информационная система Пульс, связывающая мероприятия, заявки, задачи, подтвержденные часы, достижения, сертификаты и аналитические отчеты в едином контуре данных. Практическая значимость работы состоит в возможности использовать ее как основу для автоматизации малых и средних волонтерских организаций, студенческих объединений и социальных проектов.

1 ТЕОРЕТИЧЕСКИЙ АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И СУЩЕСТВУЮЩИХ РЕШЕНИЙ УПРАВЛЕНИЯ ВОЛОНТЕРСКОЙ ДЕЯТЕЛЬНОСТЬЮ

1.1 Анализ предметной области и информационных процессов управления добровольческой деятельностью

Волонтерская деятельность за последние годы приобрела особое значение как инструмент решения социальных, образовательных, культурных и организационных задач. В работе некоммерческих организаций, образовательных учреждений, муниципальных проектов и инициативных объединений волонтеры участвуют в мероприятиях, выполняют задачи, помогают координаторам и формируют устойчивое сообщество вокруг социально значимых целей.

Рост числа добровольческих инициатив приводит к усложнению процессов управления. Организации должны учитывать сведения о волонтерах, их навыках и интересах, планировать мероприятия, распределять задачи, подтверждать участие, фиксировать затраченное время и формировать отчетность. При малом количестве участников эти процессы могут выполняться вручную, однако при увеличении масштаба ручная модель быстро становится источником ошибок и задержек.

Основными участниками предметной области являются волонтеры, координаторы, администраторы организаций и руководители программ. Волонтеры подают заявки на мероприятия, выполняют поручения, фиксируют вклад и ожидают обратной связи. Координаторы организуют события, распределяют задачи, подтверждают участие и часы. Администраторы управляют пользователями, ролями и настройками организации. Руководители программ используют аналитические сведения для оценки эффективности деятельности.

Для предметной области характерна необходимость централизованного учета. Данные о мероприятиях, заявках, задачах, часах и достижениях должны быть связаны между собой. Если сведения хранятся в разных таблицах, формах

и переписках, организация теряет целостную картину: невозможно быстро определить активных участников, подтвердить вклад конкретного волонтера или подготовить объективный отчет за период.

Отдельной проблемой является мотивация и удержание волонтеров. При отсутствии прозрачной системы признания участники не всегда видят результаты своей работы. Это снижает вовлеченность и затрудняет формирование постоянного сообщества. Элементы геймификации, такие как достижения, значки, уровни, рейтинги и персональная статистика, позволяют визуализировать вклад волонтера и поддерживать культуру признания заслуг.

Развитие web-технологий и распространение мобильных устройств создают предпосылки для разработки единой платформы управления волонтерской деятельностью. Прогрессивное web-приложение, адаптивный интерфейс и клиент-серверная архитектура позволяют обеспечить доступ к системе как с рабочего компьютера координатора, так и со смартфона волонтера. Это особенно важно для молодежной аудитории и распределенных команд.

Таблица 2 - Основные сущности предметной области

Сущность	Назначение	Пример данных
Волонтер	Участник добровольческой деятельности	ФИО, контакты, навыки, интересы, история участия
Организация	Пространство управления волонтерской программой	Название, участники, роли, настройки, логотип
Мероприятие	Событие, требующее участия волонтеров	Дата, место, формат, лимит, заявки, посещаемость
Задача	Поручение, назначаемое одному или нескольким участникам	Описание, срок, приоритет, статус, исполнители
Запись времени	Фиксация затраченных волонтерских часов	Дата, длительность, основание, статус подтверждения
Достижение	Элемент признания и геймификации	Название, условие выдачи, баллы, пользователь
Отчет	Агрегированная управленческая информация	Активность, часы, мероприятия, динамика, эффективность

1.2 Структура предметной области как системы и модель информационных потоков

Для перехода от описания участников к проектированию программного продукта предметная область рассматривается как система взаимосвязанных процессов. На рисунке 1 представлена структура, в которой операции волонтера и координатора формируют данные, используемые для подтверждения вклада и управленческой аналитики.

Структура предметной области Пульс

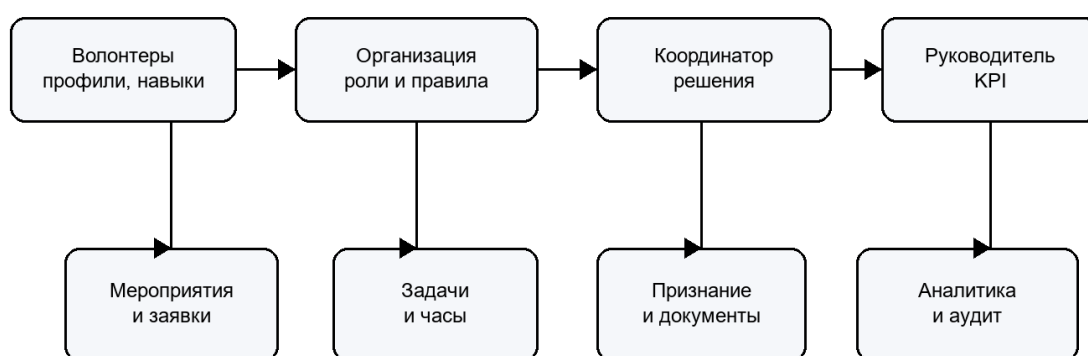


Рисунок 1 - Структура предметной области управления волонтерской деятельностью

Контекстная модель процесса разработана в нотации IDEF0: входами являются сведения о волонтерах, заявки и факты участия; управляющими воздействиями - правила доступа, регламент организации и требования защиты персональных данных; механизмами - пользователи и информационная система; выходами - подтвержденные часы, документы и отчеты. Контекстная диаграмма приведена на рисунке 2.

IDEF0 A-0: Управление волонтерской деятельностью

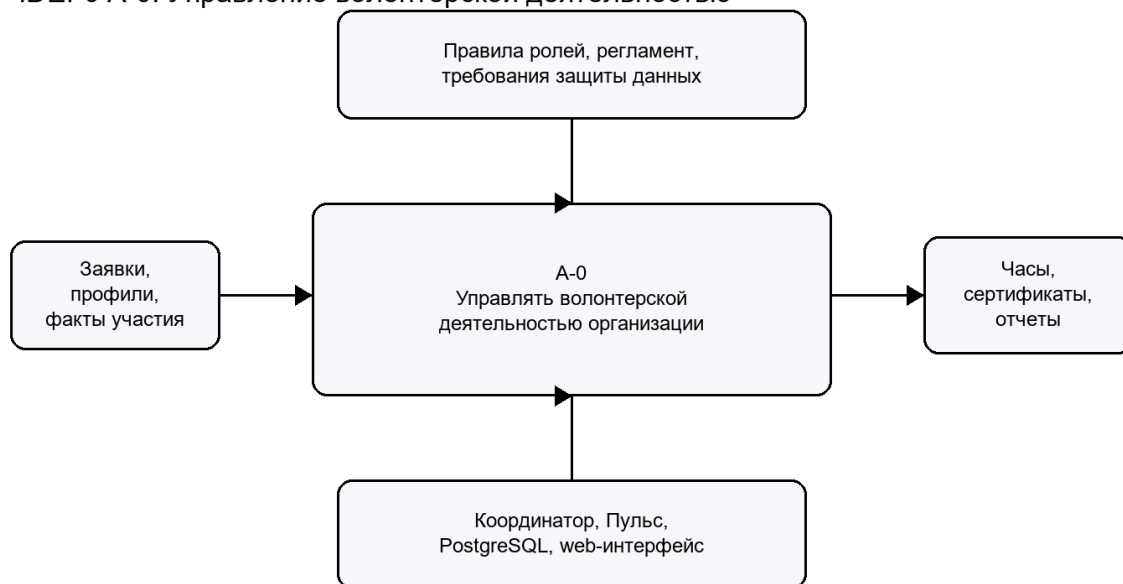


Рисунок 2 - Контекстная диаграмма IDEF0 управления волонтерской деятельностью

Модель информационных потоков на рисунке 3 конкретизирует движение данных внутри программного решения. Решение координатора переводит заявку в подтверждаемое участие; подтвержденная запись времени становится основанием для достижения, сертификата и аналитического показателя. Такая последовательность не позволяет смешивать заявленный и фактически проверенный вклад.

Модель информационных потоков

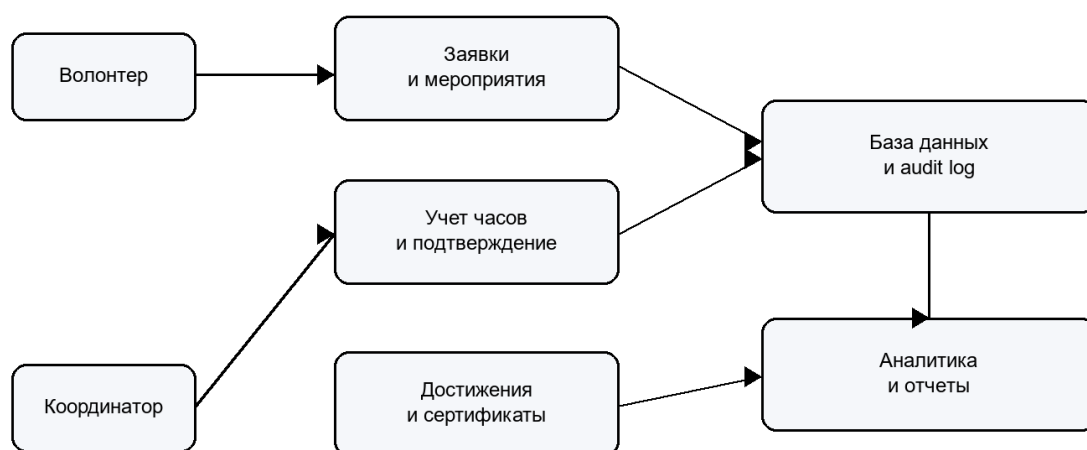


Рисунок 3 - Модель информационных потоков Пульс

1.3 Сравнительный анализ существующих цифровых платформ для привлечения и координации волонтеров

Под сравнительным анализом цифровых платформ в рамках данной работы понимается не только сопоставление отдельных функций, но и оценка того, насколько существующие решения поддерживают полный цикл волонтерской деятельности. Для проектируемой системы важен путь от первичного привлечения участника до подтверждения его вклада: волонтер должен найти подходящее мероприятие, подать заявку, получить решение координатора, выполнить задачу или принять участие в событии, зафиксировать затраченное время и увидеть результат своей активности в профиле. Для организации, в свою очередь, значимы не отдельные формы регистрации, а управляемая среда, в которой сохраняются роли, история участия, отчеты, достижения и данные для последующей аналитики.

Существующие решения закрывают этот процесс неравномерно. Одни платформы хорошо решают задачу публичного поиска добровольческих возможностей, но слабо поддерживают внутреннюю работу конкретной организации. Другие системы удобны для планирования задач или событий, однако не учитывают специфику добровольческой деятельности:

подтверждение часов, мотивацию участников, выдачу сертификатов и накопление персональной истории волонтера. Поэтому анализ аналогов целесообразно проводить через вопрос о том, какую часть жизненного цикла волонтерской программы закрывает сервис и какие элементы приходится выносить во внешние инструменты.

В качестве первой группы можно выделить универсальные CMS и конструкторы сайтов. С их помощью организация способна разместить описание проекта, опубликовать новости и собрать заявки через форму. Однако такая система остается в большей степени информационным сайтом: логика обработки заявок, разграничения ролей, учета часов, формирования достижений и построения аналитики должна создаваться отдельно. В результате организация получает набор доработок вокруг сайта, а не единую информационную систему управления добровольческой деятельностью.

Корпоративные ERP- и ECM-платформы, напротив, обладают развитой процессной логикой, средствами документооборота и механизмами контроля исполнения. Их возможности полезны для крупных организаций, где требуется строгая регламентация и интеграция с другими корпоративными системами. Для небольших волонтерских объединений, образовательных учреждений и НКО такие решения часто оказываются избыточными: они требуют сложного внедрения, обучения пользователей и финансовых затрат, при этом повседневные сценарии волонтера остаются для них вторичными.

Отдельную группу составляют сервисы управления проектами и регистрации на мероприятия. Они позволяют назначать задачи, фиксировать сроки, вести списки участников и контролировать статусы. Эти инструменты хорошо работают как вспомогательная среда координатора, но не формируют целостную добровольческую историю. В них отсутствует связь между заявкой на мероприятие, фактическим участием, подтвержденными часами, достижениями, сертификатами и отчетностью по активности. Именно эта связность данных является принципиальной для проектируемой системы.

Наиболее близкими к предметной области являются специализированные добровольческие платформы. Для уточнения требований были рассмотрены реальные публичные решения: российская платформа Добро.рф, сервис Idealist, объединенный с VolunteerMatch, платформа Timescounts для организации волонтеров на событиях и система Golden для управления добровольцами. Эти сервисы показывают, какие подходы уже применяются в сфере добровольчества, но одновременно выявляют границы готовых решений относительно цели разработки Пульс.

Платформа Добро.рф представляет собой крупную публичную экосистему добровольчества. Ее сильная сторона заключается в масштабе: пользователь получает доступ к каталогу инициатив, может искать проекты, участвовать в мероприятиях и вести личную траекторию волонтера. Такой подход особенно эффективен для массового привлечения участников и популяризации добровольческой деятельности. Вместе с тем для локальной организации платформа выступает прежде всего как внешняя экосистема, а не как полностью контролируемая информационная система, которую можно адаптировать под собственную структуру ролей, закрытые процессы, внутреннюю аналитику и правила геймификации.

Пример публичного интерфейса платформы Добро.рф представлен на рисунке 4.

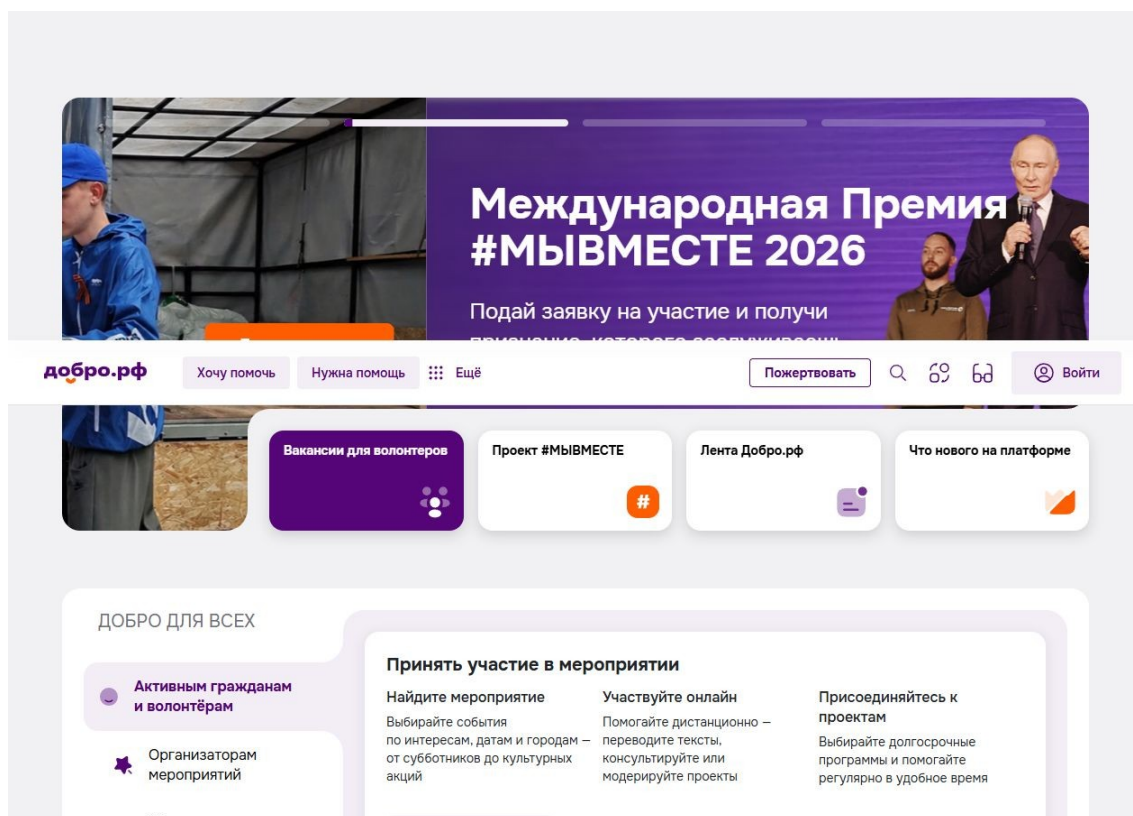


Рисунок 4 - Главная страница платформы Добро.рф

Idealist и связанный с ним VolunteerMatch ориентированы на поиск социальных проектов, вакансий и волонтерских возможностей. Пользовательский сценарий здесь строится вокруг подбора подходящей инициативы по тематике, месту и формату участия. Такая модель удобна для первичного контакта между добровольцем и организацией, однако после привлечения участника значительная часть операционной работы остается вне платформы: учет выполненных задач, подтверждение часов, формирование сертификатов и внутренняя мотивация требуют отдельных решений.

Публичная страница сервиса VolunteerMatch на платформе Idealist показана на рисунке 5.

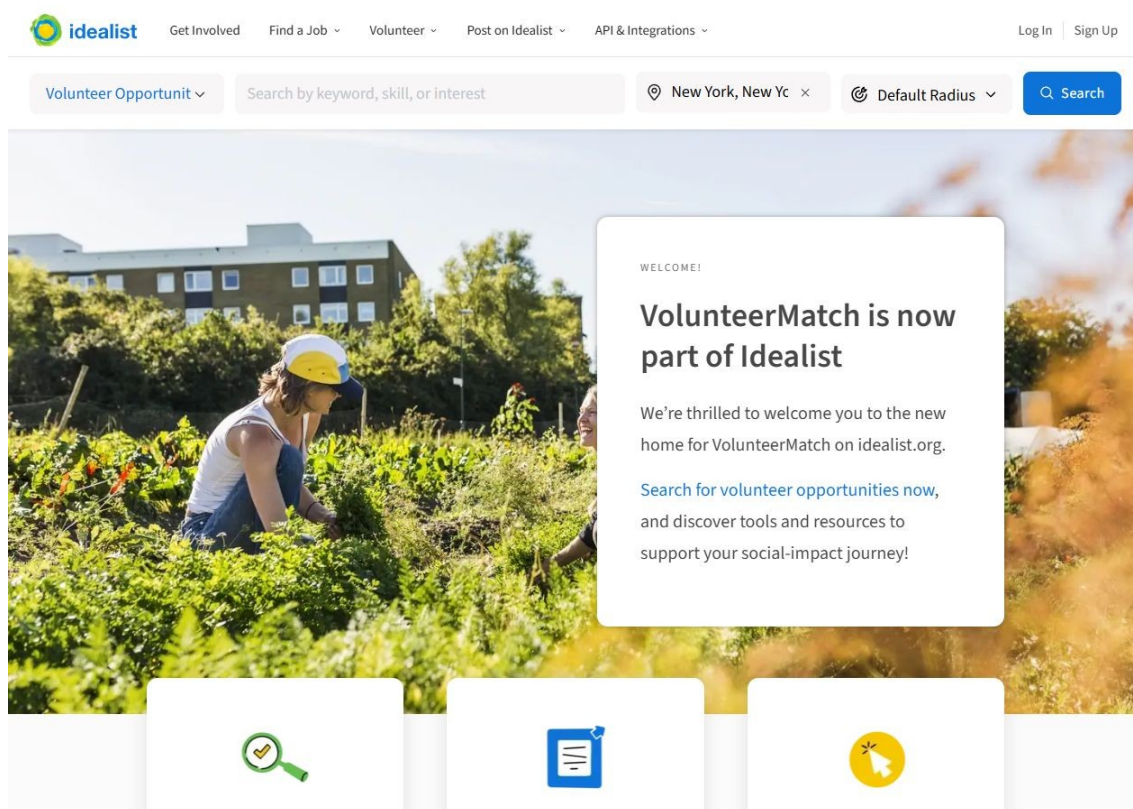


Рисунок 5 - Публичная страница VolunteerMatch на платформе Idealist

Timescounts ближе к задачам операционного управления. Сервис поддерживает рекрутинг, онбординг, расписания и работу с волонтерами на событиях. По сравнению с каталогами возможностей он сильнее ориентирован на деятельность координатора, что делает его полезным примером для анализа. При этом основная логика платформы сосредоточена вокруг событий и расписаний. Для Пульс требуется более широкая модель, где мероприятие является только одной частью процесса наряду с задачами, подтверждением времени, достижениями, сертификатами и аналитикой по организации.

Представление Timescounts, использованное при сравнении, приведено на рисунке 6.

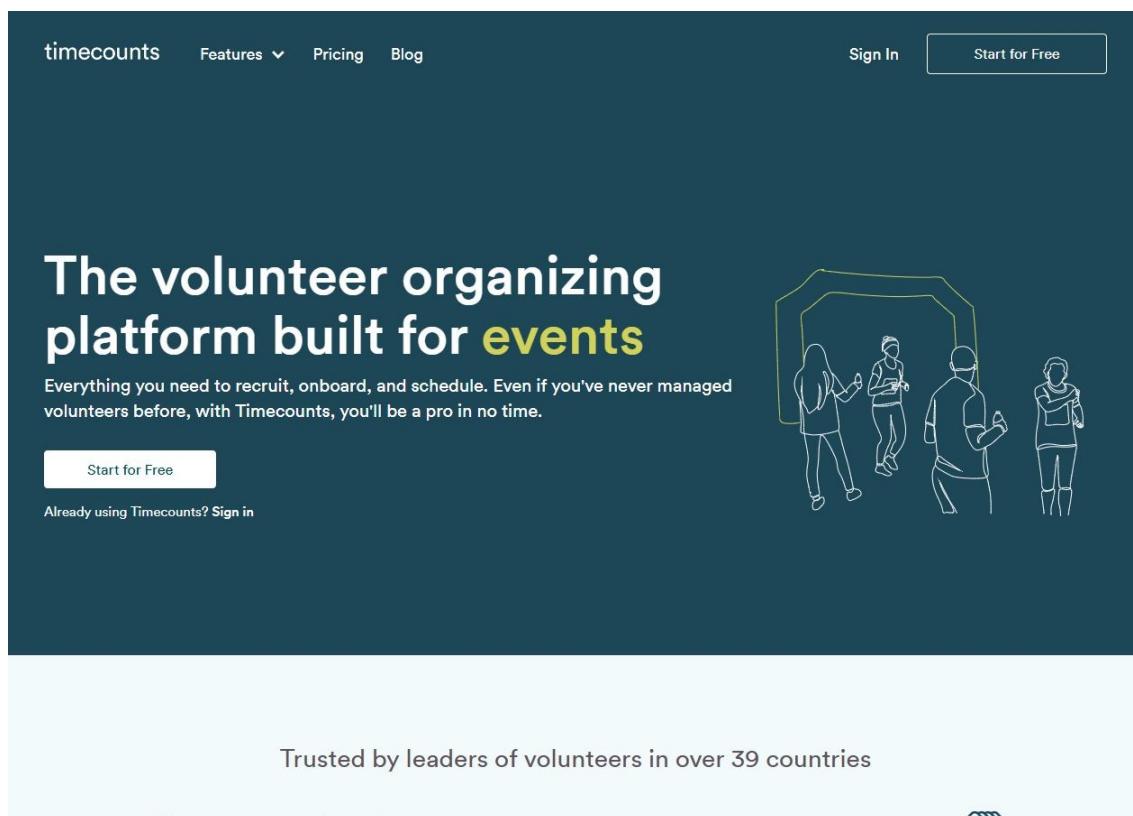


Рисунок 6 - Страница платформы Timecounts

Golden рассматривается как пример коммерческой SaaS-платформы управления волонтерами. В ней заметен акцент на автоматизацию коммуникаций, мобильный доступ и сопровождение добровольческих программ. Такой подход демонстрирует востребованность специализированных систем, но для дипломного проекта важна не покупка готового сервиса, а проектирование собственной архитектуры, модели данных и пользовательских сценариев. Кроме того, готовая внешняя платформа ограничивает контроль над структурой данных и возможностями дальнейшего развития системы под локальные процессы.

Пример интерфейса коммерческого решения Golden приведен на рисунке 7.



Рисунок 7 - Страница платформы Golden

Обобщение рассмотренных групп аналогов приведено в таблице 3. В таблице отражены не только преимущества каждого класса решений, но и ограничения, которые мешают использовать их как полноценную основу для Пульс.

Таблица 3 - Сравнение групп аналогов

Группа решений	Преимущества	Ограничения для задачи Пульс
Универсальные CMS	Гибкость, расширения, быстрый запуск информационного сайта	Нет встроенной логики заявок, часов, ролей, достижений и аналитики
ERP/ECM-платформы	Мощные инструменты документооборота и управления процессами	Высокая стоимость, избыточность, сложность внедрения для небольших организаций
Сервисы управления проектами	Удобные задачи, исполнители, статусы и сроки	Не учитывают мероприятия, волонтерские часы, сертификаты и мотивацию участников
Сервисы мероприятий	Регистрация участников и публичные страницы событий	Недостаточно функций для долгосрочной истории волонтера и отчетности
Добровольческие платформы	Ориентация на добровольчество и поиск мероприятий	Не всегда поддерживают автономное пространство организации, аналитику и геймификацию

Более детальное сопоставление рассмотренных публичных платформ представлено в таблице 4. Оно показывает, что каждая из них решает важную часть задачи, но ни одна не закрывает одновременно локальное управление организацией, учет подтвержденного вклада, мотивационные элементы и аналитическую отчетность в рамках единой модели данных.

Таблица 4 - Сравнение рассмотренных цифровых платформ

Платформа	Основная направленность	Ограничения относительно цели разработки
Добро.рф	Федеральная экосистема добровольчества: поиск проектов, участие в инициативах, личная траектория волонтера	Платформа ориентирована на широкую публичную экосистему и не является локально разворачиваемой системой управления внутренними процессами отдельной организации
Idealist / VolunteerMatch	Поиск волонтерских возможностей, вакансий и организаций, связывание участников с социальными проектами	Акцент сделан на публичном поиске и привлечении участников; функции внутреннего учета часов, задач, сертификатов и геймификации ограничены
Timescounts	Организация волонтеров для событий, рекрутинг, онбординг и расписания	Система ближе к event-менеджменту и требует адаптации под многоорганизационную модель, аналитику и выдачу подтверждающих документов
Golden	Платформа управления волонтерами с автоматизацией, коммуникацией и мобильным приложением	Коммерческое SaaS-решение, ориентированное на готовую платформу; в рамках дипломного проекта требуется собственная архитектура и контролируемая модель данных

Проведенный анализ показывает, что Пульс не должна повторять модель обычного каталога мероприятий или универсального трекера задач. Для достижения цели дипломного проекта требуется система, в которой организация получает собственное пространство управления, координатор работает с мероприятиями, заявками и задачами, а волонтер видит понятную историю своего участия. При этом каждое действие должно оставлять проверяемый след в данных: заявка переходит в участие, участие подтверждается часами, часы влияют на достижения, а накопленные сведения используются в отчетах.

Таким образом, ключевым отличием Пульс является связность пользовательских сценариев и данных. В системе заявка на мероприятие, посещаемость, запись времени, достижение и сертификат не существуют как разрозненные элементы. Они образуют единую цепочку, позволяющую

отследить путь волонтера от регистрации до подтвержденного вклада и управленческой аналитики. Данный вывод используется далее при описании процесса управления волонтерской деятельностью и формировании функциональных требований к разрабатываемой системе.

1.4 Описание процесса организации мероприятий, обработки заявок и учета волонтерских часов

Процесс управления волонтерской деятельностью включает несколько взаимосвязанных этапов. На первом этапе организация формирует пространство работы: определяет координаторов, добавляет участников, настраивает роли и справочники. На втором этапе координатор планирует мероприятие или задачу, указывает сроки, описание, формат, ограничения и требования к участникам.

После публикации мероприятия волонтеры подают заявки. Система должна сохранять статус каждой заявки, комментарии и дату подачи. Координатор рассматривает заявки, подтверждает участие, отклоняет неподходящие заявки или переводит часть участников в лист ожидания. Наличие статусов делает процесс прозрачным и позволяет участнику понимать свое текущее положение.

Во время проведения мероприятия или выполнения задачи фиксируется фактическое участие. После завершения координатор отмечает посещаемость и подтверждает количество часов. Если волонтер самостоятельно создает запись времени, она должна пройти проверку. Только подтвержденные часы могут учитываться в профиле, аналитике, рейтингах и сертификатах.

Следующий этап связан с мотивацией. На основании подтвержденных действий система может начислять достижения, баллы или уровни. Геймификация при этом должна быть встроена в реальные процессы, а не существовать отдельно от них. Достижение должно отражать конкретный вклад: участие, количество часов, выполнение задач, регулярность или инициативность.

Завершающим этапом является аналитика и отчетность. Руководитель или администратор получает агрегированные показатели: количество активных волонтеров, проведенных мероприятий, выполненных задач, подтвержденных часов, выданных сертификатов и динамику активности по периодам. Эти сведения используются для планирования будущих мероприятий и оценки эффективности программ.

Последовательность операций координатора и волонтера обобщена на рисунке 8.

Сценарий проведения мероприятия

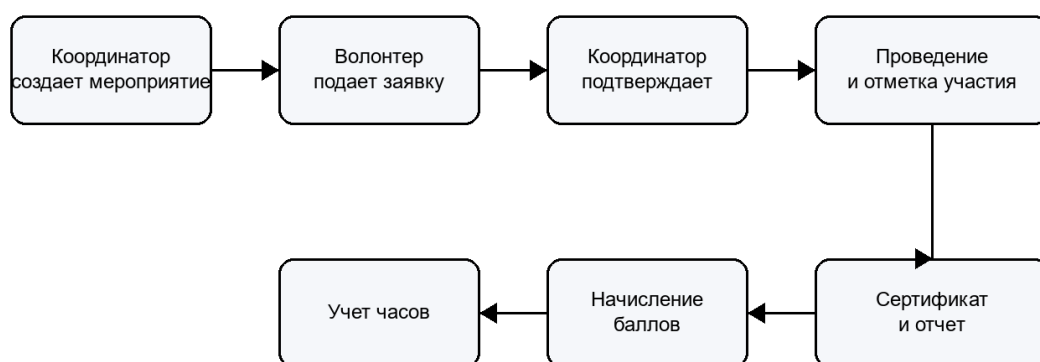


Рисунок 8 - Организация процесса управления волонтерской деятельностью

Таблица 5 - Роли пользователей в процессе управления

Роль	Функции в процессе	Результат работы
Волонтер	Подает заявки, выполняет задачи, фиксирует часы, получает достижения	История участия, подтвержденные часы, сертификаты
Координатор	Создает мероприятия и задачи, рассматривает заявки, подтверждает участие	Актуальные списки участников, корректные статусы, проверенные часы
Администратор организации	Управляет участниками, ролями, настройками и отчетами	Настроенное пространство организации и контроль данных
Руководитель программы	Анализирует показатели и принимает управленческие решения	Сводная аналитика по активности и эффективности

Технический уровень процесса реализуется через централизованную базу данных, серверное API и web-интерфейс. Все операции выполняются через систему, поэтому данные остаются актуальными и связанными. Такой подход снижает количество ошибок, характерных для ручного переноса сведений между таблицами, чатами и документами.

Для Пульс важно поддерживать многоорганизационную модель. Один пользователь может состоять в нескольких организациях и иметь разные роли. Например, в одной организации он может быть координатором, а в другой - обычным волонтером. Поэтому права доступа должны проверяться с учетом контекста организации, а не только глобального признака пользователя.

1.5 Цель, дерево целей и задачи разработки Пульс

Результаты анализа показывают, что разработка Пульс должна быть направлена не на публикацию событий как таковую, а на подтверждаемое управление полным жизненным циклом волонтерского вклада. Дерево целей, представленное на рисунке 9, связывает выявленные проблемы с функциями программного продукта и проверяемыми результатами.

Дерево целей Пульс



Рисунок 9 - Дерево целей разработки Пульс

Таблица 6 - Трассировка целей и результатов системы

Подцель	Реализуемый механизм	Проверяемый результат
Централизовать данные	Профили, организации, мероприятия, задачи, записи времени в PostgreSQL	Данные о событии и вкладе связаны ключами и доступны по роли
Автоматизировать операции	Заявки, подтверждение часов, сертификаты, уведомления	Сценарий от заявки до документа выполняется в системе
Оценивать результаты	Analytics, audit log, выгрузки отчетов	Dashboard отображает KPI по подтвержденным данным

Целью работы является повышение эффективности управления волонтерской деятельностью посредством проектирования и разработки информационной системы Пульс с элементами геймификации и аналитики. В соответствии с деревом целей сформулированы три задачи: выполнить теоретико-информационный анализ и постановку требований; спроектировать и реализовать программное решение; проверить работоспособность и оценить полученные результаты.

Граница автоматизации определяется жизненным циклом подтверждаемого вклада. В Пульс не моделируются все внешние процессы благотворительной организации, например бухгалтерский учет или сбор пожертвований. В контур системы включены именно операции, которые связывают участника, мероприятие или задачу, проверенное время, признание результата и управленческий отчет.

Выбор такой границы позволяет избежать подмены предметной цели набором несвязанных функций. Мероприятие само по себе не доказывает результат волонтерской работы: для этого необходимы заявка, решение координатора, подтвержденная запись времени и возможность вывести агрегированный показатель. Поэтому критерием полноты системы является связность этих данных.

Для волонтера достижение цели выражается в доступности мероприятий, понятном статусе заявки, отражении подтвержденных часов, достижений и сертификатов. Для координатора результатом являются управляемые списки

участников и проверяемые операции подтверждения. Для руководителя результат проявляется в аналитических показателях и выгрузках, основанных на операционных данных.

Достоверность результата требует разграничения ролей. Пользователь не должен самостоятельно подтверждать собственные часы либо получать доступ к данным другой организации только вследствие авторизации. Это определяет обязательность серверных проверок принадлежности к организации, статусов операций и фиксации значимых действий в журнале аудита.

Из дерева целей следуют критерии последующей проверки: существование связанных сущностей в базе данных; наличие маршрутов обработки заявки, времени, сертификата и аналитики; доступность экранов для соответствующих ролей; успешная сборка компонентов; испытание сценария от регистрации до отчета на подготовленных данных.

Таким образом, постановка цели переводит выявленные проблемы предметной области в проверяемые проектные результаты. Каждая обязательная схема второй главы и каждый сценарий третьей главы должны показывать не декларативное наличие функции, а ее место в достижении целей пользователей и организации.

Результаты исследования предметной области, сопоставления аналогов и постановки целей обобщены в таблицах 2-6.

1.6 Выводы по первой главе

Необходимость разработки Пульс обусловлена недостатками существующих решений и практическими потребностями небольших и средних волонтерских организаций. В реальной работе часто используются связки из таблиц, мессенджеров, онлайн-форм и ручных отчетов. Такая модель проста на старте, но плохо масштабируется и не обеспечивает достаточной надежности данных.

Типовыми проблемами являются отсутствие централизованного учета волонтеров и их активности, сложность планирования мероприятий, невозможность быстро отследить историю участия, низкая прозрачность

признания заслуг и нехватка аналитических инструментов. Координаторы тратят значительное время на ручную сверку списков, рассылку уведомлений и подготовку отчетности.

Пульс должна закрыть эти дефициты за счет единого web-приложения. В систему закладываются профили волонтеров, организации, мероприятия, задачи, учет времени, уведомления, база знаний, достижения, рейтинги, сертификаты и аналитика. Такой состав модулей отражает полный жизненный цикл волонтерской деятельности.

Централизация данных обеспечивает хранение сведений в единой базе с учетом ролей и прав доступа. Специализированная бизнес-логика позволяет поддерживать сценарии, характерные именно для волонтерства: подачу заявки, рассмотрение координатором, подтверждение участия, начисление достижений, фиксацию времени и формирование отчетов.

Многоорганизационный режим позволяет использовать одну систему для нескольких независимых организаций. Каждая организация получает собственных участников, мероприятия, задачи и настройки, а пользователь может работать в разных пространствах с разными ролями. Это важно для образовательных учреждений, сетей НКО и проектов с несколькими направлениями.

Встроенная геймификация повышает вовлеченность участников. Значки, уровни и рейтинги не должны быть внешним декоративным элементом. Они должны опираться на подтвержденные действия и помогать волонтеру видеть накопленный вклад. Для координатора такая модель также полезна, поскольку позволяет выделять активных участников и поощрять регулярность.

Развитая аналитика необходима для управленческих решений. Сводные панели позволяют отслеживать количество активных волонтеров, закрытых задач, проведенных мероприятий, отработанных часов и выданных сертификатов. Эти показатели помогают оценивать эффективность программ, планировать нагрузку и готовить отчеты для руководства и партнеров.

Таким образом, разработка Пульс обусловлена не только общей цифровизацией, но и конкретными дефицитами существующей практики управления волонтерской деятельностью. Специализированная web-ориентированная система позволяет объединить учет, координацию, мотивацию и аналитику в одном программном продукте.

Итоговые положения по результатам сопоставления существующих решений:

В первой главе рассмотрены особенности современной волонтерской деятельности, основные участники процесса и проблемы, возникающие при ручной организации работы. Показано, что разрозненные таблицы, мессенджеры и отдельные формы не обеспечивают достаточной прозрачности, усложняют подтверждение вклада и увеличивают нагрузку на координаторов.

Анализ предметной области позволил выделить базовые сущности, необходимые для эффективного управления: волонтеры, организации, мероприятия, задачи, записи времени, достижения и отчеты. Именно эти сущности должны стать ядром разрабатываемой информационной системы, поскольку они отражают полный жизненный цикл участия волонтера.

Рассмотрение аналогов показало, что универсальные CMS, ERP- и ECM-платформы, сервисы управления проектами и платформы мероприятий закрывают только часть потребностей. Они либо требуют значительной доработки, либо являются избыточными, либо не поддерживают геймификацию, многоорганизационный режим и аналитику в нужном объеме.

На основании выполненного анализа обоснована необходимость разработки специализированной системы Пульс. Первая глава формирует теоретическую основу проекта и задает требования, которые далее конкретизируются в архитектуре, модели данных, серверной и клиентской реализации.

2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ ПУЛЬС

2.1 Концептуальная модель и архитектура web-приложения

Концептуальная модель отделяет предметный смысл системы от выбранных технологий. Как показано на рисунке 10, роли пользователя инициируют операции предметного ядра, правила доступа и статусы обеспечивают достоверность переходов, а хранилище данных позволяет получить проверяемый результат: вклад, документ и отчет.

Концептуальная модель программного продукта

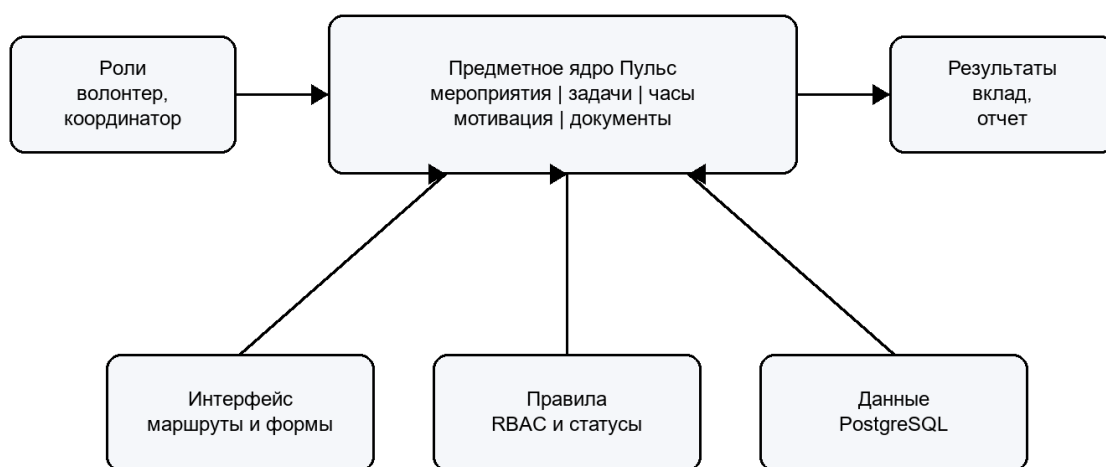


Рисунок 10 - Концептуальная модель программного обеспечения Пульс

Пульс спроектирована как web-приложение с разделением на клиентскую часть, серверное API и уровень хранения данных. Такой подход соответствует характеру задачи: пользователи работают через браузер, бизнес-логика сосредоточена на сервере, а данные хранятся в реляционной базе.

Клиентская часть реализована на Vue.js 3 с использованием TypeScript, Vite и Vue Router. Она отвечает за маршрутизацию, отображение страниц, формы ввода, таблицы, фильтры, навигацию и взаимодействие пользователя с API. Серверная часть реализована на Go и предоставляет REST API для модулей авторизации, организаций, мероприятий, задач, учета времени, уведомлений, аналитики и администрирования.

В качестве системы управления базами данных используется PostgreSQL. Выбор реляционной модели обусловлен большим количеством связей между пользователями, организациями, мероприятиями, задачами, заявками, часами и достижениями. Для локального запуска и проверки используется Docker Compose, что упрощает развертывание backend, frontend и базы данных в согласованной среде.

Архитектурное разделение также упрощает дальнейшее масштабирование. При росте нагрузки клиентская часть может обслуживаться отдельным web-сервером или CDN, серверное API может масштабироваться горизонтально, а база данных может быть вынесена на отдельный управляемый сервер. Для дипломного проекта достаточно локального контейнерного запуска, но структура не препятствует развитию.

REST API выбран как основной способ взаимодействия из-за простоты интеграции с web-интерфейсом и удобства отладки. Каждый предметный модуль имеет собственную группу маршрутов, а обмен данными выполняется в формате JSON. Такой подход понятен для frontend-разработки и хорошо соответствует CRUD-операциям над сущностями системы.

Отдельный worker-процесс предусмотрен для операций, которые не должны блокировать основной запрос пользователя. К ним относятся отправка уведомлений, подготовка отчетов, начисление достижений после завершения мероприятия и иные фоновые задачи. Даже если часть таких функций реализуется постепенно, архитектура заранее учитывает необходимость асинхронной обработки.

Техническая архитектура, реализующая концептуальную модель, представлена на рисунке 11. Она подтверждается исходным кодом frontend на Vue.js/TypeScript, API на Go/Gin, миграциями PostgreSQL и отдельными модулями аналитики, сертификатов, уведомлений и аудита.

Архитектура Пульс

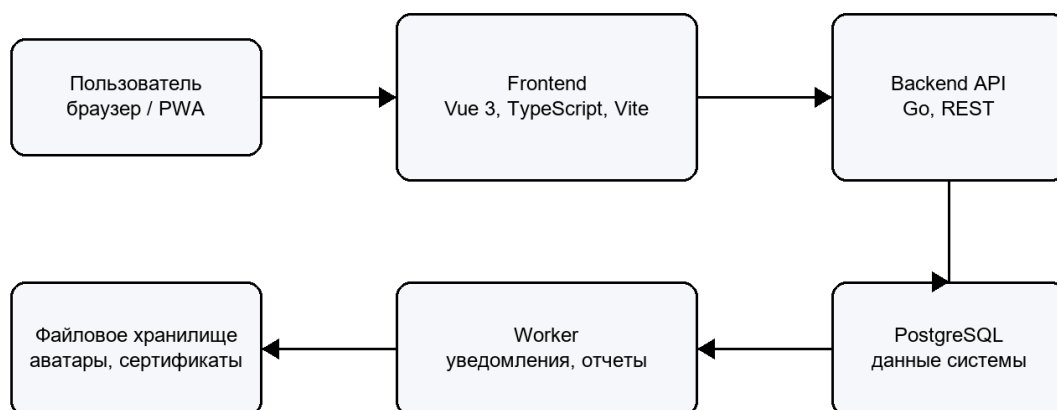


Рисунок 11 - Архитектура информационной системы Пульс

Схема функционирования на рисунке 12 показывает сквозной результат работы продукта: операция считается значимой для отчета только после прохождения предусмотренных проверок и подтверждений.

Сценарий проведения мероприятия

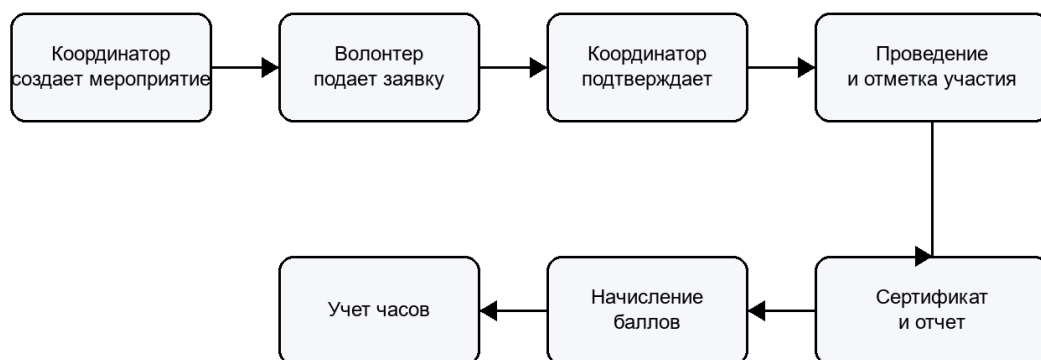


Рисунок 12 - Схема функционирования Пульс от мероприятия к отчету

2.2 Функциональные требования и пользовательские роли

Ролевая модель является основой разграничения доступа в системе. Один пользователь может состоять в разных организациях, а права определяются не только глобальной системной ролью, но и членством внутри конкретной организации. Это позволяет использовать систему как для одной организации, так и как основу для дальнейшего multi-tenant развития.

Неавторизованный пользователь имеет доступ только к входу, регистрации и восстановлению пароля. Волонтер получает доступ к личному кабинету, мероприятиям, задачам, часам, достижениям, уведомлениям и базе знаний. Координатор управляет операционными сущностями своей организации. Администратор организации управляет участниками и настройками. Суперадминистратор работает с системной административной панелью.

Для каждой роли определяются не только разрешенные страницы, но и допустимые действия внутри страницы. Например, волонтер может видеть мероприятие и подать заявку, но не может подтвердить чужую заявку. Координатор может рассматривать заявки и отмечать посещаемость, но только в рамках организации, где он обладает соответствующей ролью.

Состояния сущностей также являются частью требований. Мероприятие может быть черновиком, опубликованным, завершенным или отмененным. Заявка может ожидать рассмотрения, быть подтвержденной, отклоненной, отмененной или находиться в листе ожидания. Задача имеет собственный жизненный цикл от создания до завершения. Явное хранение статусов делает процесс управляемым и проверяемым.

Для снижения ошибок пользовательский интерфейс должен подсказывать допустимые действия. Например, после завершения мероприятия координатору доступна отметка посещаемости и подтверждение часов, а действия по редактированию ключевых параметров должны быть ограничены. Это помогает сохранить целостность данных.

Таблица 7 - Функциональные требования по ролям

Роль	Основные функции	Ограничения
Гость	Регистрация, вход, восстановление пароля	Нет доступа к рабочим разделам
Волонтер	Профиль, заявки, задачи, часы, достижения, уведомления	Не может управлять чужими данными и настройками организации
Координатор	Мероприятия, заявки, задачи, посещаемость, подтверждение часов	Работает только в доступных организациях
Администратор организации	Участники, роли, данные организации, отчеты	Не управляет глобальными настройками приложения
Суперадминистратор	Системная админ-панель, организации, глобальные сущности	Используется для технического администрирования

Состав предметных модулей, реализующих функциональные требования, приведен на рисунке 13.

Функциональные модули системы

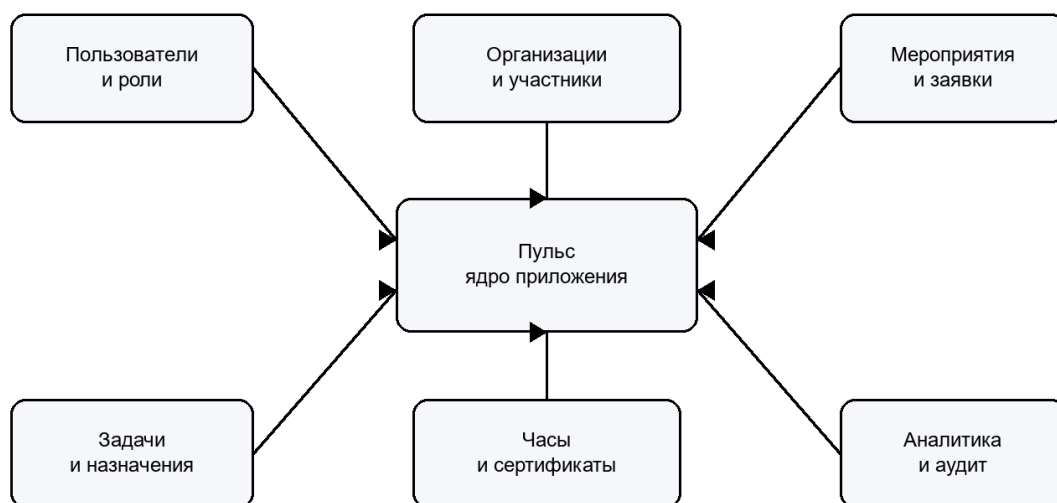


Рисунок 13 - Функциональные модули системы Пульс

2.3 Проектирование базы данных

База данных проектировалась исходя из необходимости хранить как справочную, так и операционную информацию. Центральными сущностями являются users, organizations, organization_members, volunteer_profiles, events,

tasks, time_entries, achievements, notifications, certificates и audit_log. Между ними существуют устойчивые связи, отражающие реальные процессы системы.

Таблица users хранит учетные записи и базовые данные пользователя. Профиль волонтера вынесен в отдельную сущность, что позволяет отделить данные авторизации от предметных характеристик: навыков, интересов, контактной информации и истории участия. Организации связаны с пользователями через таблицу членства, где фиксируется роль участника внутри организации.

Мероприятия и задачи привязаны к организациям. Заявки на мероприятия позволяют хранить статус участия, комментарии и решения координатора. Учет времени связывается с пользователем, организацией, мероприятием или задачей и проходит через подтверждение. Такой подход дает возможность формировать отчеты по разным срезам: по волонтеру, мероприятию, организации, периоду и статусу.

При проектировании учитывалась необходимость восстановления истории. Поэтому для ряда сущностей используются служебные поля даты создания, обновления и мягкого удаления. Мягкое удаление позволяет скрыть запись из интерфейса, но сохранить возможность аудита и анализа, если запись уже участвовала в связанных процессах.

Для справочников навыков, интересов, категорий новостей и категорий базы знаний используется отдельное хранение. Это позволяет администраторам постепенно настраивать систему под конкретную организацию, не меняя программный код. В перспективе такие справочники могут быть частично глобальными и частично локальными для организации.

Индексы и внешние ключи должны обеспечивать быстрый поиск по наиболее частым сценариям: список мероприятий организации, задачи пользователя, записи времени за период, заявки по мероприятию, уведомления текущего пользователя. Это особенно важно для страниц со списками и фильтрами.

Состав данных удобно рассматривать по функциональным группам, представленным на рисунке 14. Такое представление связывает предметные модули с таблицами и показывает, какие данные участвуют в формировании итоговых отчетов.

Визуализация структуры данных

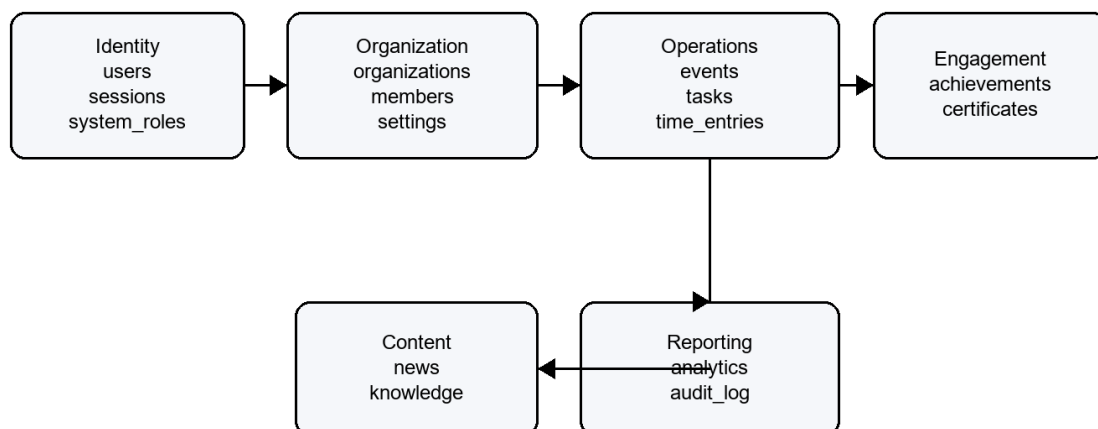


Рисунок 14 - Визуализация структуры используемых данных Пульс

Информационно-логическая модель базы данных приведена на рисунке 15. Модель построена по фактическим миграциям проекта: ключевыми связями являются членство пользователя в организации, заявки на мероприятие, подтвержденные записи времени и формирование сертификата.

Информационно-логическая ER-модель

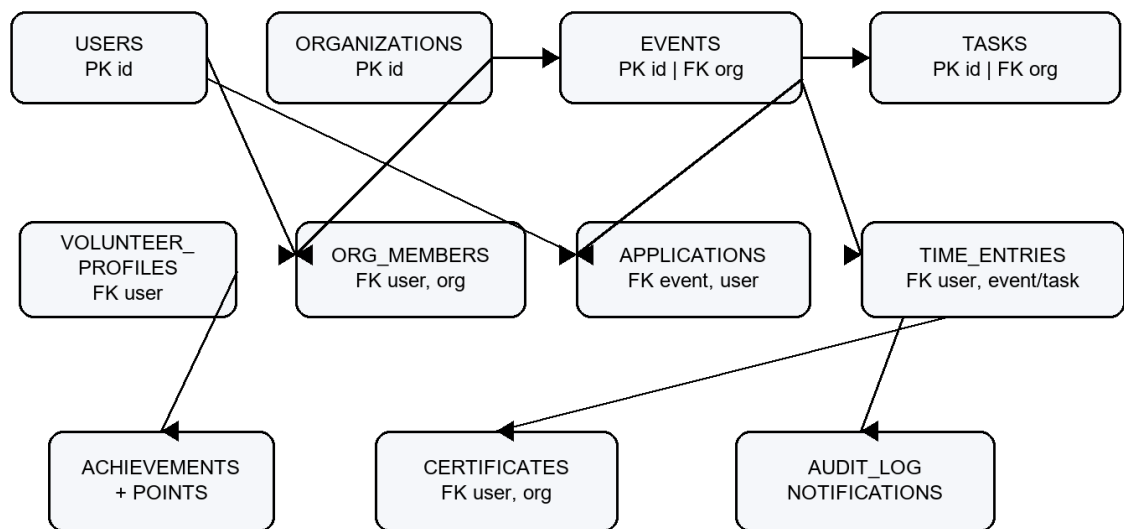


Рисунок 15 - Информационно-логическая модель базы данных Пульс

Физическая модель на рисунке 16 конкретизирует реляционную реализацию PostgreSQL: UUID-ключи, внешние ключи, перечислимые статусы, ограничения целостности и индексы отражены в миграциях `001`, `005`, `008`, `009` и `014` проекта.

Физическая модель PostgreSQL: ключевые ограничения

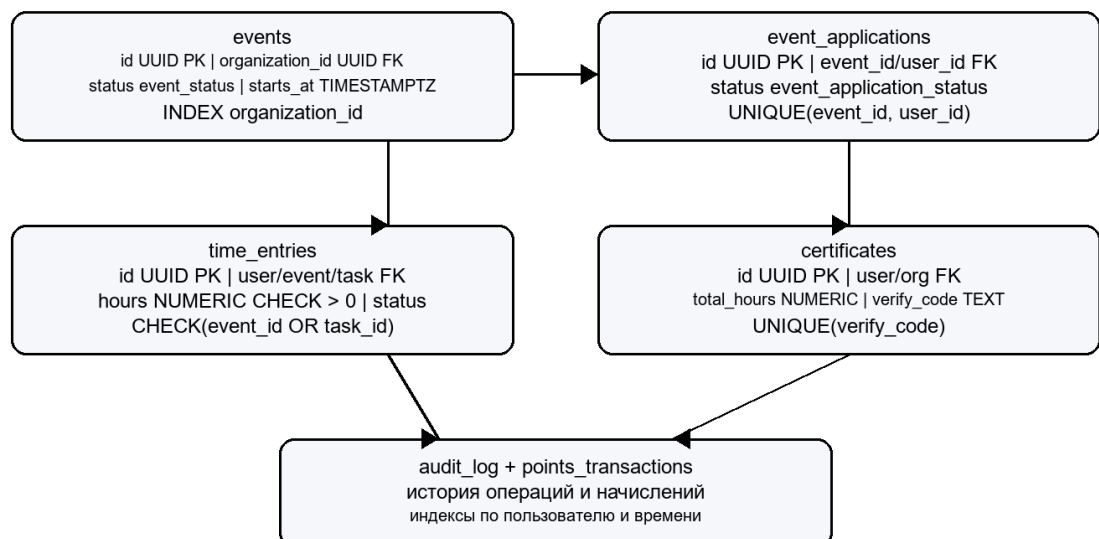


Рисунок 16 - Физическая модель ключевых таблиц PostgreSQL

Таблица 8 - Основные сущности базы данных

Сущность	Назначение	Основные связи
users	Учетные записи и данные авторизации	Связана с профилем, членством, задачами, часами, уведомлениями
organizations	Волонтерские организации и настройки	Связана с участниками, мероприятиями, задачами, сертификатами
events	Мероприятия, смены, заявки и посещаемость	Связана с организацией, заявками, отзывами, часами
tasks	Задачи и назначения исполнителям	Связана с организацией, назначениями, комментариями, часами
time_entries	Записи волонтерских часов и статусы подтверждения	Связана с пользователем, организацией, мероприятием или задачей
achievements	Правила достижений и выданные достижения	Связана с пользователями и транзакциями баллов
certificates	Сертификаты и справки о волонтерской деятельности	Связана с пользователем и организацией
audit_log	Журнал значимых действий пользователей	Связан с пользователем и изменяемыми сущностями

Таблица 9 - Реализованные физические ограничения данных

Таблица	Ограничение/индекс	Назначение
event_applications	UNIQUE (event_id, user_id)	Запрет повторной заявки одного волонтера на одно мероприятие
time_entries	CHECK (hours > 0); CHECK (event_id IS NOT NULL OR task_id IS NOT NULL)	Корректность учетного времени и его основания
certificates	UNIQUE (verify_code)	Однозначная публичная проверка выданного документа
points_transactions	UNIQUE (user_id, achievement_id, source_type)	Защита от двойного начисления награды
audit_log	INDEX created_at, user_id, action	Получение истории действий и аналитика аудита

2.4 Проектирование серверной части

Серверная часть реализует бизнес-логику приложения и предоставляет REST API. Код организован по модулям: auth, users, organizations, events, tasks,

timeentries, notifications, gamification, analytics, certificates, knowledge, admin и другим. Для каждого модуля выделяются доменные структуры, репозиторий, сервис и HTTP-обработчик.

Разделение на слои повышает сопровождаемость проекта. Обработчик отвечает за прием HTTP-запроса и формирование ответа. Сервис содержит бизнес-правила: проверку прав, изменение статусов, начисление баллов, подтверждение часов. Репозиторий инкапсулирует SQL-запросы и работу с базой данных. Такой подход упрощает тестирование и снижает связность между транспортным уровнем и хранилищем.

Авторизация основана на пользовательской сессии и проверке ролей. Для операций изменения данных сервер проверяет не только факт входа, но и принадлежность пользователя к организации, в рамках которой выполняется действие. Например, координатор может изменить заявку только для мероприятия своей организации, а администратор организации не получает автоматически системные права суперадминистратора.

Важным элементом серверной части является аудит. Для операций создания, изменения и удаления записываются события, позволяющие восстановить историю действий. Это особенно важно для систем, где подтверждаются часы и формируются документы, поскольку организация должна понимать, кто и когда изменил статус записи.

Обработка ошибок строится так, чтобы клиентская часть могла показать пользователю понятное сообщение. Ошибки валидации отделяются от ошибок доступа и внутренних ошибок сервера. Например, отсутствие обязательного поля должно возвращать один тип ответа, а попытка изменить данные чужой организации - другой.

Для операций со списками используется постраничная выдача. Это предотвращает загрузку слишком большого объема данных в одном запросе и делает интерфейс стабильнее при росте числа пользователей, мероприятий или задач. Параметры поиска и фильтрации передаются через query-параметры API.

Отдельное внимание уделяется сертификатам. Сервер формирует документ, сохраняет сведения о нем и присваивает проверочный код. Публичный endpoint проверки не раскрывает лишние персональные данные, но позволяет подтвердить подлинность документа и его связь с организацией.

Таблица 10 - Основные группы API

Группа API	Назначение
/auth	Регистрация, вход, обновление сессии, выход, восстановление и смена пароля
/organizations	Создание организаций, управление участниками и ролями
/events	Мероприятия, заявки, посещаемость, смены и обратная связь
/tasks	Задачи, назначения, комментарии, вложения и история статусов
/time-entries	Создание, редактирование, подтверждение и отклонение часов
/analytics	Отчеты по активности, мероприятиям, задачам, геймификации и аудиту
/certificates	Генерация, скачивание и публичная проверка сертификатов
/admin	Универсальная системная административная панель

Критические серверные операции представлены блок-схемами на рисунках 17-21. В схемах использованы терминаторы, блоки обработки и блоки принятия решений, соответствующие назначению условных обозначений ГОСТ 19.701-90.

Блок-схема обработки заявки

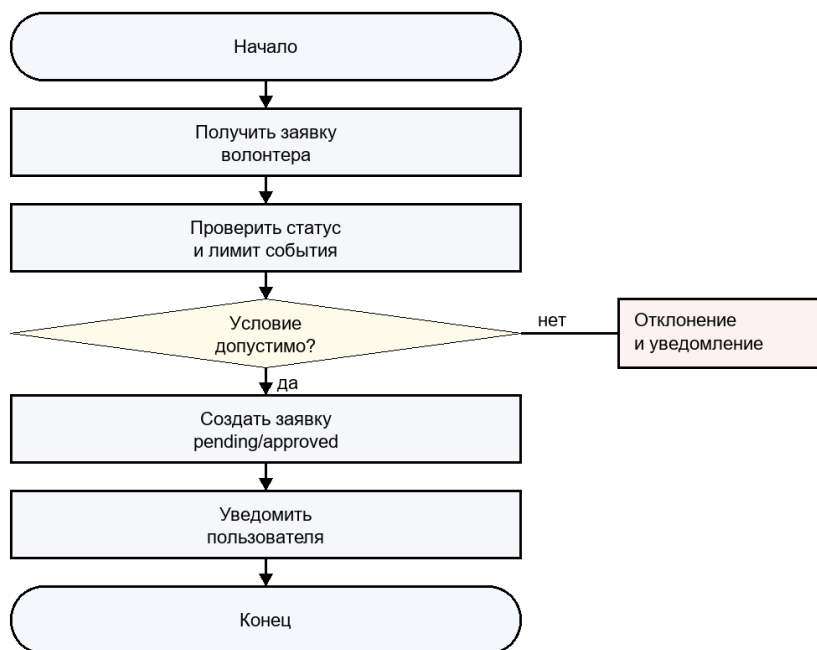


Рисунок 17 - Блок-схема алгоритма обработки заявки на мероприятие

Блок-схема подтверждения часов

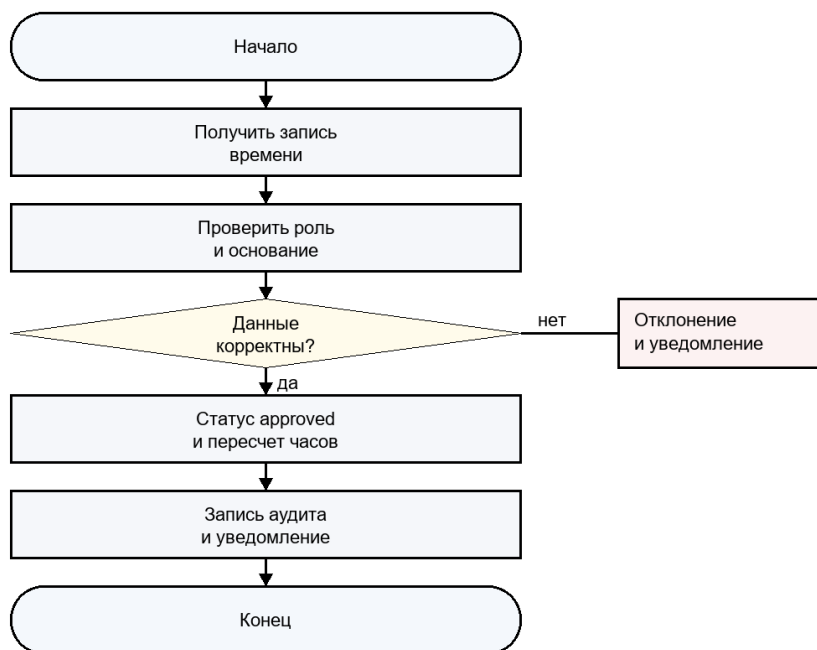


Рисунок 18 - Блок-схема алгоритма подтверждения волонтерских часов

Блок-схема начисления достижения

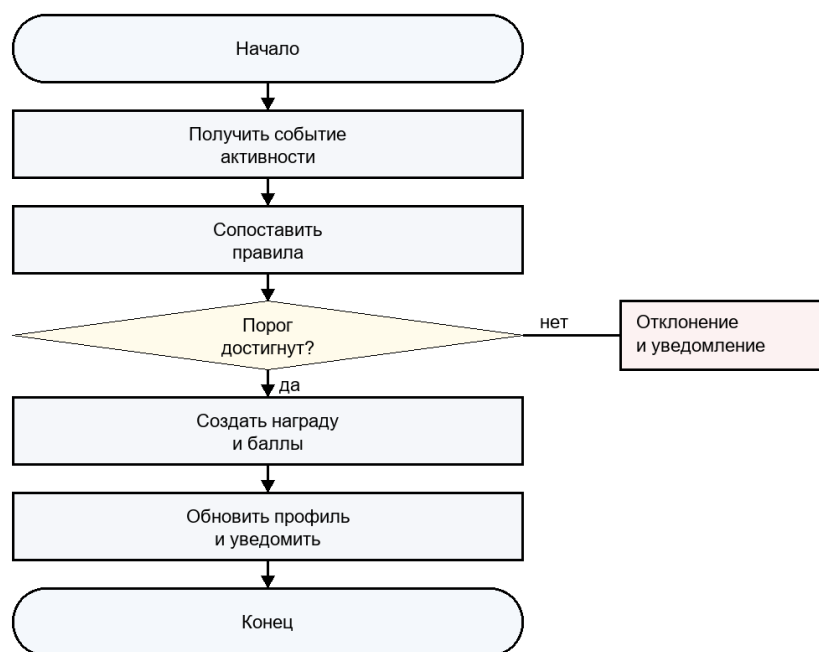


Рисунок 19 - Блок-схема алгоритма начисления достижения

Блок-схема генерации сертификата

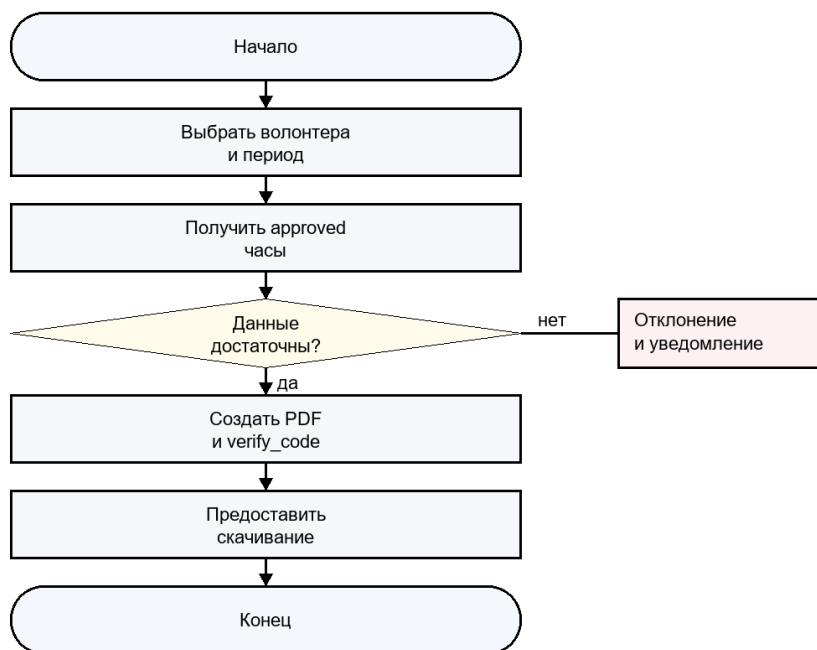


Рисунок 20 - Блок-схема алгоритма формирования сертификата

Блок-схема формирования аналитики

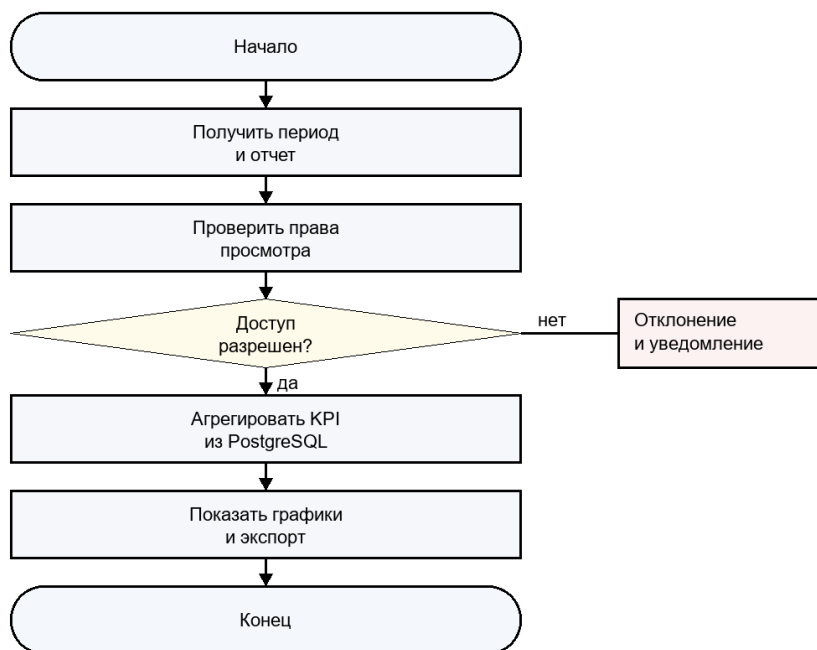


Рисунок 21 - Блок-схема алгоритма формирования аналитического отчета

2.5 Проектирование клиентской части

Клиентская часть Пульс построена как одностраничное приложение. Основная навигация размещена в боковом меню, а верхняя панель содержит элементы профиля, уведомлений и быстрого доступа. Такой интерфейс подходит для регулярной работы координаторов и администраторов, поскольку основные разделы доступны из единого рабочего пространства.

Страницы приложения соответствуют предметным модулям: DashboardPage, EventsPage, EventDetailPage, TasksPage, TaskDetailPage, VolunteersPage, OrganizationsPage, TimeEntriesPage, AnalyticsPage, AchievementsPage, CertificatesPage, KnowledgeBasePage, NotificationsPage и AdminPanelPage. Для обмена с сервером используются отдельные API-слои внутри entities, что уменьшает дублирование запросов в компонентах.

Формы ввода реализуются через переиспользуемые компоненты. Для списков используются постраничная навигация, фильтры и поиск. Для ролей с

ограниченным доступом интерфейс скрывает недоступные действия, однако окончательная проверка прав остается на сервере. Это важно, поскольку клиентская проверка улучшает удобство, но не является механизмом безопасности.

Особое внимание уделено адаптивности. Система должна быть доступна не только с рабочего компьютера координатора, но и с ноутбука или мобильного устройства волонтера. Поэтому интерфейс не опирается на сложные таблицы там, где пользователю достаточно карточек, списков и компактных форм.

Для состояния авторизации используется централизованное хранилище. Это позволяет не передавать данные текущего пользователя вручную между страницами и единообразно реагировать на истечение сессии. При загрузке приложения выполняется проверка текущего пользователя, после чего маршрутизатор определяет доступность разделов.

API-клиенты в папке сущностей отвечают за конкретные запросы к backend. Такой подход делает компоненты интерфейса проще: страница вызывает функцию, передает параметры и получает типизированный результат. При изменении endpoint достаточно обновить соответствующий API-модуль.

Для страниц с большим количеством записей используются фильтры и пагинация. Например, список задач можно ограничить по статусу, организации или поисковой строке. Это важно для координаторов, которые ежедневно работают с оперативными списками и должны быстро находить нужную запись.

Визуальный стиль интерфейса выбран рабочим и сдержанным. Для системы управления важнее читаемость, предсказуемая навигация и плотность полезной информации, чем декоративность. Основной экран должен помогать пользователю принять решение и выполнить действие за минимальное число шагов.

2.6 Реализация модулей аналитики, геймификации и сертификатов

Модуль аналитики предназначен для получения управленческой информации. Он агрегирует данные по волонтерам, мероприятиям, задачам, подтвержденным часам, начисленным достижениям и действиям пользователей. Наличие аналитики позволяет руководителю организации видеть не только отдельные записи, но и динамику активности.

Геймификация реализуется через достижения, баллы и лидерборд. Система может начислять достижения за участие в мероприятиях, выполнение задач, накопление часов и регулярную активность. При этом геймификация рассматривается как вспомогательный механизм обратной связи, а не как замена содержательной мотивации волонтера.

Модуль сертификатов формирует документы, подтверждающие участие и вклад пользователя. Сертификат связывается с организацией и пользователем, имеет код проверки и может быть скачан в PDF. Публичная проверка по коду позволяет внешнему получателю убедиться, что документ действительно был сформирован системой.

Связка аналитики, геймификации и сертификатов повышает ценность системы. Волонтер получает видимый результат участия, координатор получает инструмент подтверждения вклада, а организация получает основание для отчетности и принятия управленческих решений.

Аналитика строится на уже накопленных операционных данных, поэтому ее точность зависит от корректности процессов подтверждения. Неподтвержденные часы не должны попадать в итоговые показатели наравне с подтвержденными. Это правило делает отчеты надежнее и снижает риск завышения активности.

Достижения проектируются как расширяемый справочник правил. В базовой версии достаточно достижений за количество часов, участие в мероприятиях и выполнение задач. В дальнейшем правила могут учитывать регулярность участия, направления волонтерства, качество обратной связи или работу в команде.

Сертификаты и справки являются важной частью практической ценности системы. Для студента или волонтера документ подтверждает вклад, а для организации упрощает оформление результатов. Проверочный код снижает риск подделки документа и позволяет внешнему получателю выполнить самостоятельную проверку.

Таблица 11 - Реализованные модули Пульс

Модуль	Результат реализации
Авторизация	Регистрация, вход, refresh-сессии, выход, восстановление и смена пароля
Организации	Управление организациями, участниками, ролями, логотипом и данными
Мероприятия	Создание, редактирование, заявки, статусы, смены, завершение, посещаемость
Задачи	Создание задач, назначения, статусы, комментарии, вложения, история изменений
Учет времени	Создание записей, подтверждение и отклонение часов, связь с задачами и мероприятиями
База знаний	Статьи, категории, публикация, редактирование и поиск материалов
Уведомления	In-app уведомления о событиях и изменениях статусов
Сертификаты	Генерация, скачивание и публичная проверка документов
Админ-панель	Постраничное управление системными сущностями для суперадминистратора

2.6.1 Описание предметных модулей

Модуль авторизации обеспечивает начальную точку входа в систему. Он отвечает за регистрацию, вход, обновление сессии, выход, восстановление пароля, подтверждение email и смену пароля. Для остальных модулей он предоставляет сведения о текущем пользователе и его правах.

Модуль пользователей и волонтерских профилей хранит сведения, необходимые для работы с участником. В профиле фиксируются контактные данные, навыки, интересы, история участия, подтвержденные часы и достижения. Разделение учетной записи и профиля позволяет не смешивать технические данные входа с предметной информацией.

Модуль организаций реализует пространство, внутри которого работают участники. Организация объединяет волонтеров, координаторов и

администраторов, а также является владельцем мероприятий, задач, новостей, сертификатов и аналитических показателей. Такая модель позволяет развивать систему в сторону нескольких независимых клиентов.

Модуль мероприятий поддерживает создание событий, публикацию, подачу заявок, рассмотрение заявок, учет смен, отметку посещаемости и завершение. Он является одним из центральных модулей, потому что через мероприятия проходит значительная часть волонтерской активности.

Модуль задач предназначен для распределения поручений между участниками. Задача имеет приоритет, срок, статус, исполнителей, комментарии, вложения и историю изменений. Это позволяет использовать систему не только для мероприятий, но и для постоянной операционной работы организации.

Модуль учета времени связывает вклад волонтера с конкретным основанием: мероприятием, задачей или иной деятельностью внутри организации. Подтверждение часов координатором отделяет заявленные сведения от проверенных, что повышает достоверность отчетности.

Модуль уведомлений информирует пользователя о значимых изменениях: статусе заявки, назначении задачи, подтверждении часов, появлении новых материалов или административных действиях. Наличие уведомлений снижает зависимость процессов от внешних мессенджеров.

Модуль базы знаний позволяет организации хранить инструкции, правила участия, ответы на частые вопросы и обучающие материалы. Для новых волонтеров это снижает порог входа, а для координаторов уменьшает количество повторяющихся объяснений.

Модуль аналитики формирует управленческие отчеты на основе операционных данных. Он помогает оценить активность волонтеров, эффективность мероприятий, выполнение задач, начисление достижений и действия пользователей. Аналитика делает систему полезной для руководителя, а не только для координатора.

Модуль сертификатов завершает цикл подтверждения вклада. Документ формируется на основе проверенных данных, связывается с пользователем и организацией, получает код проверки и может быть передан внешнему получателю. Это повышает практическую ценность системы для образовательной и социальной среды.

2.6.2 Обеспечение безопасности и целостности данных

Безопасность Пульс рассматривается на нескольких уровнях: защита учетных записей, разграничение доступа, валидация входных данных, контроль операций изменения и защита персональных сведений. Для системы, работающей с данными волонтеров, эти вопросы имеют практическое значение, поскольку в профилях могут храниться контакты, история участия и сведения о подтвержденных часах.

Пароли пользователей не должны храниться в открытом виде. При регистрации и смене пароля сервер сохраняет только криптографический хэш. При входе пользователь передает пароль по защищенному соединению, сервер сравнивает его с хэшем и выдает сессию только при успешной проверке. Такой подход снижает последствия возможной компрометации базы данных.

Разграничение доступа строится на проверке роли и принадлежности пользователя к организации. Это особенно важно для многоорганизационной модели. Пользователь может быть координатором в одной организации и обычным волонтером в другой, поэтому нельзя опираться только на глобальный признак роли. Каждая операция должна проверять контекст организации.

Валидация данных выполняется как на клиенте, так и на сервере. Клиентская валидация помогает пользователю быстрее исправить ошибку в форме, но не считается достаточной для защиты. Сервер повторно проверяет обязательные поля, допустимые значения статусов, формат дат, ограничения по лимитам участников и права пользователя на изменение записи.

Для сохранения целостности данных важно ограничивать переходы между статусами. Например, отмененное мероприятие не должно принимать

новые заявки, завершённое мероприятие не должно произвольно менять дату проведения, а подтвержденные часы должны изменяться только пользователем с соответствующими правами. Явные правила переходов делают поведение системы предсказуемым.

Журнал аудита фиксирует значимые операции: создание, изменение и удаление записей, изменение статусов, подтверждение часов, управление ролями. Наличие аудита повышает доверие к системе и помогает разбирать спорные ситуации. Если данные были изменены ошибочно, организация может установить, какое действие привело к проблеме.

2.6.3 Организация пользовательского интерфейса

Интерфейс Пульс проектировался как рабочее пространство, а не как рекламная страница. Основная задача пользователя - быстро перейти к нужному разделу, увидеть актуальные данные и выполнить действие. Поэтому навигация строится вокруг постоянного бокового меню, а страницы используют повторяемые элементы: заголовок, фильтры, список, карточку деталей и форму.

Для волонтера важны простые сценарии: увидеть доступные мероприятия, подать заявку, посмотреть назначенные задачи, добавить часы и получить уведомление о решении координатора. Эти действия не должны требовать знания внутренней структуры организации. Поэтому интерфейс волонтера скрывает административные детали и показывает только необходимые статусы.

Для координатора интерфейс должен быть более плотным. Ему необходимо работать со списками заявок, участников, задач и часов. Поэтому используются таблицы, фильтры, постраничная навигация и быстрые действия. При этом формы должны оставаться понятными, чтобы координатор мог создавать мероприятие или задачу без обращения к технической документации.

Для администратора организации важны настройки и контроль. На страницах организации отображаются участники, роли, основные сведения, логотип и управленческие действия. Такая структура позволяет разделить

повседневную операционную работу координатора и более редкие административные задачи.

Суперадминистраторская панель реализует универсальное управление сущностями. Она необходима не для ежедневной работы волонтерской организации, а для технической поддержки приложения, исправления данных и контроля системных справочников. Доступ к ней должен быть максимально ограничен.

Адаптивность интерфейса учитывает разные устройства. Координатор чаще работает за компьютером, но волонтер может использовать телефон. Поэтому ключевые пользовательские действия должны корректно отображаться на узких экранах, а сложные административные таблицы должны сохранять читаемость на рабочем экране.

Навигационные переходы и распределение рабочих экранов по ролям отражены на рисунке 22. Карта экранов показывает не список компонентов, а достижимый путь пользователя от входа к подтвержденному результату.

Карта экранов по ролям

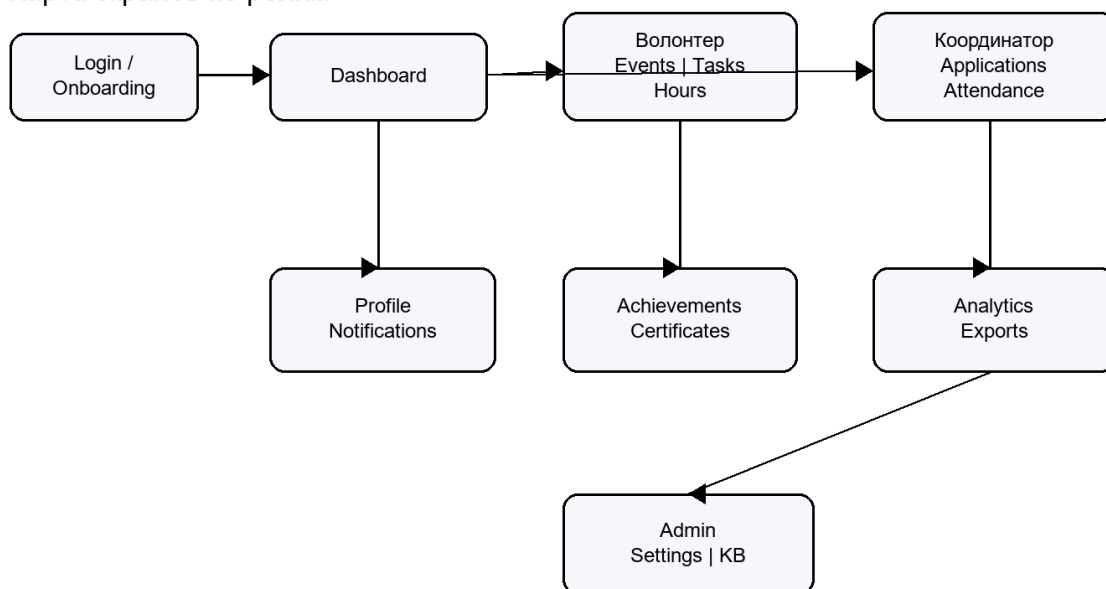


Рисунок 22 - Карта экранов Пульс с разделением ролей

Таблица 12 - Карта основных экранов клиентской части

Экран	Назначение	Основные действия
LoginPage	Вход пользователя в систему	Ввод email и пароля, переход к восстановлению доступа
OnboardingPage	Первичное знакомство и регистрационный сценарий	Регистрация, выбор начального действия
DashboardPage	Сводная рабочая панель	Просмотр ключевых показателей и быстрых переходов
EventsPage	Список мероприятий	Фильтрация, просмотр, создание мероприятия координатором
EventDetailPage	Карточка мероприятия	Подача заявки, рассмотрение заявок, отметка посещаемости
TasksPage	Список задач	Фильтрация, создание, назначение и изменение статуса
TaskDetailPage	Карточка задачи	Просмотр описания, комментарии, вложения, учет времени
TimeEntriesPage	Учет волонтерских часов	Создание, редактирование, подтверждение и отклонение записей
VolunteersPage	Список волонтеров организации	Поиск, фильтрация, просмотр профиля и активности
OrganizationsPage	Список организаций	Просмотр доступных организаций и создание новой при наличии прав
OrganizationDetailPage	Карточка организации	Управление участниками, ролями, логотипом и настройками
AnalyticsPage	Аналитика организации	Просмотр отчетов по активности, мероприятиям, задачам и часам
AchievementsPage	Достижения и рейтинг	Просмотр личных достижений и лидерборда
CertificatesPage	Сертификаты и справки	Генерация, скачивание и просмотр документов
CertificateVerifyPage	Публичная проверка сертификата	Проверка документа по коду без входа в систему
KnowledgeBasePage	База знаний	Просмотр статей, поиск и переход к материалам
KnowledgeFormPage	Форма статьи базы знаний	Создание и редактирование материала
NewsListPage	Новости организации	Просмотр объявлений и публикаций
NotificationsPage	Уведомления пользователя	Просмотр событий и отметка уведомлений прочитанными

Экран	Назначение	Основные действия
AdminPanelPage	Системное администрирование	Постраничное управление разрешенными сущностями

До фиксации итогового интерфейса макеты определяют состав рабочих областей и результат действия пользователя. Макеты форм входа, мероприятия и учета часов показаны на рисунке 23; они задают входные данные для ключевого сценария.

Макеты пользовательских форм

The image displays three vertical rectangular wireframe forms, each representing a different user interface component. The first form on the left is for login, containing labels for 'Вход' (Login), 'Email' with an input field, 'Пароль' (Password) with an input field, and a '[Войти]' (Login) button. The middle form is for events, with labels for 'Мероприятие' (Event), 'Название и дата' (Name and date), 'Описание' (Description), 'Статус заявки' (Application status), and a '[Подать заявку]' (Submit application) button. The third form on the right is for time accounting, featuring labels for 'Учет часов' (Time accounting), 'Основание [мероприятие]' (Basis [event]), 'Часы [5]' (Hours [5]), and an '[Отправить]' (Send) button.

Рисунок 23 - Макеты пользовательских форм Пульс

Макет аналитической визуализации на рисунке 24 соответствует реализованным возможностям `AnalyticsPage.vue`: показатели, графики, проблемные зоны, фильтры и экспорт.

Макет визуализации итогов работы системы

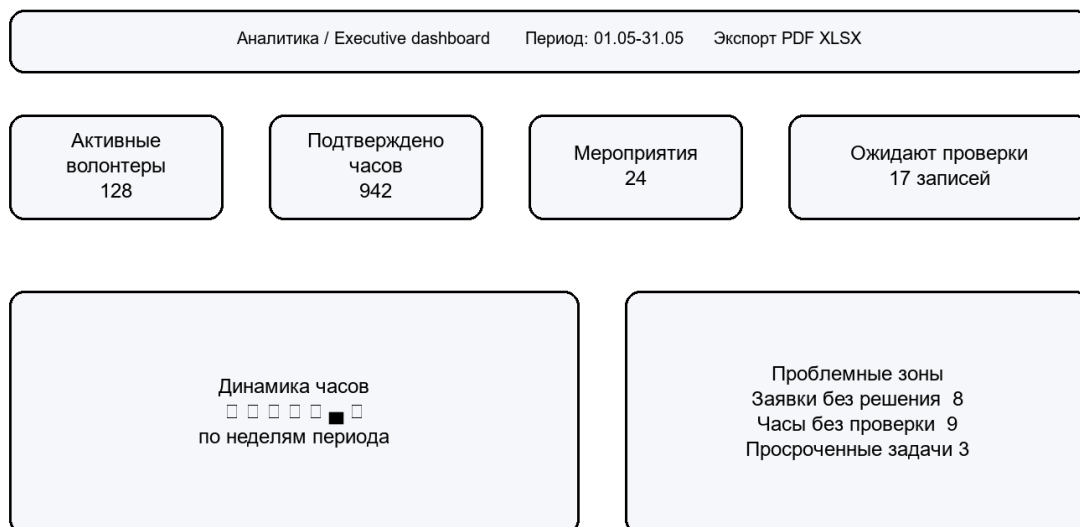


Рисунок 24 - Макет визуализации итогов работы Пульс

Таблица 13 - Макеты пользовательских форм и ожидаемые результаты

Форма/экран	Ключевые элементы макета	Результат действия
Вход и регистрация	Email, пароль, восстановление доступа	Авторизованная сессия
Карточка мероприятия	Описание, дата, лимит, кнопка заявки, статус	Созданная заявка или просмотр решения
Панель координатора	Заявки, фильтр статуса, массовое действие	Подтверждение участников
Учет часов	Основание, количество часов, подтверждение	Approved time entry
Аналитика	KPI, период, графики, риски, экспорт	Проверяемый отчет за период
Сертификат	Пользователь, часы, код проверки	PDF и публичный verify_code

Модули новостей, уведомлений и базы знаний требуют управляемого наполнения. Контент-план в таблице 14 не заявляет новый функционал: он использует фактически существующие маршруты `/news`, `/notifications` и `/knowledge-base` и описывает организационное применение реализованных возможностей.

Таблица 14 - Контент-план коммуникаций в Пульс

Контент	Аудитория и триггер	Канал/ответственный	Показатель
Анонс мероприятия	Волонтеры; после публикации события	Новости; координатор	Количество заявок
Решение по заявке	Заявитель; изменение статуса	In-app уведомление; система	Доставка статуса
Подтверждение часов	Волонтер; проверка записи	In-app уведомление; система	Часы в профиле
Правила участия	Новые волонтеры; постоянно	База знаний; администратор	Доступность инструкции
Напоминание о проверке	Координатор; ожидающие часы	Уведомление; организация	Снижение pending записей

2.6.4 Расширенный профиль и настраиваемые поля участника

В версии V4 профиль участника переработан из статической карточки в рабочее пространство с вкладками «Личная информация», «Задачи», «Работа», «Мероприятия» и «Достижения и документы». Вкладки используют уже существующие предметные сущности задач, событий, геймификации и сертификатов, не заявляя несуществующих процессов.

Пользователь может загрузить собственный аватар. Клиент передает изображение в endpoint `/files/profile-avatars`, сервер проверяет расширение, фактический MIME-тип и ограничение 5 МБ, сохраняет файл в каталоге uploads и возвращает URL. После сохранения профиля обновленный адрес изображения используется в карточке и верхней панели.

Для администратора организации реализован конструктор сведений профиля. Базовые группы «Личная информация», «Контакты», «Работа» и «Результаты» создаются автоматически и помечены как системные: API отклоняет их удаление. Администратор может добавлять собственные группы и поля, выбирать тип данных, обязательность, доступность для пользователя и варианты выбора.

Физическая модель расширена таблицами `profile\_field\_groups`, `profile\_field\_definitions`, `profile\_field\_options` и `profile\_field\_values`. Каждая запись привязана к организации; поэтому состав полей одной организации не

раскрывается другой. Изменяющие запросы проходят через существующий audit middleware, а доступ проверяется серверным authorizer.

На рисунке 25 показан реализованный экран настройки: защищенные системные группы свернуты, а пользовательская группа «Подготовка волонтера» содержит активное поле «Готовность к выездам». На рисунке 26 приведено отображение этого поля во вкладке «Работа» участника с сохраненным значением.

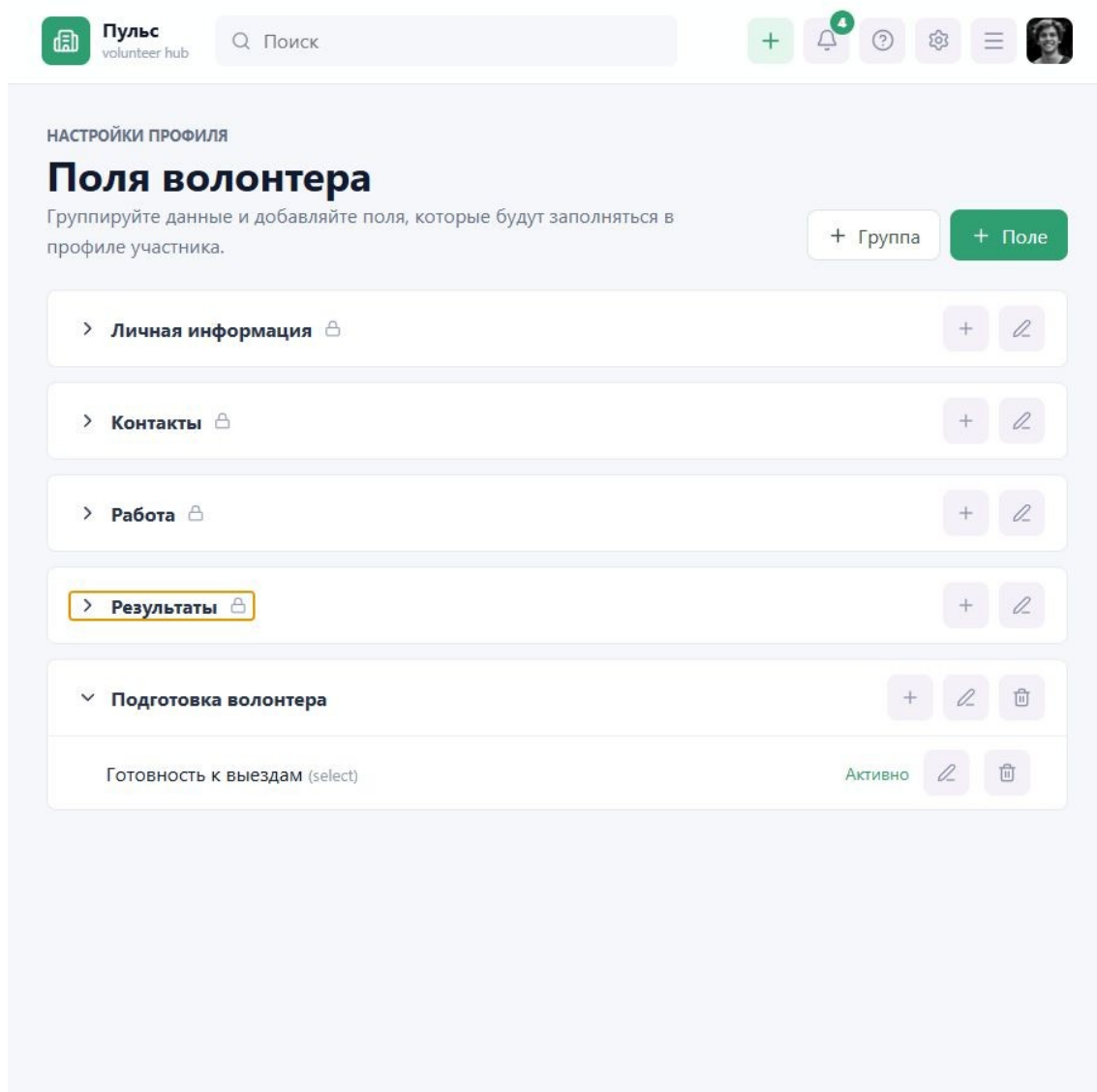


Рисунок 25 - Конструктор групп и полей профиля в системе Пульс

Путь волонтера

Поиск

Морозов Илья
volunteer1@test.local

Организация
Добрые руки

Подтвержденные часы
18.5 ч.

Баллы
200

Работа и опыт

Город
Екатеринбург

Статус
Активный

Опыт и описание
Готов помогать на городских событиях и социальных акциях.

Интересы

Сохранить

Подготовка волонтера

Готовность к выездам*
Готов(а)

Сохранить дополнительные данные

Рисунок 26 - Вкладка работы волонтера с настраиваемым полем

2.6.5 Развертывание, CI/CD и сопровождение

В ходе доработки V4 в репозиторий добавлен GitHub Actions workflow ``.github/workflows/quality.yml``. Он выполняет три независимых этапа контроля: backend verification, frontend build и container smoke test. Такой pipeline превращает правила сборки из текста документа в воспроизводимую проверку каждого push и pull request.

Backend job устанавливает версию Go из ``go.mod``, загружает зависимости и выполняет ``go vet ./...``, ``go test ./...`` и ``go build ./...``. Frontend job использует Node.js 22, команду ``npm ci`` и ``npm run build``, включающую TypeScript-проверку. Container job после успешных сборок валидирует Docker Compose,

собирает сервисы, запускает приложение и обращается к health endpoint и frontend.

Для локального запуска проекта используется Docker Compose. Он объединяет backend, frontend, базу данных и сопутствующие сервисы в единый сценарий запуска. Это снижает количество ручных действий и помогает воспроизводить окружение на другой машине.

Миграции базы данных позволяют пошагово изменять структуру таблиц. Это важно для проекта, который развивается итерационно: при добавлении сертификатов, уведомлений или новых связей не требуется вручную изменять схему базы. Миграции фиксируют историю изменений и позволяют развернуть проект с нуля.

Для production-развертывания предусмотрены отдельные Dockerfile и конфигурация прокси. Такой подход отделяет режим разработки от режима эксплуатации. В разработке важны быстрые перезапуски и удобство отладки, а в эксплуатации - стабильная сборка, предсказуемые переменные окружения и корректная маршрутизация запросов.

Сопровождение системы предполагает резервное копирование базы данных, контроль логов, обновление зависимостей и проверку доступности сервиса. Даже если в рамках дипломной работы эти процессы описаны на уровне проектного решения, их учет показывает готовность архитектуры к реальному использованию.

Важным направлением сопровождения является управление клиентскими экземплярами. Пульс может развиваться как коробочная система для отдельной организации или как SaaS-платформа. В обоих случаях необходимы процедуры создания организации, настройки домена, загрузки логотипа, выдачи начальных ролей и проверки работоспособности.

Таблица 15 - Эксплуатационные требования к системе

Требование	Описание
Резервное копирование	Регулярное сохранение базы данных и файловых вложений
Логирование	Фиксация ошибок приложения и значимых действий пользователей
Мониторинг	Контроль доступности API, базы данных и фоновых задач
Обновление	Применение миграций и поставка новых версий без потери данных
Настройка клиента	Создание организации, назначение ролей, загрузка логотипа и базовых справочников

Разработанные проектные решения, роли, данные, интерфейсы, настраиваемый профиль и требования эксплуатации систематизированы в таблицах 7-15 и на рисунках 25-26.

2.7 Выводы по второму разделу

Сформирована концептуальная модель программного продукта, связывающая роли, предметные операции, контроль достоверности и итоговые результаты: подтвержденный вклад, документ и отчет.

Обоснована и представлена архитектура web-приложения с клиентской частью на Vue.js и TypeScript, серверным API на Go, хранением в PostgreSQL и модулями аудита, аналитики и сертификатов.

Разработаны визуальная структура данных, информационно-логическая ER-модель и физическая модель ключевых таблиц, которые проверены по миграциям проекта.

Сформированы алгоритмические схемы обработки заявок, часов, достижений, сертификатов и аналитики; их ветвления соответствуют правилам статусов и ограничениям данных.

Разработана карта экранов по пользовательским ролям и описаны макеты рабочих форм, связывающие каждое действие с получаемым результатом.

Для итогов работы продукта разработан макет аналитической панели; его показатели и экспорт соответствуют реализованному компоненту AnalyticsPage и backend-модулю analytics.

Реализованы вкладочный профиль, загрузка аватара и администрируемые поля участника с хранением значений в организации и серверной защитой системных полей.

Добавлен CI/CD pipeline GitHub Actions, автоматически проверяющий backend, frontend и контейнерный запуск приложения.

Составлен контент-план для существующих модулей новостей, уведомлений и базы знаний, определяющий их прикладное использование в деятельности организации.

Таким образом, проектные результаты второй главы закрывают выделенные требования методического пособия и образуют основу для проверяемой эксплуатации, описанной в третьей главе.

3 ПРОВЕРКА РАБОТОСПОСОБНОСТИ И ОЦЕНКА РЕЗУЛЬТАТОВ

3.1 Организация испытаний

Проверка работоспособности Пульс выполнялась по основным пользовательским сценариям. Цель испытаний состояла в подтверждении того, что реализованные модули корректно взаимодействуют друг с другом и обеспечивают полный цикл работы: от регистрации пользователя до формирования отчетных данных и сертификата.

Backend проверяется запуском тестов, применением миграций на чистой базе и ручной проверкой API. Frontend проверяется сборкой проекта, переходами по основным страницам, заполнением форм и выполнением операций создания, редактирования и удаления сущностей. Отдельно проверяются права доступа, поскольку ошибки разграничения ролей могут привести к раскрытию или изменению чужих данных.

Испытания целесообразно проводить на подготовленном демонстрационном наборе данных. В него входят несколько пользователей с разными ролями, одна или несколько организаций, мероприятия в разных статусах, задачи, записи времени, достижения и сертификаты. Такой набор позволяет быстро проверить не только пустые формы, но и реальные списки.

Проверка backend начинается с миграций, потому что ошибки схемы базы данных проявляются во всех остальных модулях. После успешного применения миграций проверяются базовые endpoint авторизации, затем CRUD-операции и сценарии, где несколько сущностей взаимодействуют между собой.

Проверка frontend включает не только сборку, но и ручной проход по интерфейсу. Важно убедиться, что пользователь видит только допустимые пункты меню, формы не отправляют некорректные данные, ошибки API отображаются понятным образом, а после успешного действия список или карточка обновляются.

Таблица 16 - Основные проверки системы

Направление	Проверяемые действия	Ожидаемый результат
Авторизация	Регистрация, вход, refresh, logout, смена пароля	Пользователь получает доступ только после успешной проверки учетных данных
Организации	Создание организации, добавление участника, назначение роли	Данные доступны в пределах организации и с учетом роли
Мероприятия	Создание мероприятия, подача и рассмотрение заявки, завершение	Статусы заявок и мероприятия изменяются корректно
Часы	Создание записи, подтверждение, отклонение	Подтвержденные часы учитываются в профиле и отчетах
Достижения	Начисление достижений и баллов после активности	Пользователь видит достижения и место в рейтинге
Сертификаты	Генерация, скачивание, публичная проверка	Документ имеет проверочный код и доступен для проверки
Аналитика	Открытие отчетов и дашборда	Отображаются агрегированные показатели по доступным данным

В ходе подготовки настоящей редакции были фактически запущены доступные автоматизированные команды проверки. Результаты приведены в таблице 17; они подтверждают компилируемость backend и успешную production-сборку frontend, но не подменяют сценарные испытания с подключенной базой данных.

Таблица 17 - Фактически выполненные проверки сборки от 26.05.2026

Команда	Фактический результат	Интерпретация
go test ./...	Успешно; все пакеты скомпилированы, для пакетов выведено [no test files]	Backend собирается, но автоматические unit/integration tests в репозитории отсутствуют
npm run build	Успешно; Vite обработал 1782 модуля, создан production bundle	Frontend проходит TypeScript-проверку и сборку
Миграция 020 и API профиля	Успешно на запущенной PostgreSQL: группы, поле, значение и запрет удаления системного поля	Подтвержден реализованный контур настраиваемого профиля
Загрузка аватара	Успешно; endpoint вернул URL PNG в /uploads/avatars	Подтвержден файловый сценарий профиля

Для V4 фактически выполнен дополнительный сценарий нового функционала. В запущенном backend применена миграция

`020\_profile\_fields.sql`; API вернул четыре системные группы, администратор создал группу «Подготовка волонтера» и поле «Готовность к выездам», волонтер сохранил значение «Готов(а)». Попытка удалить системное поле через API получила HTTP 409, что подтверждает защиту базовой схемы. Загрузка PNG через `/files/profile-avatars` вернула URL сохраненного изображения.

Снимки реализованных состояний приведены на рисунках 25 и 26 и повторно включены в приложение Д. Верифицированный сценарий расширяет доказательства V3: он подтверждает не только наличие маршрутов в исходном коде, но и работу миграции, разграничения удаления, записи пользовательского значения и загрузки файла на запущенном приложении.

3.2 Демонстрационный сценарий работы системы

Демонстрационный сценарий отражает типовую работу волонтерской организации. Сначала суперадминистратор открывает систему и создает организацию либо использует уже существующую. Затем администратор организации добавляет пользователей и назначает координатора. Координатор создает мероприятие, указывает формат, дату, место, лимит участников и публикует его для волонтеров.

Волонтер входит в систему, открывает список мероприятий и подает заявку. Координатор рассматривает заявку и подтверждает участие. После проведения мероприятия координатор отмечает посещаемость и подтверждает часы. Система отражает часы в профиле пользователя, начисляет баллы и достижения, а затем позволяет сформировать сертификат.

В завершающей части сценария администратор открывает аналитический раздел и проверяет показатели активности. Суперадминистратор может открыть журнал аудита и убедиться, что операции создания, изменения и удаления были зафиксированы. Такой сценарий подтверждает связность модулей и показывает практическую применимость системы.

Сценарий также показывает, что система поддерживает разные уровни ответственности. Волонтер инициирует участие, но не может самостоятельно подтвердить часы. Координатор управляет мероприятием, но действует в

пределах своей организации. Администратор видит более широкий контур и может управлять участниками. Суперадминистратор обслуживает систему в целом.

При демонстрации важно проверять не только положительный путь, но и ограничения. Например, волонтер не должен видеть административную панель, координатор не должен изменять данные другой организации, а неподтвержденная запись времени не должна увеличивать итоговые часы в сертификате.

Такая проверка позволяет выявить ошибки, которые не всегда обнаруживаются при изолированном тестировании отдельных endpoint. В реальной системе ценность создается именно связкой модулей, поэтому полный пользовательский сценарий является обязательной частью приемки.

Таблица 18 - Демонстрационные исходные данные сценария

Объект	Тестовое состояние	Проверяемое изменение
Организация	Волонтерский центр Пульс Demo	Доступ координатора и волонтера по ролям
Мероприятие	Помощь на городском событии, published, лимит 30	Заявка и решение координатора
Волонтер	Профиль с доступом к мероприятиям	Часы, достижение и сертификат
Запись времени	5 часов по завершеному мероприятию	Переход pending -> approved
Аналитика	Период, включающий событие	Появление часов, посещения и активности в KPI

Таблица 19 - Сценарии пользователя по дереву целей

Цель/роль	Последовательность действий	Ожидаемое доказательство
Участие; волонтер	Открыть мероприятие -> подать заявку -> просмотреть статус	Заявка связана с событием и пользователем
Проверка вклада; координатор	Рассмотреть заявку -> отметить участие -> подтвердить часы	Approved time entry и журнал операции
Признание; координатор/волонтер	Сформировать сертификат -> проверить по коду	Документ с уникальным verify_code
Оценка; руководитель	Открыть Analytics -> выбрать период -> экспортировать отчет	KPI только по подтвержденным данным

1) суперадминистратор входит в систему и проверяет список организаций;

- 2) администратор организации добавляет участника и назначает координатора;
- 3) координатор создает мероприятие и публикует его;
- 4) волонтер подает заявку на участие;
- 5) координатор подтверждает заявку и отмечает посещаемость;
- 6) волонтер или координатор добавляет запись времени;
- 7) координатор подтверждает часы;
- 8) система начисляет баллы и достижения;
- 9) координатор формирует сертификат;
- 10) проверяющий открывает публичную страницу проверки сертификата.

3.3 Анализ результатов разработки

В результате выполнения работы получена информационная система, реализующая основные процессы управления волонтерской деятельностью. Разработка охватывает как операционные сценарии координатора, так и пользовательские сценарии волонтера. Наличие административной панели и аудита делает систему пригодной для использования не только как демонстрационного учебного проекта, но и как основы для практического внедрения.

К сильным сторонам реализации относятся модульная структура backend, разделение API-слоя и интерфейса, использование PostgreSQL для связанных данных, наличие миграций, поддержка Docker Compose, а также широкий набор предметных модулей. Система не ограничивается регистрацией на мероприятия, а поддерживает задачи, часы, достижения, уведомления, базу знаний, сертификаты и аналитику.

Ограничения текущей версии связаны с тем, что проект выполнялся в рамках выпускной квалификационной работы. Часть перспективных функций может быть расширена: интеграции с календарями и мессенджерами, расширенная настройка правил геймификации, полнотекстовый поиск, импорт больших списков волонтеров, развитая система шаблонов документов и мониторинг production-окружения.

С точки зрения архитектуры результат можно считать расширяемым. Новые модули могут добавляться через отдельные группы маршрутов backend, новые таблицы миграций и новые страницы frontend. При этом уже существующая модель организаций и ролей позволяет сохранять общий принцип разграничения доступа.

С точки зрения предметной области результат закрывает базовый управленческий цикл. Организация получает учет участников, мероприятий, задач и часов; волонтер получает личную историю и достижения; координатор получает инструменты ежедневной работы; руководитель получает аналитику и документы.

С точки зрения учебной ценности проект демонстрирует применение нескольких компетенций: анализ предметной области, проектирование базы данных, разработка backend и frontend, работа с контейнерным окружением, проектирование интерфейса и подготовка документации.

Таблица 20 - Соответствие задач работы полученным результатам

Задача	Полученный результат
Анализ предметной области	Выявлены роли, процессы, проблемы ручного учета и требования к системе
Проектирование архитектуры	Определена трехзвенная web-архитектура с REST API и PostgreSQL
Проектирование базы данных	Выделены сущности пользователей, организаций, мероприятий, задач, часов, достижений и сертификатов
Разработка backend	Реализованы модульные обработчики, сервисы и репозитории на Go
Разработка frontend	Созданы страницы и компоненты на Vue.js для основных пользовательских сценариев
Проверка работоспособности	Сформирована программа испытаний и проверены ключевые сценарии

3.4 Квантификация пользовательского интерфейса и метрики проверки

Для оценки трудоемкости типовых действий выполнена квантификация интерфейса на уровне проектного сценария. Количество действий определяется по реализованным страницам и маршрутам; время является расчетной оценкой

для последовательного ввода без сетевой задержки и должно уточняться при очной пользовательской проверке.

Таблица 21 - Квантификация основных пользовательских сценариев

Сценарий	Экранные переходы	Действия ввода/команд	Расчетная оценка времени
Волонтер подает заявку	Login -> Events -> EventDetail	Вход, выбор события, кнопка заявки: 5 действий	до 35 с
Координатор подтверждает часы	Login -> TimeEntries -> запись	Фильтр, просмотр основания, подтверждение: 6 действий	до 45 с
Руководитель получает KPI	Login -> Analytics -> report	Выбор отчета, периода, экспорт: 6 действий	до 50 с

Метрики автоматической проверки включают компиляцию backend, сборку frontend и сценарную проверку нового модуля профиля на запущенной PostgreSQL. Автоматизированных unit-тестов backend пока нет: `'go test ./...'` выводит `'[no test files]'`. Для последующих изменений создан workflow GitHub Actions, а программа полного приемочного цикла приведена в приложении Б.

3.5 Экономическая и практическая значимость

Практическая значимость Пульс заключается в снижении трудозатрат координаторов на учет заявок, часов и отчетов. Если организация ведет процессы вручную, сотрудник регулярно тратит время на перенос данных между таблицами, сверку списков, подготовку справок и поиск истории участия. Единая система уменьшает число ручных операций и снижает риск ошибок.

Экономический эффект для небольшой организации выражается прежде всего в экономии рабочего времени. Например, автоматическое формирование списков участников, подтверждение часов и подготовка сертификатов позволяют координатору уделять больше внимания содержательной работе с добровольцами. Для образовательных организаций это также повышает прозрачность учета внеучебной активности студентов.

С точки зрения дальнейшего развития Пульс может быть оформлена как коробочное решение для одной организации или как SaaS-платформа с

изолированными пространствами организаций. Уже реализованные сущности организаций, членства и ролей создают основу для масштабирования продукта. Дополнительную ценность могут дать настройки бренда, публичные страницы организаций, конструктор процессов согласования и расширенные отчеты.

Для оценки экономической целесообразности можно рассматривать сокращение времени на повторяющиеся операции. Если координатор еженедельно вручную сводит заявки, посещаемость и часы, автоматизация даже части этих действий дает заметную экономию. При увеличении числа мероприятий эффект растет, потому что система повторно использует уже введенные данные.

Косвенный эффект выражается в повышении качества данных. Когда сведения вводятся один раз и используются в связанных модулях, уменьшается число расхождений между списками. Это особенно важно для отчетности, где ошибка в часах или участниках может привести к необходимости повторной проверки документов.

Практическая ценность также связана с удержанием волонтеров. Прозрачная история участия, достижения, уведомления и сертификаты помогают участнику видеть результат своей работы. Для организации это означает более устойчивое сообщество и меньшие затраты на постоянное привлечение новых людей.

Таблица 22 - Оценка сокращения ручных операций координатора

Операция	Ручной подход	Подход с Пульс
Сбор заявок	Форма, таблица и ручное уведомление участников	Заявка создается в системе и получает статус
Подтверждение участия	Сверка списка и переписка с каждым участником	Координатор меняет статус заявки в карточке мероприятия
Учет часов	Ручной подсчет по посещаемости и отдельным сообщениям	Записи времени связаны с мероприятием или задачами
Подготовка сертификата	Ручное заполнение шаблона и проверка данных	Документ формируется по подтвержденным данным
Отчетность	Сведение нескольких таблиц за период	Показатели строятся на операционных данных системы

Даже без точного коммерческого расчета видно, что основной экономический эффект связан с повторяемостью операций. Чем больше мероприятий проводит организация, тем чаще координатор выполняет однотипные действия. Автоматизация таких действий дает накопительный эффект и уменьшает зависимость качества учета от внимательности одного сотрудника.

Дополнительная экономическая ценность связана с подготовкой документов. Если справки и сертификаты формируются вручную, сотрудник должен проверить ФИО, организацию, даты, количество часов и основание выдачи. В Пульс эти данные уже связаны с пользователем, мероприятием и подтвержденными часами, поэтому риск ошибки ниже.

Для студенческого волонтерства важен не только прямой экономический эффект, но и управленческая прозрачность. Университет или факультет может видеть активность по группам, направлениям и периодам, а студент получает подтверждение своего участия. Это повышает качество внеучебной работы и облегчает подготовку отчетности.

Для благотворительных организаций система может быть полезна при подготовке отчетов перед партнерами. Наличие истории мероприятий, подтвержденных часов и сертификатов позволяет быстрее отвечать на вопросы о результатах программы и демонстрировать вклад добровольцев.

Развитие Пульс как продукта может идти поэтапно. На первом этапе система используется как локальный инструмент одной организации. На втором этапе добавляются настройки бренда, шаблоны документов и расширенная аналитика. На третьем этапе возможен переход к SaaS-модели, где каждая организация работает в собственном пространстве.

В SaaS-модели важны биллинг, изоляция данных, управление тарифами, резервное копирование и мониторинг. Эти функции не являются обязательными для дипломного MVP, но текущая модель организаций, ролей и прав доступа создает основу для их последующего добавления.

3.5.1 План опытной эксплуатации и приемочные критерии

Опытная эксплуатация Пульс должна проводиться не как демонстрация отдельных страниц, а как ограниченный по времени рабочий цикл волонтерской организации. Для этого требуется тестовая организация, пользователи с ролями волонтера, координатора и администратора, опубликованное мероприятие и заранее согласованный перечень проверяемых результатов.

Стартовая загрузка данных включает создание организации, назначение ролей, заполнение справочных сведений и регистрацию мероприятия. После этого волонтеры выполняют действия через интерфейс, а координатор обрабатывает заявки и часы в рамках собственных полномочий. Такой подход позволяет проверить не только корректность формы, но и согласованность статусов в последовательности действий.

Продолжительность пробного цикла рационально определять одной завершенной активностью: от публикации мероприятия до формирования итоговой аналитики и сертификата. Если фактическое мероприятие недоступно до защиты, испытание выполняется на демонстрационном наборе с теми же ролями и статусами, при этом в протоколе явно указывается тестовый характер данных.

В настоящей редакции документа подтверждены сборка клиентской части и компилируемость серверной части. Результаты, требующие запущенной базы данных и действий пользователей, включены в программу эксплуатации, но не объявляются выполненными без протокола запуска и сохраненных доказательств.

Критерием приемки считается не отсутствие любых замечаний, а достижение обязательного контура: заявка создается и меняет статус по полномочиям; часы учитываются только после подтверждения; итоговый документ или показатель основан на подтвержденных данных; нарушения прав доступа отклоняются; результаты доступны ответственному пользователю.

Таблица 23 - Этапы опытной эксплуатации Пульс

Этап	Действия	Фиксируемый результат
Подготовка	Развернуть окружение, применить миграции, создать роли и организацию	Версия приложения, журнал миграций, перечень учетных записей
Запуск события	Создать и опубликовать мероприятие, предоставить доступ волонтерам	Карточка события со статусом и параметрами участия
Участие	Подать и рассмотреть заявки, отметить посещаемость	История статусов заявки и список участников
Учет вклада	Внести и проверить часы, начислить результат активности	Approved time entry, баллы или достижение
Итоги	Открыть аналитику, сформировать сертификат и проверить код	KPI, экспорт или документ с проверочным кодом
Завершение	Собрать замечания и сопоставить результат с критериями	Протокол замечаний и решение о приемке

Таблица 24 - Приемочные критерии обязательного контура

Объект проверки	Критерий приемки	Способ подтверждения
Права роли	Недопустимая операция отклонена сервером	Запрос под ролью без полномочий и ответ API
Заявка	Создана один раз и имеет отслеживаемый статус	Экран заявки и запись события
Запись времени	В KPI участвуют только подтвержденные часы	Сравнение до и после утверждения записи
Сертификат	Документ имеет уникальный код проверки	Генерация и открытие публичной проверки
Аналитика	Показатели соответствуют данным выбранного периода	Dashboard, выгрузка и контрольная выборка
Аудит	Значимые изменения сохраняют пользователя и время	Просмотр записей аудита администратором

3.5.2 Протокол сценарных проверок

Сценарная проверка строится от целей пользователей, а не от списка технических маршрутов. Каждому критичному результату назначается исходное состояние, последовательность действий, ожидаемый результат и доказательство. Доказательством может выступать снимок экрана, ответ API, запись базы данных или журнал аудита.

Для текущей ВКР различаются три статуса доказательств. Статус «подтверждено запуском» присваивается только выполненным командам сборки. Статус «подтверждено реализацией» означает наличие кода, маршрута или миграции, проверенных по репозиторию. Статус «подлежит опытной

проверке» используется для сценариев, которые требуют рабочей базы данных и взаимодействия ролей.

Такое разделение исключает недостоверный вывод о выполненном испытании на основании одного лишь наличия программного кода. Реализованный endpoint показывает возможность выполнения операции, но успешность полного пользовательского сценария подтверждается только фактическим прохождением в развернутом окружении.

Негативные сценарии имеют ту же значимость, что и основной путь. Для волонтерской системы критично исключить повторную заявку, подтверждение часов без соответствующих прав, просмотр сведений другой организации и формирование итогового документа на неподтвержденных данных.

Таблица 25 - Матрица подтвержденности сценариев

Сценарий	Доступное доказательство	Статус в рамках ВКР
Сборка backend	Вывод `go test ./...`: пакеты скомпилированы, `[no test files]`	Подтверждено запуском; тестового покрытия нет
Сборка frontend	Вывод Vite build: обработано 1782 модуля	Подтверждено запуском
Хранение связанных данных	Миграции PostgreSQL и ограничения ключей	Подтверждено реализацией
Подача заявки	Маршруты events/applications и репозиторий операций	Подтверждено реализацией; требуется проход UI/API
Подтверждение часов	Маршруты time-entries и связанные ограничения	Подлежит опытной проверке
Вывод аналитики	AnalyticsPage и SQL-агрегации backend	Подтверждено реализацией; KPI требуют данных
Сертификат	Маршруты generate/verify и поле verify code	Подлежит опытной проверке

Таблица 26 - Негативные сценарии приемки

Проверка	Ожидаемое поведение	Причина обязательности
Повторная заявка на событие	Повторное создание блокируется	Исключение двойного учета участника
Подтверждение часов волонтером	Операция отклоняется по роли	Защита достоверности вклада
Просмотр чужой организации	Доступ ограничивается членством и ролью	Изоляция данных организаций
Сертификат без подтвержденных часов	Документ не выдается либо не содержит неподтвержденный вклад	Достоверность документа
Недоступный отчет	Показатели не раскрываются неподходящей роли	Защита управленческих данных
Ошибка входных данных	API возвращает контролируемую ошибку, интерфейс сообщает ее пользователю	Предсказуемость рабочего процесса

3.5.3 Контроль данных, ролей и аудита

Проверка данных начинается с физической модели. Ограничения уникальности заявок и кодов сертификатов, внешние ключи и допустимые статусы являются первой линией защиты от логических ошибок. Эти механизмы подтверждены по миграциям, однако их фактическое срабатывание должно быть зафиксировано при запуске тестовой базы.

Второй уровень контроля сосредоточен в серверной логике. Приложение должно проверять текущего пользователя и контекст организации до изменения данных. Наличие маршрута в клиентском интерфейсе не является основанием доверять запросу: пользователь может обратиться к API напрямую, поэтому решение принимается на backend.

Третий уровень связан с аудитом. Для операций, влияющих на участие, часы, роли, сертификаты и отчеты, администратору требуется возможность восстановить ход изменения данных. В опытной эксплуатации рекомендуется зафиксировать идентификаторы сценарных операций и сопоставить их с журналом аудита после завершения цикла.

Аналитические показатели также нуждаются в трассируемости. Пользователь управленческой панели должен понимать, какой период выбран и какие данные входят в итог. Проверка KPI заключается в сравнении

отображаемого значения с небольшим контрольным набором событий и подтвержденных записей времени.

Отдельно проверяется публичная проверка сертификата. Она должна выдавать ограниченный набор сведений, достаточный для подтверждения подлинности документа, но не раскрывать лишнюю информацию о профиле волонтера или внутренних данных организации.

Таблица 27 - Трассировка средств контроля в реализации

Риск	Реализованное основание контроля	Проверка при эксплуатации
Дублирование заявки	UNIQUE-связь заявки события и пользователя	Повторить подачу одной заявки
Некорректные часы	Ограничения time_entries и операция подтверждения	Создать, отклонить и утвердить записи
Повторное начисление	Ограничение транзакций баллов	Повторно инициировать одно основание награды
Подделка документа	Уникальный verify_code сертификата	Сверить действительный и случайный код
Необъяснимое изменение	Модуль audit и маршруты просмотра аудита	Проверить запись после изменения статуса
Ошибка управленческого итога	Backend analytics и экран AnalyticsPage	Сопоставить KPI с контрольными данными

3.5.4 Метрики эксплуатации и ограничения результатов

Метрики опытной эксплуатации делятся на технические, процессные и пользовательские. Технические показатели фиксируют возможность запуска и отсутствие критических отказов. Процессные характеризуют прохождение заявок, подтверждение часов и формирование результата. Пользовательские отражают трудоемкость действий и замечания участников.

Для небольшого пилота достаточно контрольного набора, который можно проверить вручную: количество созданных мероприятий, заявок по статусам, подтвержденных часов, выданных сертификатов и операций, попавших в журнал. Эти показатели не подменяют статистическое исследование, но обеспечивают достоверную проверку связности системы.

Измерение времени сценариев следует проводить на одинаковых исходных данных. Пользователь выполняет заранее заданный путь, а наблюдатель фиксирует время завершения, ошибки и возвраты на предыдущие

шаги. Указанные ранее расчетные оценки до 35-50 секунд становятся подтвержденными только после такого измерения.

Для оценки интерфейса полезно собирать замечания по понятности названий, видимости статуса, доступности следующего действия и корректности сообщений об ошибках. Исправления, выявленные в пилоте, должны фиксироваться отдельным перечнем и не маскироваться как уже реализованные свойства системы.

Текущая редакция ВКР честно ограничивает выводы: выполнены команды сборки, проанализированы миграции и исходный код, разработана программа проверки. Полный протокол опытной эксплуатации с рабочей базой и участниками требует отдельного запуска, учетных данных и сохранения доказательств.

После такого запуска третья глава может быть дополнена фактическими значениями процессных показателей, снимками экранов с тестовыми итогами, журналом обнаруженных замечаний и решением о соответствии приемочным критериям. Подготовленная структура допускает такую замену без изменения проектной части работы.

Таблица 28 - Метрики опытной эксплуатации и порядок фиксации

Метрика	Источник данных	Состояние подтверждения
Успешная сборка frontend	Лог `npm run build`	Получено 26.05.2026
Сборка backend	Лог `go test ./...`	Получено 26.05.2026; тесты отсутствуют
Применимость миграций	Лог запуска на чистой PostgreSQL	Требуется выполнения
Заявки по статусам	Событие и аналитический отчет	Требуется демонстрационных данных
Подтвержденные часы	Time entries и KPI	Требуется сценарного прохода
Сертификат и проверка кода	Экран и публичный маршрут проверки	Требуется сценарного прохода
Время сценария пользователя	Наблюдение по контрольному пути	Требуется пользовательского замера
Замечания интерфейса	Протокол участников пилота	Требуется опытной эксплуатации

Таблица 29 - Ограничения доказательной базы и способ закрытия

Ограничение	Влияние на вывод	Необходимое действие
В репозитории нет автоматизированных тестов	Сборка не доказывает бизнес-сценарии	Добавить unit/integration tests и сохранить отчет
Не запущена чистая PostgreSQL в ходе редакции	Не подтвержден цикл миграций	Развернуть окружение и зафиксировать лог
Нет протокола действий ролей	Сценарии основаны на реализации, а не наблюдении	Выполнить приемочный проход и приложить результаты
Расчетные оценки времени не измерены	Нельзя заявлять фактическую эргономичность	Провести измерение на контрольных сценариях
Официальный отзыв не подписан	Комплект документов требует оформления	Получить утвержденный документ руководителя

3.5.5 Регламент пилотного запуска и фиксации результатов

Пилотный запуск необходимо проводить в воспроизводимом окружении. Перед началом фиксируются версия исходного кода, параметры контейнеров, версия PostgreSQL и дата применения миграций. Это дает возможность отличить ошибку программного продукта от различий в конфигурации рабочего места.

Для защиты персональных данных испытание предпочтительно выполнять на тестовых учетных записях и вымышленных событиях. Проверяемые процессы от этого не меняются: роли, заявки, часы, сертификаты и отчеты формируются теми же маршрутами и в той же схеме данных, что и в реальной эксплуатации.

Каждый участник пилота получает роль и краткое задание. Волонтер должен найти событие и подать заявку; координатор - рассмотреть ее и подтвердить вклад; администратор - проверить итоговую аналитику и аудит. Разделение заданий предотвращает ситуацию, когда весь сценарий проверяется одной привилегированной учетной записью.

Перед началом прохождения фиксируется исходное состояние: отсутствие заявки выбранного волонтера, нулевое либо известное число подтвержденных часов и известные показатели аналитики. После выполнения действий эти значения сравниваются с итоговым состоянием, что делает проверку повторяемой.

Артефакты проверки должны храниться вместе с протоколом: журнал команд запуска, снимки ключевых экранов, выгрузка показателей и перечень замечаний. При наличии ошибки указываются роль, действие, ожидаемый и фактический результат, а также возможность воспроизведения.

Результат пилота оформляется решением: обязательный контур принят; принят с замечаниями, не препятствующими демонстрации; либо требует исправления до повторной проверки. Такое решение необходимо основывать на критериях таблицы 24, а не на субъективном впечатлении от интерфейса.

Таблица 30 - Журнал прохождения пилотного сценария

III аг	Роль и действие	Сохраняемое доказательство	Критерий успеха
1	Администратор создает тестовую организацию и роли	Снимок карточки организации	Роли доступны только назначенным пользователям
2	Координатор публикует мероприятие	Карточка со статусом published	Событие доступно волонтеру
3	Волонтер подает заявку	Статус заявки и идентификатор	Одна заявка на событие
4	Координатор принимает заявку и подтверждает часы	История статуса и запись времени	Вклад учтен после проверки
5	Администратор проверяет отчет	Dashboard либо экспорт	KPI отражает утвержденные данные
6	Координатор формирует сертификат	Документ и страница проверки кода	Результат доступен для верификации

Таблица 31 - Комплект артефактов опытной эксплуатации

Артефакт	Назначение	Где используется в выводах
Лог развертывания и миграций	Подтвердить работоспособность схемы на чистой базе	Техническая готовность системы
Лог backend/frontend	Зафиксировать ошибки обработки и сборки	Стабильность запуска
Снимки действий ролей	Подтвердить прохождение пользовательского пути	Функциональная пригодность
Контрольные данные и KPI	Сопоставить операции с аналитическим итогом	Достоверность отчетности
Сертификат и проверочный код	Показать проверяемый итог участия	Подтверждение результата
Реестр замечаний	Определить необходимость исправлений	Решение о приемке

3.5.6 Рекомендации по развитию проверки и внедрению

Первым направлением после защиты является автоматизация проверок backend. В репозитории отсутствуют тестовые файлы, поэтому успешная команда ``go test ./...`` подтверждает только компиляцию пакетов. Для критичных

операций требуются тесты статусов заявки, прав на подтверждение часов, расчета аналитики и проверки сертификата.

Второе направление связано с интеграционным окружением. Необходимо обеспечить сценарий запуска чистой базы данных, применения миграций и заполнения минимальных тестовых данных. Такой сценарий позволяет обнаружить ошибки ограничений, последовательности миграций и SQL-агрегаций до пользовательской демонстрации.

Третьим направлением является автоматизация клиентских сценариев. Для основных ролей целесообразны e2e-проверки входа, создания мероприятия, заявки, подтверждения времени и открытия аналитики. Эти тесты должны выполняться на контролируемом наборе данных и проверять видимые состояния интерфейса.

Четвертое направление - эксплуатационное наблюдение. После пилота необходимо проанализировать частоту ошибок, затруднения пользователей и действия, требующие лишних переходов. Только после получения наблюдений допустимо утверждать фактическое сокращение трудоемкости, а не расчетную возможность такого эффекта.

Пятое направление - оформление официального комплекта. Подготовленный документ содержит проектные и проверочные материалы, однако отзыв руководителя, подписи, регистрационные данные задания и при необходимости акт внедрения должны быть получены в утвержденных образовательной организацией формах.

Таким образом, третья глава отделяет доказанные свойства реализации от планируемых подтверждений. Система имеет проектную и программную основу для заявленных процессов, а предложенный регламент позволяет завершить эмпирическую проверку без изменения целей и архитектуры работы.

Таблица 32 - Приоритеты последующего подтверждения качества

Приоритет	Работа	Проверяемый риск	Ожидаемый артефакт
Высокий	Запуск PostgreSQL и миграций на чистой базе	Ошибка физической модели данных	Лог успешного применения миграций
Высокий	Сценарий заявка -> часы -> аналитика -> сертификат	Разрыв обязательного контура	Протокол и снимки результатов
Высокий	Автотесты серверных полномочий	Недопустимое изменение данных	Отчет unit/integration tests
Средний	E2E-путь основных ролей	Регрессия пользовательского интерфейса	Отчет браузерных проверок
Средний	Измерение времени и замечаний пилота	Необоснованная оценка удобства	Заполненная форма наблюдений
Документы	Получение подписанных документов	Неполный официальный комплект ВКР	Отзыв, подписи, при наличии акт

Итоговый протокол опытной эксплуатации должен заполняться после выполнения всех обязательных сценариев. В нем фиксируются конфигурация запуска, проверенные роли, полученные показатели, выявленные замечания и решение о готовности демонстрационного контура.

Замечания классифицируются по влиянию на результат: критическое замечание препятствует подтверждению обязательного сценария; существенное не блокирует весь цикл, но требует исправления до внедрения; рекомендательное относится к удобству или дальнейшему развитию.

Если выявляется критическая ошибка, повторная проверка проводится после исправления именно того сценария, в котором обнаружено отклонение, и связанных с ним результатов. Например, исправление подтверждения часов требует повторной проверки аналитики и сертификата, поскольку они используют итоговые часы.

Форма протокола в таблице 33 предназначена для заполнения после фактического пилота. В настоящей работе она приведена как обязательный инструмент завершения испытаний и не заменяется неподтвержденными числовыми значениями.

Таблица 33 - Форма итогового протокола опытной эксплуатации

Раздел протокола	Фиксируемые сведения	Основание решения
Окружение	Версия приложения, БД, миграции, дата запуска	Воспроизводимость результата
Состав участников	Тестовые роли и выполненные задания	Покрытие пользовательских целей
Обязательный цикл	Заявка, часы, аналитика, сертификат	Достижение приемочных критериев
Негативные проверки	Отклонение запрещенных операций	Сохранность данных и полномочий
Метрики	KPI, время сценариев, количество замечаний	Оценка результата эксплуатации
Решение	Принято, принято с замечаниями или повторная проверка	Подписанный итоговый протокол

Проверки, пользовательские сценарии, показатели, программа опытной эксплуатации и оценка практического результата отражены в таблицах 16-33.

3.6 Выводы по третьему разделу

Определена программа испытаний, охватывающая роли, операции, статусы, формирование документов и итоговую аналитику.

Фактический запуск ``go test ./...`` подтвердил компилируемость backend, одновременно показав отсутствие автоматизированных тестовых файлов; этот пробел необходимо устранить при дальнейшем развитии.

Фактический запуск ``npm run build`` подтвердил успешную проверку типов и production-сборку frontend Пульс.

На запущенном приложении проверены миграция профильных полей, сохранение пользовательского значения, запрет удаления системного поля и загрузка аватара; реализованный результат подтвержден экранными снимками.

Добавленный workflow GitHub Actions фиксирует автоматическую проверку backend, frontend и контейнерного smoke-сценария для последующих изменений.

Демонстрационные сценарии связаны с деревом целей и позволяют проверить путь от заявки волонтера до подтвержденного времени, документа и KPI.

Выполнена расчетная квантификация ключевых действий интерфейса; она задает основу для последующей пользовательской проверки с измерением времени.

Полученный программный продукт имеет практическую значимость для автоматизации волонтерской деятельности и может развиваться после расширения автоматизированных испытаний и подтверждения эксплуатации на рабочем наборе данных.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была спроектирована и разработана информационная система Пульс, предназначенная для управления волонтерской деятельностью с элементами геймификации и аналитики. Работа включала анализ предметной области, постановку требований, проектирование архитектуры, разработку серверной и клиентской частей, проектирование базы данных и проверку основных сценариев.

В первом разделе рассмотрены особенности волонтерской деятельности как объекта автоматизации. Выявлены проблемы ручного учета: разрозненность данных, сложность подтверждения часов, отсутствие прозрачной истории участия, высокая нагрузка на координаторов и недостаток аналитики. На основании анализа обоснована необходимость создания специализированной системы.

Во втором разделе выполнено проектирование и описание реализации Пульс. Система построена на клиент-серверной архитектуре: frontend реализован на Vue.js 3 и TypeScript, backend - на Go, база данных - PostgreSQL. В составе приложения реализованы модули авторизации, организаций, мероприятий, задач, учета времени, уведомлений, базы знаний, геймификации, аналитики, сертификатов и системного администрирования.

В третьем разделе рассмотрена проверка работоспособности и оценка результатов. Демонстрационный сценарий подтвердил возможность прохождения полного цикла работы: от создания организации и мероприятия до подтверждения часов, начисления достижений и формирования сертификата. Полученные результаты соответствуют поставленной цели и задачам.

Практическая значимость работы состоит в создании программной основы для автоматизации деятельности волонтерских организаций, студенческих объединений и социальных проектов. Дальнейшее развитие системы может быть связано с интеграциями календарей и мессенджеров,

расширением аналитики, настройкой правил геймификации, импортом данных и развитием multi-tenant модели.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления.
2. ГОСТ Р 7.0.5-2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления.
3. ГОСТ 19.701-90 (ИСО 5807-85). Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения.
4. ГОСТ Р 2.105-2019. Единая система конструкторской документации. Общие требования к текстовым документам.
5. СМК-О-СМГТУ-36-20. Выпускная квалификационная работа. Структура, содержание, общие правила выполнения и оформления.
6. Выпускная квалификационная работа: от бакалавриата до аспирантуры: учебное пособие / О.С. Логунова, Л.Г. Егорова, М.Ю. Наркевич, Ю.В. Кочержинская. Магнитогорск: МГТУ им. Г.И. Носова, 2025.
7. Федеральный закон от 11.08.1995 N 135-ФЗ «О благотворительной деятельности и добровольчестве (волонтерстве)».
8. Ассоциация Добро.рф. АВЦ 10 лет: показатели развития добровольчества и данные ВЦИОМ. URL: <https://avc10.dobro.ru/> (дата обращения: 26.05.2026).
9. Добро.рф. Международный добровольческий резерв: сведения о цифровом контуре Dobro.ru. URL: <https://mission.dobro.ru/reserve> (дата обращения: 26.05.2026).
10. Федеральный закон от 27.07.2006 N 152-ФЗ «О персональных данных».
11. Калянов Г.Н. Моделирование, анализ, реорганизация и автоматизация бизнес-процессов. М.: Финансы и статистика, 2006. 240 с.
12. Маклаков С.В. BPwin и ERwin. CASE-средства разработки информационных систем. М.: Диалог-МИФИ, 2000. 256 с.
13. Советов Б.Я., Цехановский В.В., Чертовской В.Д. Базы данных: теория и практика. М.: Юрайт, 2024. 463 с.

14. Астахова И.Ф., Чулюков В.А., Половинкин И.П. Проектирование баз данных. Воронеж: ВГУ, 2017. 74 с.
15. Горожанина Е.И. Проектирование баз данных и баз знаний. Самара: ПГУТИ, 2021. 108 с.
16. Гринченко Н.Н., Хизриева Н.И. Базы данных. Программирование на SQL. Рязань: РГРТУ, 2023. 240 с.
17. Fielding R.T. Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation. University of California, Irvine, 2000.
18. Newman S. Building Microservices. Designing Fine-Grained Systems. O'Reilly Media, 2021.
19. Fowler M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.
20. Martin R.C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.
21. Ричардсон Л., Руби С. RESTful Web Services. СПб.: Символ-Плюс, 2008. 448 с.
22. Тузовский А.Ф. Проектирование и разработка web-приложений. Томск: ТПУ, 2014. 219 с.
23. Калиберда Е.А., Кравченко К.В. Разработка web-приложений. Омск: ОмГТУ, 2023. 100 с.
24. Ермаков С.Р. Веб-разработка. М.: РТУ МИРЭА, 2025. 95 с.
25. Никулин В.В., Олейников А.А., Сорокин А.А., Олейникова А.В. Разработка серверной части веб-ресурса. СПб.: Лань, 2023. 132 с.
26. Баланов А.Н. Бэкенд-разработка веб-приложений: архитектура, проектирование и управление проектами. СПб.: Лань, 2025. 304 с.
27. Булгакова И.А. Разработка и прототипирование веб-сайтов и интерфейсов онлайн. СПб.: Лань, 2024. 128 с.
28. Макарова Т.В. Веб-дизайн. Омск: ОмГТУ, 2015. 148 с.
29. Медникова О.В. Проектирование интерфейсов. М.: РУТ (МИИТ), 2019. 68 с.

30. Проектирование графических интерфейсов программных систем: практикум / сост. Р.А. Ещенко и др. Хабаровск: ДВГУПС, 2024. 117 с.
31. Игнатьев А.В. Тестирование программного обеспечения. СПб.: Лань, 2025. 56 с.
32. Бубнов А.А., Бубнов С.А., Тишкина В.В. Тестирование программного обеспечения. Рязань: РГРТУ, 2024. 164 с.
33. Зубкова Т.М. Технология разработки программного обеспечения. СПб.: Лань, 2022. 324 с.
34. Баланов А.Н. DevOps: интеграция и автоматизация. СПб.: Лань, 2026. 240 с.
35. Сейерс Э.Х., Милл А. Docker на практике. М.: ДМК Пресс, 2020. 516 с.
36. Шнайер Б. Прикладная криптография. Протоколы, алгоритмы и исходные тексты на языке С. М.: Вильямс, 2017. 1024 с.
37. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. СПб.: Питер, 2021. 368 с.
38. Ларман К. Применение UML 2.0 и шаблонов проектирования. М.: Вильямс, 2019. 736 с.
39. Коберн А. Современные методы описания функциональных требований к системам. М.: Лори, 2002. 263 с.
40. Нильсен Я. Веб-дизайн: удобство использования веб-сайтов. М.: Вильямс, 2007. 368 с.
41. Манухина О.В. Информационные системы. Чита: ЗабГУ, 2021. 135 с.
42. Пульс. Техническое задание на разработку информационной системы управления волонтерами с элементами геймификации и аналитики. 2026.
43. Пульс. Описание реализованного функционала по модулям. 2026.
44. Пульс. Обзор API и ER-диаграмма. 2026.
45. Пульс. Программа и чеклист испытаний. 2026.
46. Пульс. Сценарий демонстрации и описание панели аналитики. 2026.

ПРИЛОЖЕНИЕ А МОДЕЛИ СТРУКТУРЫ БАЗЫ ДАННЫХ

На рисунках А.1-А.2 приведены укрупненные графические модели базы данных Пульс и текстовая трассировка основных связей между сущностями. Модели сформированы по фактическим миграциям проекта.

Информационно-логическая ER-модель

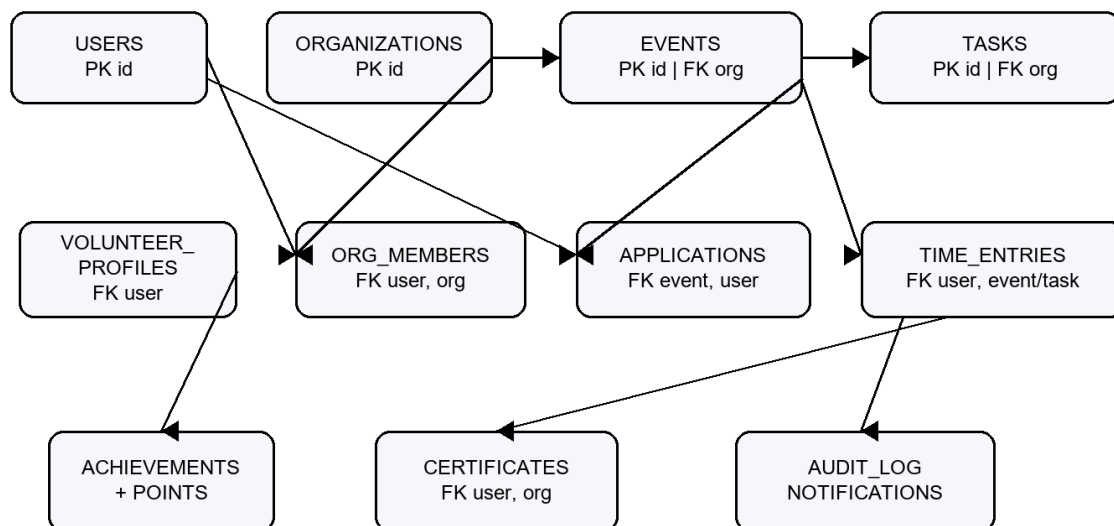


Рисунок А.1 - Информационно-логическая модель базы данных Пульс

Физическая модель PostgreSQL: ключевые ограничения

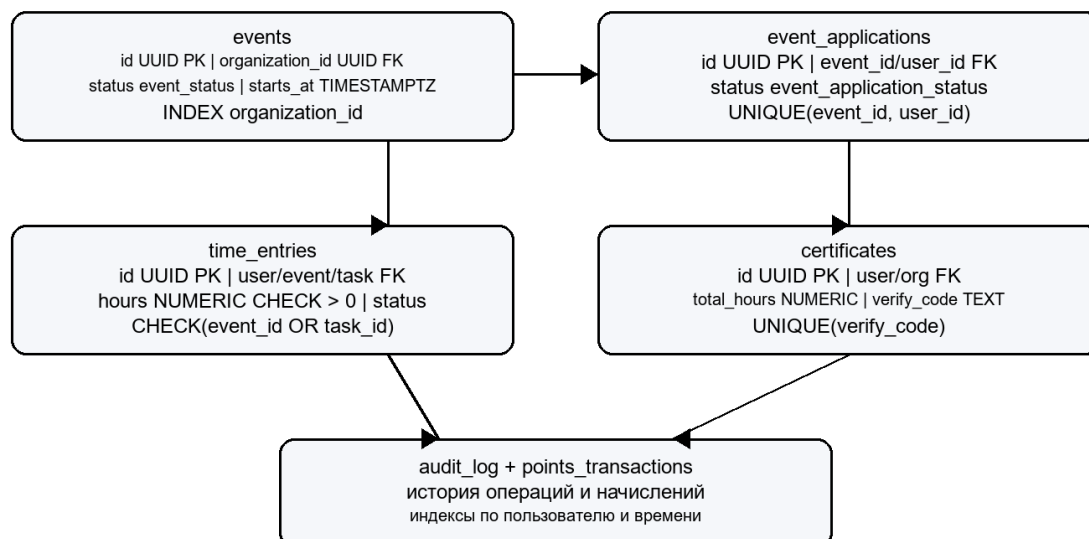


Рисунок А.2 - Физическая модель ключевых таблиц PostgreSQL

USERS ||--o{ ORGANIZATION_MEMBERS : has

ORGANIZATIONS ||--o{ ORGANIZATION_MEMBERS : includes
ORGANIZATIONS ||--o{ EVENTS : owns
ORGANIZATIONS ||--o{ TASKS : owns
USERS ||--|| VOLUNTEER_PROFILES : profile
EVENTS ||--o{ EVENT_APPLICATIONS : receives
EVENTS ||--o{ EVENT_ATTENDANCE : tracks
TASKS ||--o{ TASK_ASSIGNMENTS : assigns
USERS ||--o{ TIME_ENTRIES : submits
USERS ||--o{ VOLUNTEER_ACHIEVEMENTS : earns
USERS ||--o{ NOTIFICATIONS : receives
USERS ||--o{ CERTIFICATES : receives
ORGANIZATIONS ||--o{ CERTIFICATES : issues

ПРИЛОЖЕНИЕ Б ПРОГРАММА ИСПЫТАНИЙ

Программа испытаний включает проверку backend, frontend и демонстрационного сценария. Backend проверяется запуском тестов, применением миграций и ручной проверкой API. Frontend проверяется сборкой, навигацией, адаптивностью и выполнением основных операций в интерфейсе.

- 1) выполнить проверку миграций базы данных на чистом окружении;
- 2) проверить регистрацию, вход, обновление сессии, выход и смену пароля;
- 3) проверить CRUD организаций, мероприятий, задач, новостей и базы знаний;
- 4) проверить подачу заявки на мероприятие и изменение ее статуса координатором;
- 5) проверить создание, подтверждение и отклонение записей времени;
- 6) проверить начисление достижений и отображение рейтинга;
- 7) проверить генерацию, скачивание и публичную проверку сертификата;
- 8) проверить отображение отчетов аналитики и журнала аудита;
- 9) выполнить сборку frontend и проверить переходы по основному меню.

ПРИЛОЖЕНИЕ В ФРАГМЕНТ API СИСТЕМЫ

В таблице В.1 приведен укрупненный перечень основных endpoint, используемых клиентской частью Пульс. Фактическая структура API может уточняться при развитии проекта, однако приведенная группировка отражает реализованную архитектуру модулей.

Таблица В.1 - Основные endpoint REST API

Группа	Метод и путь	Назначение
Авторизация	POST /api/v1/auth/register	Регистрация нового пользователя
Авторизация	POST /api/v1/auth/login	Вход пользователя в систему
Авторизация	GET /api/v1/auth/me	Получение сведений о текущем пользователе
Организации	GET /api/v1/organizations	Получение списка доступных организаций
Организации	POST /api/v1/organizations	Создание организации пользователем с правами администратора
Организации	POST /api/v1/organizations/{id}/members	Добавление участника в организацию
Мероприятия	GET /api/v1/events	Получение списка мероприятий с фильтрами
Мероприятия	POST /api/v1/events	Создание мероприятия координатором
Мероприятия	POST /api/v1/events/{id}/applications	Подача заявки волонтером
Мероприятия	PATCH /api/v1/events/{id}/applications/{applicationId}	Изменение статуса заявки координатором
Задачи	GET /api/v1/tasks	Получение списка задач
Задачи	POST /api/v1/tasks	Создание задачи
Учет времени	POST /api/v1/time-entries	Создание записи волонтерских часов
Учет времени	PATCH /api/v1/time-entries/{id}/approve	Подтверждение записи времени
Аналитика	GET /api/v1/analytics/overview	Получение сводных показателей
Сертификаты	POST /api/v1/certificates/generate	Генерация сертификата или справки
Сертификаты	GET /api/v1/certificates/verify/{code}	Публичная проверка сертификата по коду

Пример структуры запроса на создание мероприятия:

```
{
  "organizationId": 1,
  "title": "Помощь в проведении городского мероприятия",
  "format": "offline",
  "startsAt": "2026-06-10T10:00:00+05:00",
```

```
"endsAt": "2026-06-10T15:00:00+05:00",  
"location": "Магнитогорск, площадка мероприятия",  
"capacity": 30  
}
```

Пример ответа API содержит идентификатор созданного мероприятия, текущий статус, сведения об организации и служебные даты создания и изменения. Клиентская часть использует эти данные для перехода на страницу мероприятия и обновления списка.

ПРИЛОЖЕНИЕ Г БЛОК-СХЕМЫ И ОПИСАНИЕ КЛЮЧЕВЫХ АЛГОРИТМОВ

На рисунках Г.1-Г.5 приведены алгоритмические схемы операций, использующих статусы и проверки целостности данных. Схемы дублируются в приложении в увеличенном масштабе для удобства проверки.

Блок-схема обработки заявки

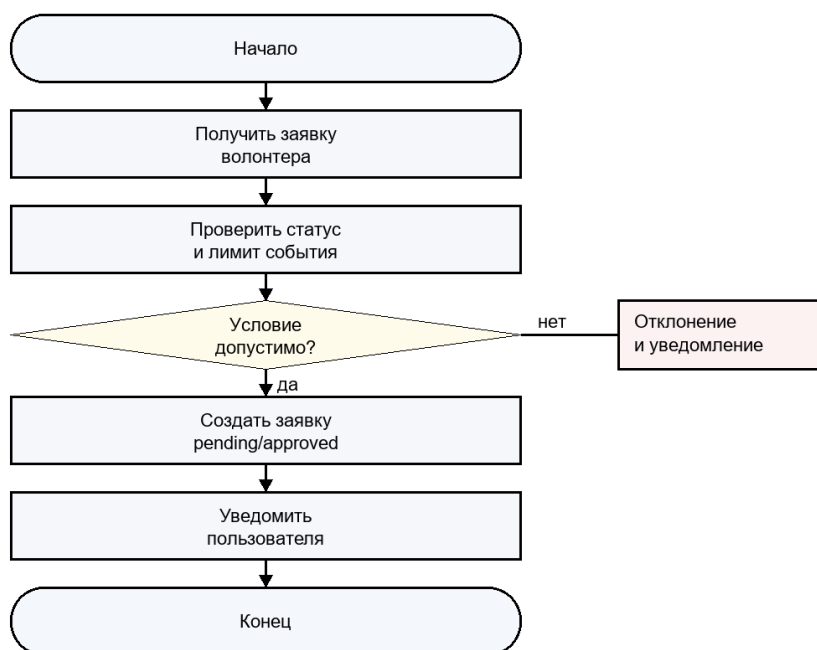


Рисунок Г.1 - Обработка заявки на мероприятие

Блок-схема подтверждения часов

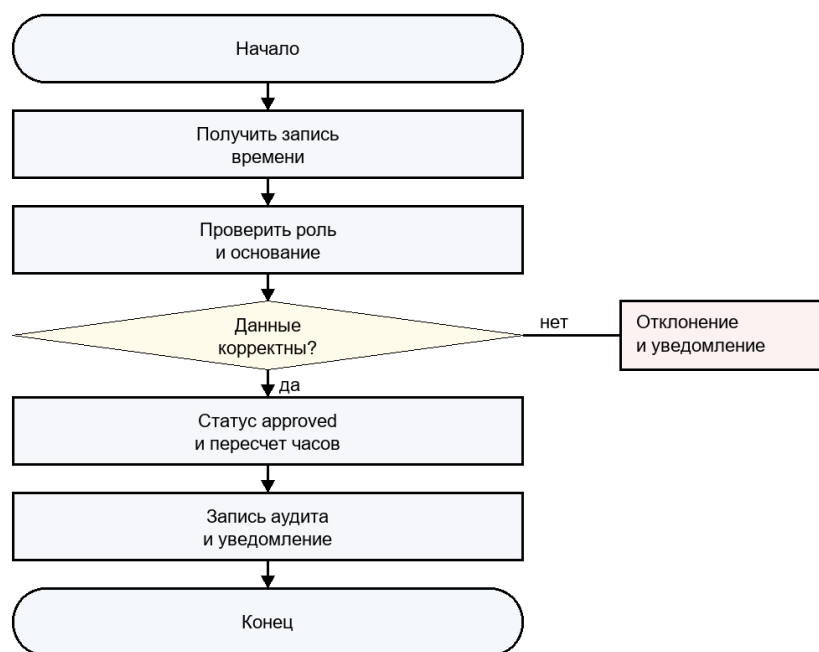


Рисунок Г.2 - Подтверждение волонтерских часов

Блок-схема начисления достижения

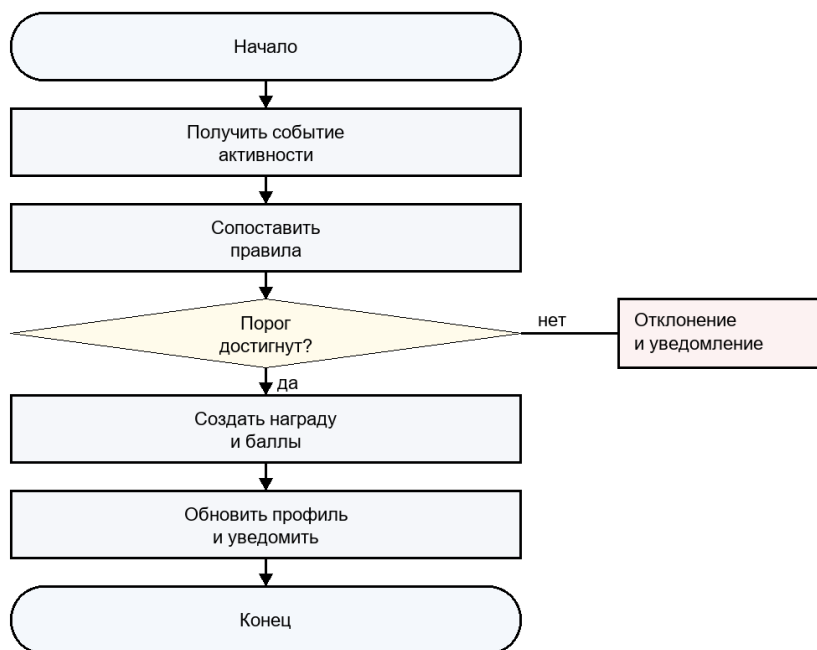


Рисунок Г.3 - Начисление достижения

Блок-схема генерации сертификата

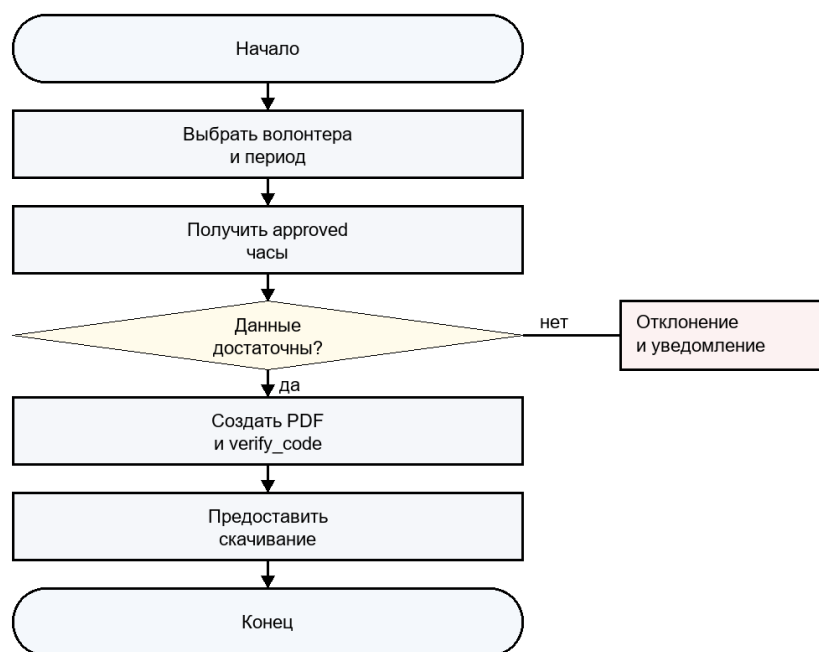


Рисунок Г.4 - Формирование сертификата

Блок-схема формирования аналитики

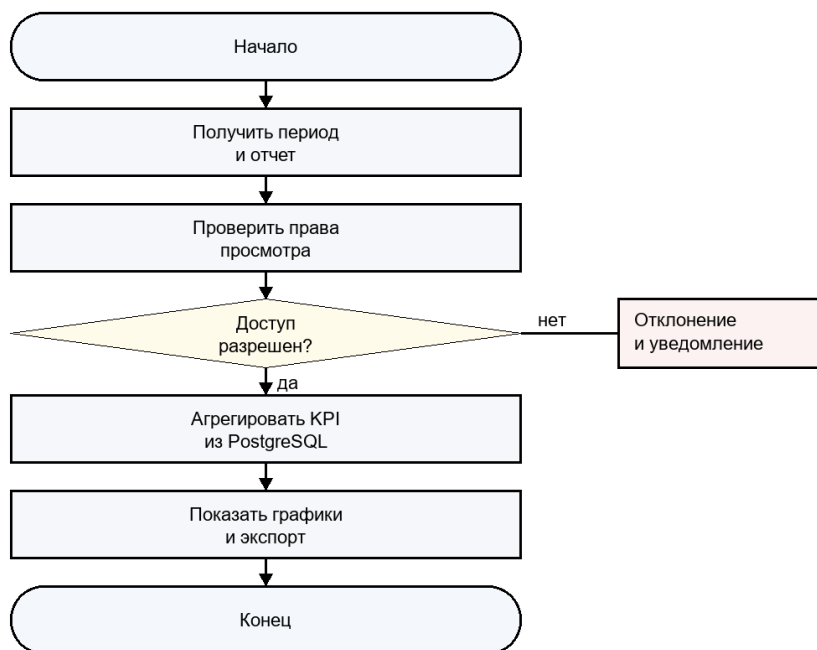


Рисунок Г.5 - Формирование аналитики

Алгоритм обработки заявки на мероприятие:

- 1) Пользователь открывает опубликованное мероприятие и отправляет заявку.
- 2) Сервер проверяет, что пользователь авторизован и состоит в допустимой организации.
- 3) Сервер проверяет, что мероприятие не отменено и не завершено.
- 4) Сервер проверяет, не существует ли уже активной заявки этого пользователя.
- 5) Если лимит участников не превышен, заявке присваивается статус ожидания рассмотрения или подтверждения согласно настройкам мероприятия.
- 6) Если лимит превышен, заявке может быть присвоен статус листа ожидания.
- 7) Координатор рассматривает заявку и принимает решение.
- 8) Пользователь получает уведомление об изменении статуса.

Алгоритм подтверждения волонтерских часов:

- 1) Волонтер создает запись времени и указывает организацию, дату, длительность и основание.
- 2) Система сохраняет запись со статусом ожидания подтверждения.
- 3) Координатор открывает список записей времени своей организации.
- 4) Координатор проверяет связь записи с мероприятием или задачей.
- 5) При корректных данных координатор подтверждает запись.
- 6) Подтвержденные часы включаются в профиль волонтера и аналитические отчеты.
- 7) При некорректных данных координатор отклоняет запись с комментарием.
- 8) Система фиксирует действие в журнале аудита и отправляет уведомление волонтеру.

Алгоритм начисления достижения:

- 1) После подтверждения часов или завершения мероприятия система получает событие активности.
- 2) Сервис геймификации загружает правила достижений, применимые к организации или системе в целом.
- 3) Для каждого правила рассчитывается, выполнено ли условие пользователем.
- 4) Если достижение еще не выдавалось и условие выполнено, создается запись о выдаче достижения.
- 5) При необходимости создается транзакция баллов.
- 6) Пользователь получает уведомление, а его профиль и рейтинг обновляются.

Алгоритм генерации сертификата:

- 1) Координатор или администратор выбирает пользователя, организацию и период, за который требуется сформировать документ.
- 2) Сервер проверяет права пользователя, запрашивающего генерацию сертификата.

3) Система выбирает только подтвержденные записи времени за указанный период.

4) Система рассчитывает суммарное количество часов и перечень оснований участия.

5) Если подтвержденных данных недостаточно, сервер возвращает сообщение о невозможности сформировать документ.

6) При успешной проверке создается запись сертификата с уникальным проверочным кодом.

7) Сервер формирует PDF-документ и сохраняет сведения о нем в базе данных.

8) Пользователь получает возможность скачать документ, а внешний проверяющий может открыть публичную страницу проверки по коду.

Алгоритм формирования сводной аналитики:

1) Пользователь открывает аналитический раздел и выбирает организацию, период и тип отчета.

2) Сервер проверяет, что пользователь имеет право просматривать аналитику выбранной организации.

3) Система получает агрегированные данные по мероприятиям, задачам, заявкам, подтвержденным часам и достижениям.

4) Неподтвержденные записи времени исключаются из итоговых показателей, но могут отображаться отдельно как ожидающие проверки.

5) Показатели группируются по периоду, направлению, мероприятию или пользователю в зависимости от выбранного отчета.

6) API возвращает структурированный результат, который frontend отображает в виде карточек, таблиц или графиков.

7) При необходимости пользователь экспортирует результаты для дальнейшей подготовки управленческого отчета.

Описанные алгоритмы позволяют отделить пользовательское действие от подтвержденного результата. Это особенно важно для учета часов и

сертификатов, поскольку документ должен формироваться только на основании проверенных данных.

В проекте такой подход применяется последовательно: заявка еще не означает участие, созданная запись времени еще не означает подтвержденный вклад, а наличие активности еще не означает автоматическую выдачу документа. Между этими состояниями существуют проверки, выполняемые пользователями с соответствующими правами.

Разделение состояний повышает надежность отчетности. Если организация использует Пульс для официального подтверждения волонтерской активности, то система должна хранить не только итоговое значение часов, но и путь его получения: мероприятие или задачу, пользователя, дату, статус и координатора, подтвердившего запись.

Таким образом, алгоритмы обработки заявок, часов, достижений, сертификатов и аналитики образуют единый контур управления данными. Каждый следующий этап использует результат предыдущего только после проверки, что снижает риск ошибочных начислений и повышает доверие к итоговым отчетам.

ПРИЛОЖЕНИЕ Д ЭКРАННЫЕ ФОРМЫ ПРИЛОЖЕНИЯ

На рисунках Д.1-Д.10 приведены основные экранные формы разработанной информационной системы Пульс. Скриншоты подтверждают наличие пользовательского интерфейса для авторизации, работы с мероприятиями, задачами, учетом часов, аналитикой, сертификатами, системным администрированием и настраиваемым профилем.

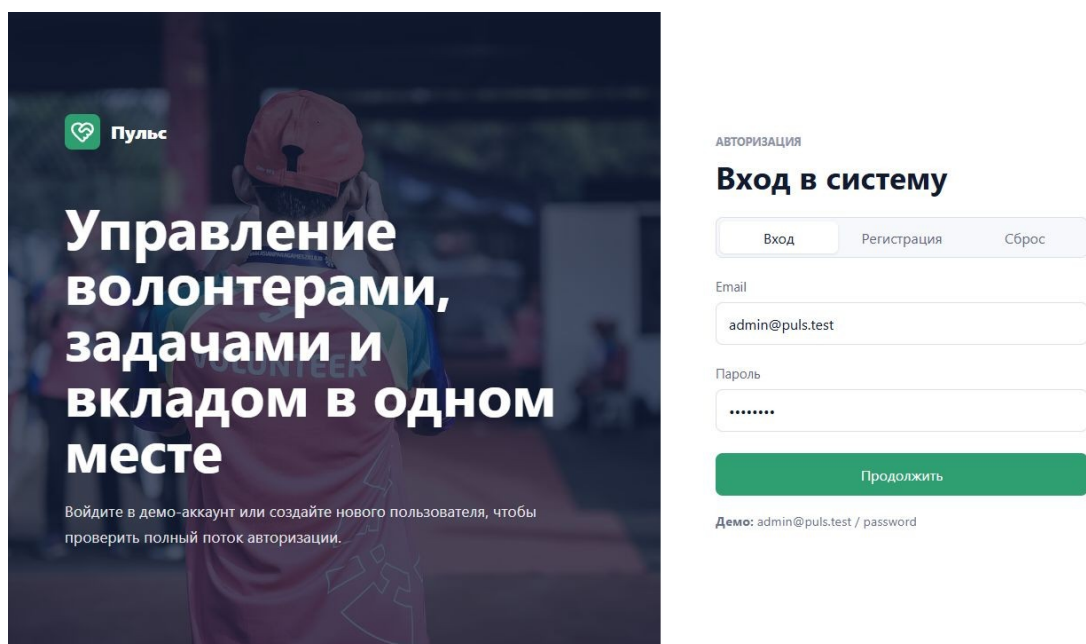


Рисунок Д.1 - Страница авторизации пользователя

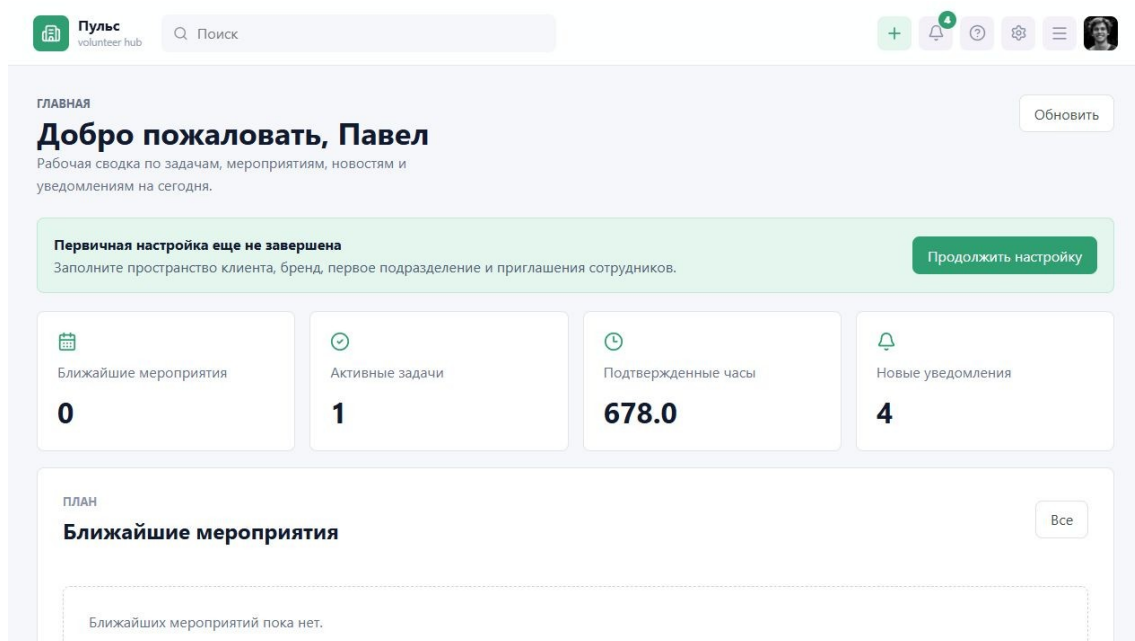


Рисунок Д.2 - Главная панель пользователя

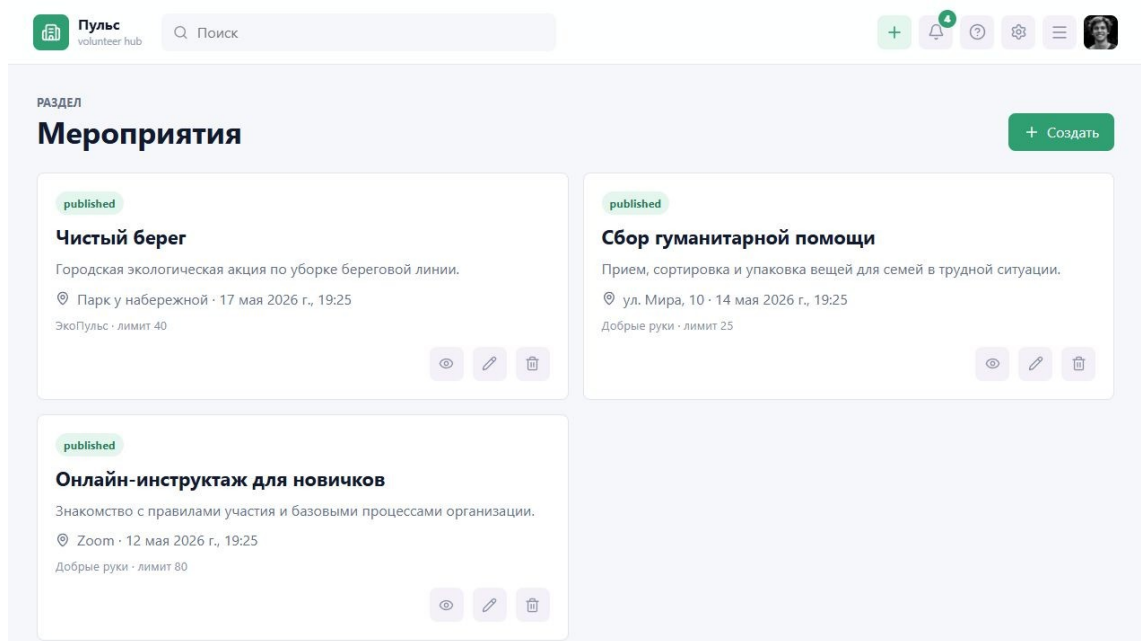


Рисунок Д.3 - Раздел управления мероприятиями

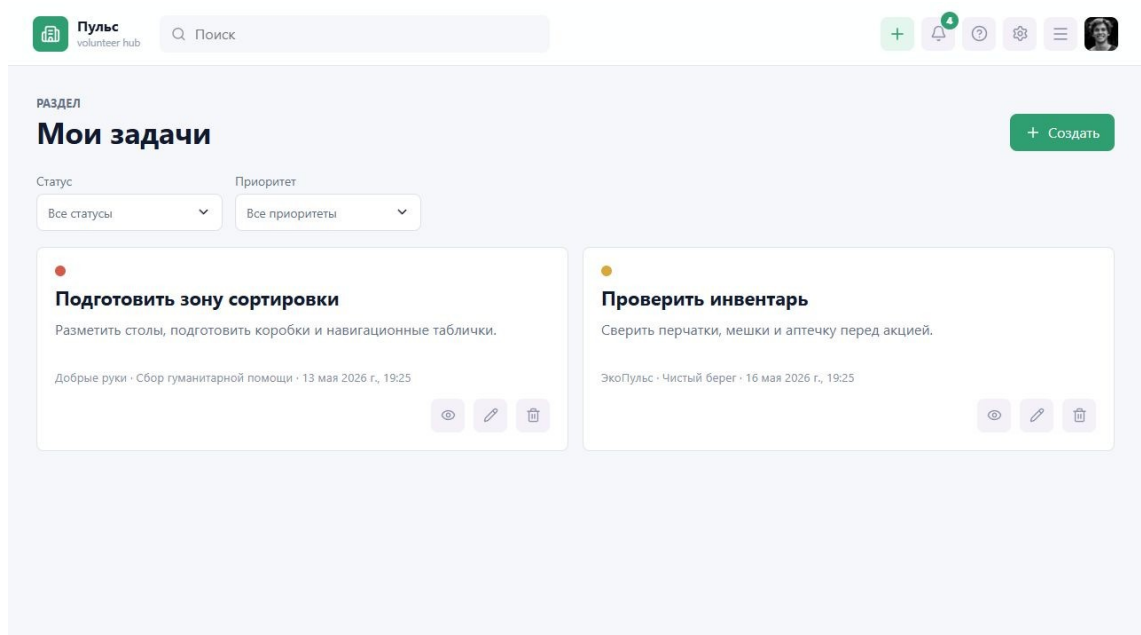


Рисунок Д.4 - Раздел задач

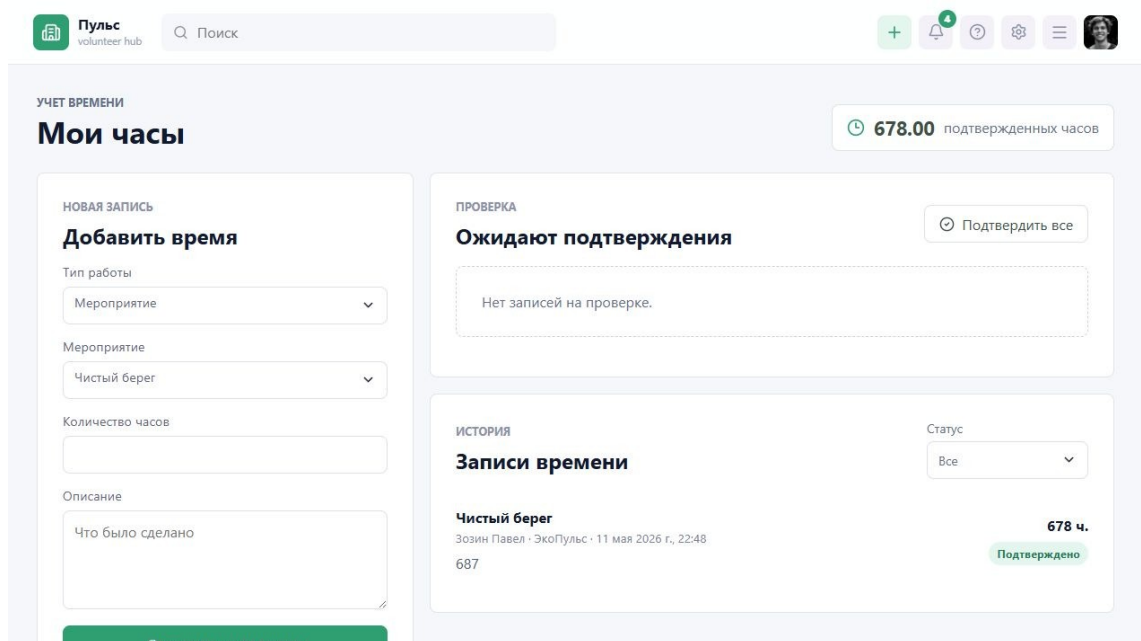


Рисунок Д.5 - Раздел учета волонтерских часов

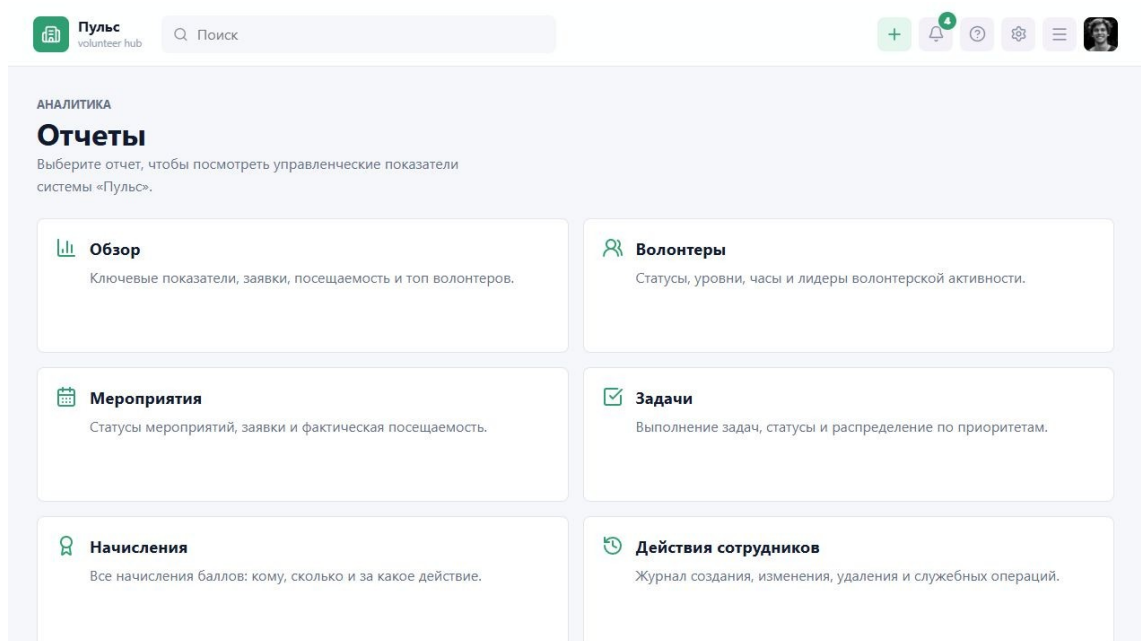


Рисунок Д.6 - Аналитическая панель

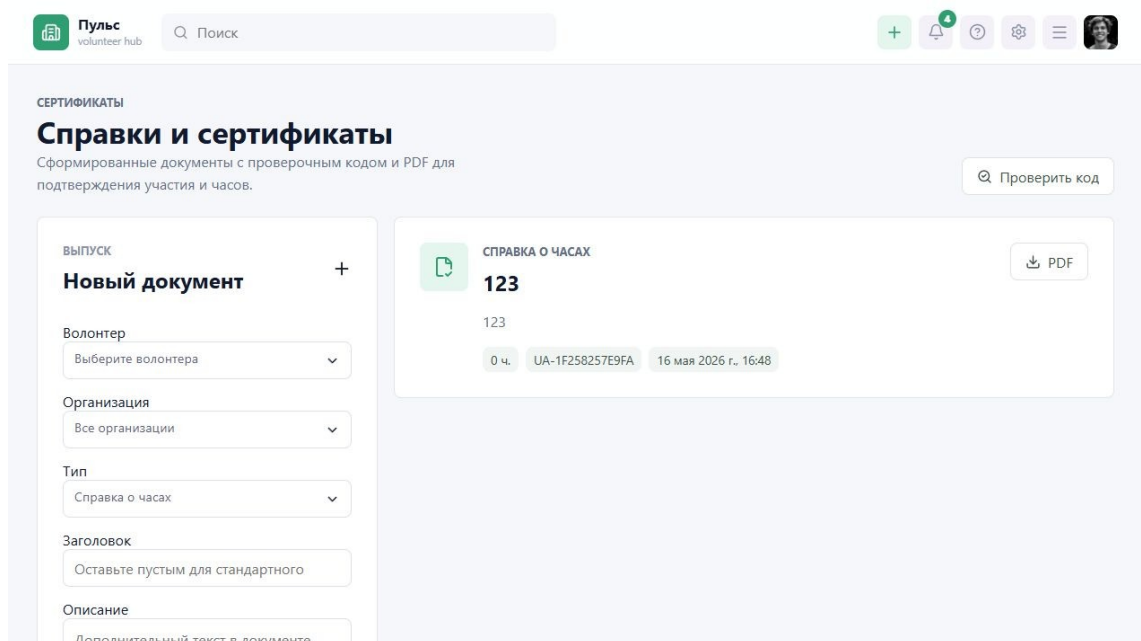


Рисунок Д.7 - Раздел сертификатов

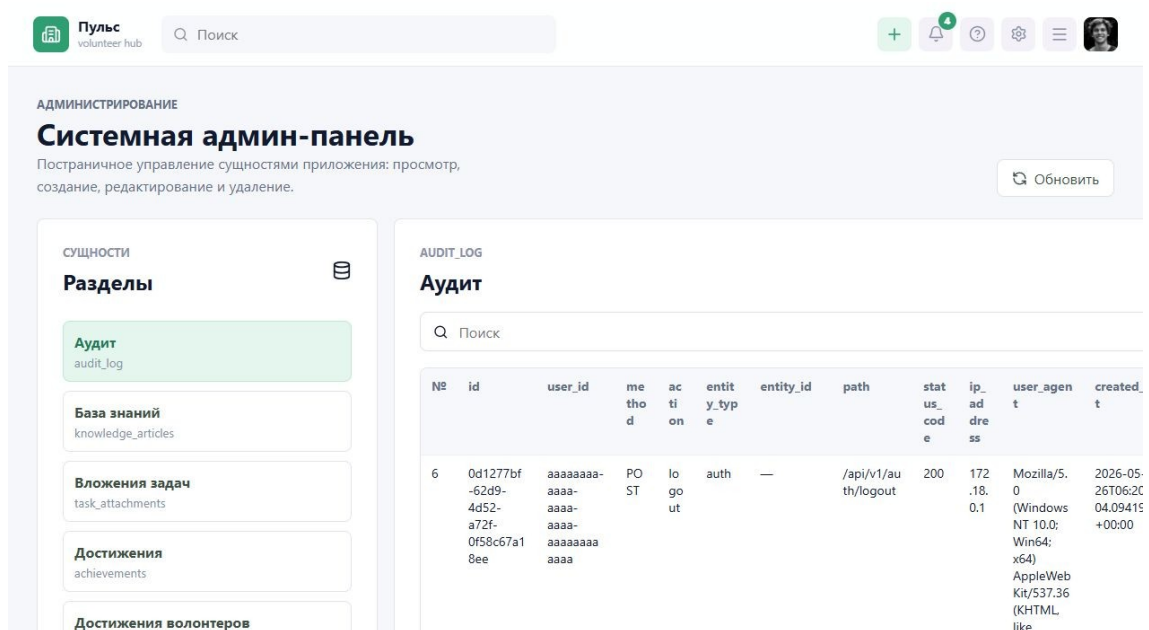


Рисунок Д.8 - Системная админ-панель

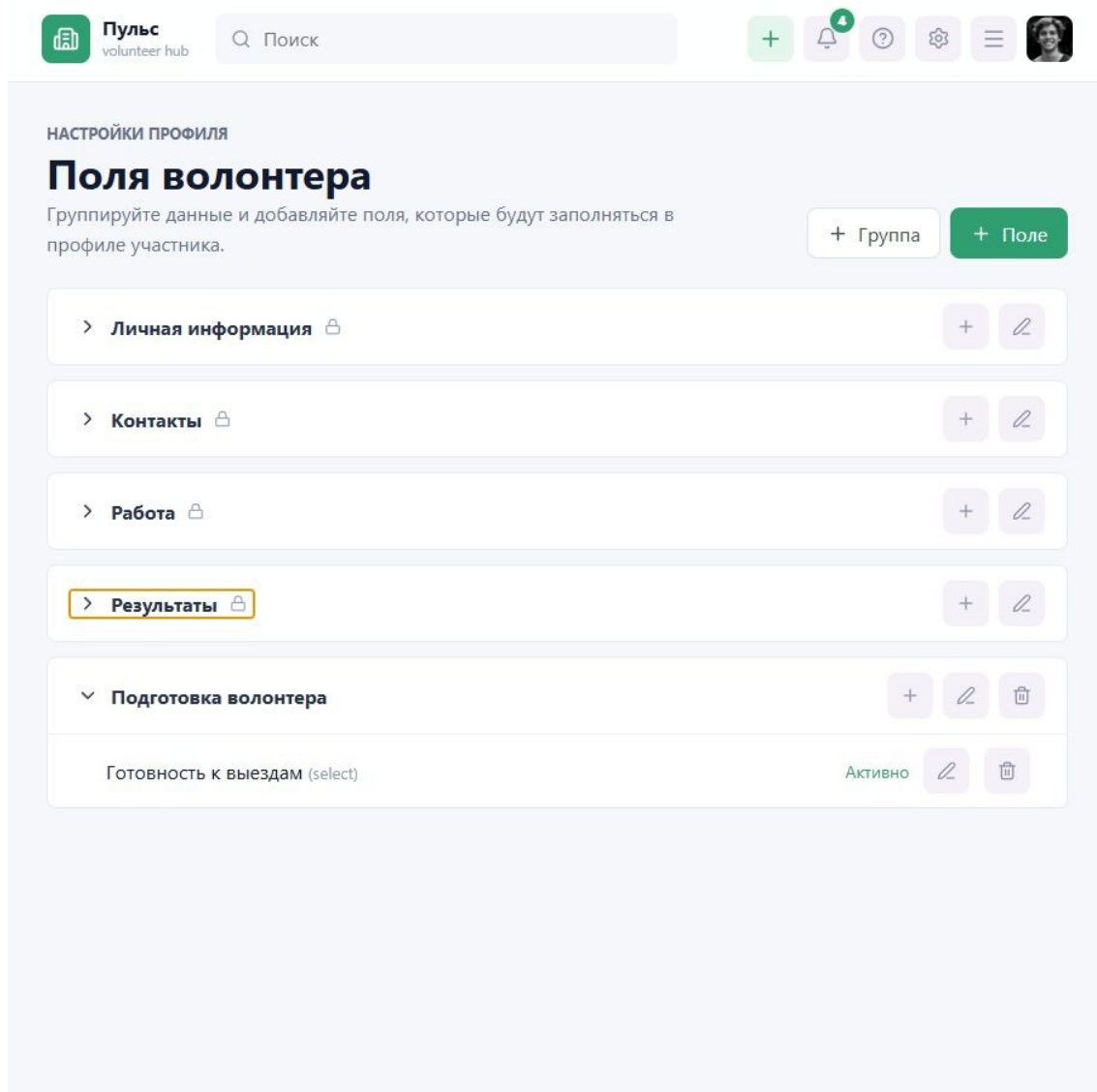


Рисунок Д.9 - Администрирование полей профиля



Пулсь

volunteer hub

+



Морозов Илья

volunteer1@test.local

Организация

Добрые руки

Подтвержденные часы

18.5 ч.

Баллы

200

Работа и опыт

Город

Екатеринбург

Статус

Активный

Опыт и описание

Готов помогать на городских событиях и социальных акциях.

Интересы

Сохранить

Подготовка волонтера

Готовность к выездам*

Готов(а)

Сохранить дополнительные данные

Рисунок Д.10 - Расширенный профиль волонтера

Листинг Д.1 - Workflow непрерывной проверки GitHub Actions

```

name: Puls quality gate

on:
  push:
    branches: ["main", "develop", "codex/**"]
  pull_request:

permissions:
  contents: read

jobs:
  backend:
    name: Backend verification
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: unityaid-back
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - name: Set up Go
        uses: actions/setup-go@v5
        with:

```

```

    go-version-file: unityaid-back/go.mod
    cache-dependency-path: unityaid-back/go.sum
  - name: Download modules
    run: go mod download
  - name: Static checks
    run: go vet ./...
  - name: Unit and package tests
    run: go test ./...
  - name: Build services
    run: go build ./...

frontend:
  name: Frontend build
  runs-on: ubuntu-latest
  defaults:
    run:
      working-directory: unityaid-front
  steps:
    - name: Checkout
      uses: actions/checkout@v4
    - name: Set up Node.js
      uses: actions/setup-node@v4
      with:
        node-version: 22
        cache: npm
        cache-dependency-path: unityaid-front/package-lock.json
    - name: Install dependencies
      run: npm ci
    - name: Type check and production build
      run: npm run build

containers:
  name: Container smoke test
  runs-on: ubuntu-latest
  needs: [backend, frontend]
  steps:
    - name: Checkout
      uses: actions/checkout@v4
    - name: Validate compose definition
      run: docker compose config --quiet
    - name: Build service images
      run: docker compose build backend frontend
    - name: Start application
      run: docker compose up -d --wait db backend frontend
    - name: Check backend and frontend endpoints
      run: |
        curl --fail --retry 5 --retry-delay 2 http://localhost:8080/api/v1/health
        curl --fail --retry 5 --retry-delay 2 http://localhost:5173
    - name: Publish container diagnostics on failure
      if: failure()
      run: docker compose logs --no-color
    - name: Stop application
      if: always()
      run: docker compose down -v

```

ПРИЛОЖЕНИЕ Е ФРАГМЕНТЫ ИСХОДНОГО КОДА

В приложении приведены реальные фрагменты исходного кода Пульс, отражающие модель данных, обработку пользовательских операций и отображение итогов работы продукта. Совокупный объем включенных листингов превышает 400 строк исходного кода; ключевой для обязательного результата код аналитики представлен в листингах Е.4-Е.6.

Листинг Е.1 - Физическая модель основных сущностей и связей базы данных

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'app_role') THEN
        CREATE TYPE app_role AS ENUM ('super_admin', 'org_admin', 'coordinator',
        'volunteer');
    END IF;

    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'member_status') THEN
        CREATE TYPE member_status AS ENUM ('active', 'inactive', 'blocked');
    END IF;

    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'event_format') THEN
        CREATE TYPE event_format AS ENUM ('online', 'offline', 'hybrid');
    END IF;

    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'event_status') THEN
        CREATE TYPE event_status AS ENUM ('draft', 'published', 'completed', 'cancelled');
    END IF;

    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'task_status') THEN
        CREATE TYPE task_status AS ENUM ('created', 'assigned', 'in_progress', 'review',
        'completed', 'cancelled');
    END IF;
END $$;

CREATE TABLE IF NOT EXISTS organizations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    slug TEXT NOT NULL UNIQUE,
    description TEXT NOT NULL DEFAULT '',
    contact_email TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    patronymic TEXT,
    avatar_url TEXT,
    locale TEXT NOT NULL DEFAULT 'ru',
    is_email_verified BOOLEAN NOT NULL DEFAULT true,
    is_active BOOLEAN NOT NULL DEFAULT true,
    last_login_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS organization_members (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
role app_role NOT NULL,
status member_status NOT NULL DEFAULT 'active',
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
UNIQUE (organization_id, user_id)
);

CREATE TABLE IF NOT EXISTS volunteer_profiles (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL UNIQUE REFERENCES users(id) ON DELETE CASCADE,
city TEXT,
phone TEXT,
bio TEXT NOT NULL DEFAULT '',
total_hours NUMERIC(8, 2) NOT NULL DEFAULT 0,
points INTEGER NOT NULL DEFAULT 0,
level INTEGER NOT NULL DEFAULT 1,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS skills (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
name TEXT NOT NULL UNIQUE,
created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS volunteer_skills (
volunteer_profile_id UUID NOT NULL REFERENCES volunteer_profiles(id) ON DELETE CASCADE,
skill_id UUID NOT NULL REFERENCES skills(id) ON DELETE CASCADE,
PRIMARY KEY (volunteer_profile_id, skill_id)
);

CREATE TABLE IF NOT EXISTS events (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
title TEXT NOT NULL,
description TEXT NOT NULL DEFAULT '',
format event_format NOT NULL,
status event_status NOT NULL DEFAULT 'draft',
starts_at TIMESTAMPTZ NOT NULL,
ends_at TIMESTAMPTZ NOT NULL,
location TEXT,
max_participants INTEGER,
created_by UUID REFERENCES users(id) ON DELETE SET NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS tasks (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
event_id UUID REFERENCES events(id) ON DELETE SET NULL,
title TEXT NOT NULL,
description TEXT NOT NULL DEFAULT '',
status task_status NOT NULL DEFAULT 'created',
priority TEXT NOT NULL DEFAULT 'medium',
due_at TIMESTAMPTZ,
created_by UUID REFERENCES users(id) ON DELETE SET NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS task_assignments (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
task_id UUID NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
assigned_at TIMESTAMPTZ NOT NULL DEFAULT now(),
UNIQUE (task_id, user_id)
);

CREATE INDEX IF NOT EXISTS idx_organization_members_user_id ON organization_members(user_id);
CREATE INDEX IF NOT EXISTS idx_events_organization_id ON events(organization_id);

```

```
CREATE INDEX IF NOT EXISTS idx_tasks_organization_id ON tasks(organization_id);
```

Листинг Е.2 - Реализация правил достижений и начисления баллов

```
CREATE TABLE IF NOT EXISTS achievements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    code TEXT NOT NULL UNIQUE,
    name TEXT NOT NULL,
    description TEXT NOT NULL DEFAULT '',
    icon TEXT NOT NULL DEFAULT 'award',
    points_reward INTEGER NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS achievement_rules (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    achievement_id UUID NOT NULL UNIQUE REFERENCES achievements(id) ON DELETE CASCADE,
    code TEXT NOT NULL UNIQUE,
    trigger_type TEXT NOT NULL,
    threshold NUMERIC(10, 2),
    points INTEGER NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS volunteer_achievements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    achievement_id UUID NOT NULL REFERENCES achievements(id) ON DELETE CASCADE,
    earned_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (user_id, achievement_id)
);

CREATE TABLE IF NOT EXISTS points_transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    achievement_id UUID REFERENCES achievements(id) ON DELETE SET NULL,
    source_type TEXT NOT NULL DEFAULT 'achievement',
    source_id UUID,
    points INTEGER NOT NULL,
    reason TEXT NOT NULL DEFAULT '',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (user_id, achievement_id, source_type)
);

INSERT INTO achievements (code, name, description, icon, points_reward)
VALUES
    ('registration', 'Первый шаг', 'Регистрация в системе «Пульс».', 'user-plus', 20),
    ('profile_completed', 'Профиль заполнен', 'Заполнены основные данные профиля волонтера.',
'id-card', 30),
    ('first participation', 'Первое участие', 'Волонтер впервые отмечен на мероприятии.',
'calendar-check', 50),
    ('task_completed', 'Задача выполнена', 'Волонтер участвовал в выполненной задаче.', 'check-
circle', 50),
    ('hours_10', '10 часов помощи', 'Накоплено 10 подтвержденных волонтерских часов.', 'clock',
100),
    ('hours_25', '25 часов помощи', 'Накоплено 25 подтвержденных волонтерских часов.', 'clock',
180),
    ('hours_50', '50 часов помощи', 'Накоплено 50 подтвержденных волонтерских часов.', 'trophy',
300),
    ('hours_100', '100 часов помощи', 'Накоплено 100 подтвержденных волонтерских часов.',
'medal', 500)
ON CONFLICT (code) DO UPDATE
SET name = EXCLUDED.name,
    description = EXCLUDED.description,
    icon = EXCLUDED.icon,
    points_reward = EXCLUDED.points_reward;

INSERT INTO achievement_rules (achievement_id, code, trigger_type, threshold, points)
SELECT id, code, code, CASE
    WHEN code = 'hours_10' THEN 10
    WHEN code = 'hours_25' THEN 25
    WHEN code = 'hours_50' THEN 50
    WHEN code = 'hours_100' THEN 100
```

```

        ELSE NULL
    END, points_reward
FROM achievements
ON CONFLICT (code) DO UPDATE
SET threshold = EXCLUDED.threshold,
    points = EXCLUDED.points;

CREATE OR REPLACE FUNCTION recalculate_user_gamification(target_user_id UUID)
RETURNS void
LANGUAGE plpgsql
AS $$
DECLARE
    total_points INTEGER;
BEGIN
    INSERT INTO volunteer_profiles (user_id)
    VALUES (target_user_id)
    ON CONFLICT (user_id) DO NOTHING;

    WITH eligible AS (
        SELECT a.id, a.points_reward, a.name
        FROM achievements a
        JOIN volunteer_profiles vp ON vp.user_id = target_user_id
        JOIN users u ON u.id = target_user_id
        WHERE
            (a.code = 'registration')
            OR (
                a.code = 'profile_completed'
                AND (
                    COALESCE(vp.bio, '') <> ''
                    OR COALESCE(vp.city, '') <> ''
                    OR COALESCE(vp.phone, '') <> ''
                    OR COALESCE(vp.interests, '') <> ''
                )
            )
            OR (
                a.code = 'first_participation'
                AND EXISTS (
                    SELECT 1 FROM event_attendance ea WHERE ea.user_id =
target_user_id
                )
            )
            OR (
                a.code = 'task_completed'
                AND EXISTS (
                    SELECT 1
                    FROM task_assignments ta
                    JOIN tasks t ON t.id = ta.task_id
                    WHERE ta.user_id = target_user_id AND t.status = 'completed'
                )
            )
            OR (a.code = 'hours_10' AND vp.total_hours >= 10)
            OR (a.code = 'hours_25' AND vp.total_hours >= 25)
            OR (a.code = 'hours_50' AND vp.total_hours >= 50)
            OR (a.code = 'hours_100' AND vp.total_hours >= 100)
    ),
    new_awards AS (
        INSERT INTO volunteer_achievements (user_id, achievement_id)
        SELECT target_user_id, id FROM eligible
        ON CONFLICT (user_id, achievement_id) DO NOTHING
        RETURNING achievement_id
    )
    INSERT INTO points_transactions (user_id, achievement_id, points, reason)
    SELECT target_user_id, a.id, a.points_reward, a.name
    FROM achievements a
    JOIN new_awards na ON na.achievement_id = a.id
    WHERE a.points_reward <> 0
    ON CONFLICT (user_id, achievement_id, source_type) DO NOTHING;

    SELECT COALESCE(SUM(a.points_reward), 0)
    INTO total_points
    FROM volunteer_achievements va
    JOIN achievements a ON a.id = va.achievement_id
    WHERE va.user_id = target_user_id;

    UPDATE volunteer_profiles
    SET points = total_points,
        level = GREATEST(1, FLOOR(total_points / 100.0)::INTEGER + 1),

```

```

        updated_at = now()
    WHERE user_id = target_user_id;
END;
$$;

SELECT recalculate_user_gamification(id) FROM users;

CREATE INDEX IF NOT EXISTS idx_volunteer_achievements_user_id ON volunteer_achievements(user_id);
CREATE INDEX IF NOT EXISTS idx_points_transactions_user_id ON points_transactions(user_id);

```

Листинг Е.3 - Обработка заявок и связанных операций мероприятия

```

func (r *Repository) CreateApplication(ctx context.Context, eventID string, userID string, message
string) (Application, error) {
    var existingID string
    err := r.db.QueryRow(ctx, `SELECT id::text FROM event_applications WHERE event_id = $1 AND
user_id = $2`, eventID, userID).Scan(&exist
if err == nil {
    return Application{}, ErrAlreadyExists
}
if !errors.Is(err, pgx.ErrNoRows) {
    return Application{}, err
}
status, err := r.nextApplicationStatus(ctx, eventID)
if err != nil {
    return Application{}, err
}
var id string
err = r.db.QueryRow(ctx, `
    INSERT INTO event_applications (event_id, user_id, status, message)
    VALUES ($1, $2, $3, $4)
    RETURNING id::text
`, eventID, userID, status, message).Scan(&id)
if err != nil {
    return Application{}, err
}
return r.findApplication(ctx, eventID, id)
}

func (r *Repository) UpdateApplicationStatus(ctx context.Context, eventID string, applicationID
string, status string, rejectionReason
normalized := normalizeApplicationStatus(status)
tag, err := r.db.Exec(ctx, `
    UPDATE event_applications
    SET status = $3,
        rejection_reason = CASE WHEN $3 = 'rejected' THEN $4 ELSE '' END,
        updated_at = now()
    WHERE event_id = $1 AND id = $2
`, eventID, applicationID, normalized, rejectionReason)
if err != nil {
    return Application{}, err
}
if tag.RowsAffected() == 0 {
    return Application{}, ErrNotFound
}
if normalized == "approved" {
    if err := r.RebalanceWaitlist(ctx, eventID); err != nil {
        return Application{}, err
    }
}
return r.findApplication(ctx, eventID, applicationID)
}

func (r *Repository) BulkUpdateApplications(ctx context.Context, eventID string, request
BulkApplicationStatusRequest) ([]Application,
normalized := normalizeApplicationStatus(request.Status)
rows, err := r.db.Query(ctx, `
    UPDATE event_applications
    SET status = $3,
        rejection_reason = CASE WHEN $3 = 'rejected' THEN $4 ELSE '' END,
        updated_at = now()
    WHERE event_id = $1 AND id::text = ANY($2::text[])
    RETURNING id::text
`, eventID, request.ApplicationIDs, normalized, request.RejectionReason)

```



```

        if err != nil {
            return nil, err
        }
        defer rows.Close()
        ids := []string{}
        for rows.Next() {
            var id string
            if err := rows.Scan(&id); err != nil {
                return nil, err
            }
            ids = append(ids, id)
        }
        if err := rows.Err(); err != nil {
            return nil, err
        }
        if normalized == "approved" {
            if err := r.RebalanceWaitlist(ctx, eventID); err != nil {
                return nil, err
            }
        }
        items := make([]Application, 0, len(ids))
        for _, id := range ids {
            item, err := r.findApplication(ctx, eventID, id)
            if err != nil {
                return nil, err
            }
            items = append(items, item)
        }
        return items, nil
    }
}

func (r *Repository) DeleteApplication(ctx context.Context, eventID string, applicationID string) error {
    tag, err := r.db.Exec(ctx, `DELETE FROM event_applications WHERE event_id = $1 AND id = $2`,
eventID, applicationID)
    if err != nil {
        return err
    }
    if tag.RowsAffected() == 0 {
        return ErrNotFound
    }
    return nil
}

func (r *Repository) DeleteApplicationForUser(ctx context.Context, eventID string, applicationID
string, userID string) error {
    tag, err := r.db.Exec(ctx, `DELETE FROM event_applications WHERE event_id = $1 AND id = $2
AND user_id = $3`, eventID, applicationID,
userID)
    if err != nil {
        return err
    }
    if tag.RowsAffected() == 0 {
        return ErrNotFound
    }
    return nil
}

func (r *Repository) ListAttendance(ctx context.Context, eventID string) ([]Attendance, error) {
    rows, err := r.db.Query(ctx, `
        SELECT ea.id::text, ea.event_id::text, ea.user_id::text, concat_ws(' ', u.last_name,
u.first_name), u.email,
        ea.check_in_at, ea.check_out_at, ea.hours::float8
        FROM event_attendance ea
        JOIN users u ON u.id = ea.user_id
        WHERE ea.event_id = $1
        ORDER BY u.last_name, u.first_name
`, eventID)
    if err != nil {
        return nil, err
    }
    defer rows.Close()
    items := []Attendance{}
    for rows.Next() {
        var item Attendance
        var inAt, outAt sql.NullTime
        if err := rows.Scan(&item.ID, &item.EventID, &item.UserID, &item.UserName,
&item.Email, &inAt, &outAt, &item.Hours); err != nil {

```

```

        return nil, err
    }
    if inAt.Valid {
        item.CheckInAt = &inAt.Time
    }
    if outAt.Valid {

```

Листинг Е.4 - Backend: расчет показателей аналитики и итогов работы

```

func (r *Repository) Overview(ctx context.Context, filters Filters) (OverviewReport, error) {
    metrics, err := r.metrics(ctx, filters)
    if err != nil {
        return OverviewReport{}, err
    }
    applications, err := r.series(ctx, `event_applications`, `created_at`, filters)
    if err != nil {
        return OverviewReport{}, err
    }
    attendance, err := r.series(ctx, `event_attendance`, `created_at`, filters)
    if err != nil {
        return OverviewReport{}, err
    }
    top, err := r.topVolunteers(ctx)
    if err != nil {
        return OverviewReport{}, err
    }
    return OverviewReport{Metrics: metrics, Applications: applications, Attendance: attendance,
TopVolunteers: top}, nil
}

func (r *Repository) Volunteers(ctx context.Context, filters Filters) (VolunteersReport, error) {
    metrics, err := r.volunteerMetrics(ctx, filters)
    if err != nil {
        return VolunteersReport{}, err
    }
    status, err := r.group(ctx, `SELECT vp.status::text, COUNT(*)::float8 FROM volunteer_profiles
vp GROUP BY vp.status::text ORDER BY 1`)
    if err != nil {
        return VolunteersReport{}, err
    }
    hoursByLevel, err := r.group(ctx, `SELECT CONCAT('Уровень ', level), SUM(total_hours)::float8
FROM volunteer_profiles GROUP BY level O
    if err != nil {
        return VolunteersReport{}, err
    }
    top, err := r.topVolunteers(ctx)
    if err != nil {
        return VolunteersReport{}, err
    }
    return VolunteersReport{Metrics: metrics, Status: status, HoursByLevel: hoursByLevel,
TopVolunteers: top}, nil
}

func (r *Repository) Events(ctx context.Context, filters Filters) (EventsReport, error) {
    metrics, err := r.eventMetrics(ctx, filters)
    if err != nil {
        return EventsReport{}, err
    }
    byStatus, err := r.group(ctx, `SELECT status::text, COUNT(*)::float8 FROM events GROUP BY
status::text ORDER BY 1`)
    if err != nil {
        return EventsReport{}, err
    }
    applications, err := r.series(ctx, `event_applications`, `created_at`, filters)
    if err != nil {
        return EventsReport{}, err
    }
    attendance, err := r.series(ctx, `event_attendance`, `created_at`, filters)
    if err != nil {
        return EventsReport{}, err
    }
    return EventsReport{Metrics: metrics, ByStatus: byStatus, Applications: applications,
Attendance: attendance}, nil
}

```

```

func (r *Repository) Tasks(ctx context.Context, filters Filters) (TasksReport, error) {
    metrics, err := r.taskMetrics(ctx, filters)
    if err != nil {
        return TasksReport{}, err
    }
    byStatus, err := r.group(ctx, `SELECT status::text, COUNT(*)::float8 FROM tasks GROUP BY
status::text ORDER BY 1`)
    if err != nil {
        return TasksReport{}, err
    }
    byPriority, err := r.group(ctx, `SELECT priority, COUNT(*)::float8 FROM tasks GROUP BY
priority ORDER BY priority`)
    if err != nil {
        return TasksReport{}, err
    }
    completed, err := r.series(ctx, `tasks`, `completion_confirmed_at`, filters)
    if err != nil {
        return TasksReport{}, err
    }
    return TasksReport{Metrics: metrics, ByStatus: byStatus, ByPriority: byPriority, Completed:
completed}, nil
}

func (r *Repository) Gamification(ctx context.Context, filters Filters) (GamificationReport, error)
{
    metrics, err := r.gamificationMetrics(ctx, filters)
    if err != nil {
        return GamificationReport{}, err
    }
    byReason, err := r.groupWithFilters(ctx, `
SELECT COALESCE(NULLIF(a.name, ''), NULLIF(pt.reason, '')), pt.source_type),
SUM(pt.points)::float8
FROM points_transactions pt
LEFT JOIN achievements a ON a.id = pt.achievement_id
WHERE pt.created_at BETWEEN parse_period_from($1) AND parse_period_to($2)
GROUP BY COALESCE(NULLIF(a.name, ''), NULLIF(pt.reason, '')), pt.source_type)
ORDER BY SUM(pt.points) DESC
LIMIT 10
`, filters)
    if err != nil {
        return GamificationReport{}, err
    }
    pointsByDay, err := r.seriesSum(ctx, `points_transactions`, `created_at`, `points`, filters)
    if err != nil {
        return GamificationReport{}, err
    }
    transactions, err := r.gamificationTransactions(ctx, filters)
    if err != nil {
        return GamificationReport{}, err
    }
    return GamificationReport{Metrics: metrics, ByReason: byReason, PointsByDay: pointsByDay,
Transactions: transactions}, nil
}

func (r *Repository) Audit(ctx context.Context, filters Filters) (AuditReport, error) {
    metrics, err := r.auditMetrics(ctx, filters)
    if err != nil {
        return AuditReport{}, err
    }
    byAction, err := r.groupWithFilters(ctx, `
SELECT action, COUNT(*)::float8
FROM audit_log
WHERE created_at BETWEEN parse_period_from($1) AND parse_period_to($2)
GROUP BY action
ORDER BY COUNT(*) DESC
`, filters)
    if err != nil {
        return AuditReport{}, err
    }
    byEntity, err := r.groupWithFilters(ctx, `
SELECT entity_type, COUNT(*)::float8
FROM audit_log
WHERE created_at BETWEEN parse_period_from($1) AND parse_period_to($2)
GROUP BY entity_type
ORDER BY COUNT(*) DESC
`, filters)
    if err != nil {
        return AuditReport{}, err
    }

```

```

    }
    activity, err := r.series(ctx, `audit_log`, `created_at`, filters)
    if err != nil {
        return AuditReport{}, err
    }
    entries, err := r.auditEntries(ctx, filters)
    if err != nil {
        return AuditReport{}, err
    }
    return AuditReport{Metrics: metrics, ByAction: byAction, ByEntity: byEntity, Activity:
activity, Entries: entries}, nil
}

func (r *Repository) metrics(ctx context.Context, filters Filters) ([]Metric, error) {
    var volunteers, activeVolunteers, newVolunteers, events, applications, attendance, hours,
completedTasks float64
    err := r.db.QueryRow(ctx, `
        SELECT
            (SELECT COUNT(*)::float8 FROM volunteer_profiles),
            (SELECT COUNT(*)::float8 FROM volunteer_profiles WHERE status = 'active'),
            (SELECT COUNT(*)::float8 FROM volunteer_profiles WHERE created_at BETWEEN
parse_period_from($1) AND parse_period_to($2)),
            (SELECT COUNT(*)::float8 FROM events WHERE starts_at BETWEEN
parse_period_from($1) AND parse_period_to($2)),
            (SELECT COUNT(*)::float8 FROM event_applications WHERE created_at BETWEEN
parse_period_from($1) AND parse_period_to($2)),
            (SELECT COUNT(*)::float8 FROM event_attendance WHERE created_at BETWEEN
parse_period_from($1) AND parse_period_to($2)),
            (SELECT COALESCE(SUM(hours), 0)::float8 FROM time_entries WHERE status =
'approved' AND reviewed_at BETWEEN parse_period_from($1) AND
parse_period_to($2)),
            (SELECT COUNT(*)::float8 FROM tasks WHERE status = 'completed' AND
completion_confirmed_at BETWEEN parse_period_from($1) AND parse_p
`, filters.From, filters.To).Scan(&volunteers, &activeVolunteers, &newVolunteers, &events,
&applications, &attendance, &hours, &completedTasks)
    if err != nil {
        return nil, err
    }
    return []Metric{
        {Code: "volunteers", Label: "Волонтеры", Value: volunteers},
        {Code: "activeVolunteers", Label: "Активные волонтеры", Value: activeVolunteers},
        {Code: "newVolunteers", Label: "Новые за период", Value: newVolunteers},
        {Code: "events", Label: "Мероприятия", Value: events},
        {Code: "applications", Label: "Заявки", Value: applications},
        {Code: "attendance", Label: "Посещения", Value: attendance},
        {Code: "hours", Label: "Подтвержденные часы", Value: hours},
        {Code: "completedTasks", Label: "Выполненные задачи", Value: completedTasks},
    }, nil
}

func (r *Repository) volunteerMetrics(ctx context.Context, filters Filters) ([]Metric, error) {
    all, err := r.metrics(ctx, filters)
    if err != nil {
        return nil, err
    }
    return all[:3], nil
}

func (r *Repository) eventMetrics(ctx context.Context, filters Filters) ([]Metric, error) {
    all, err := r.metrics(ctx, filters)
    if err != nil {
        return nil, err
    }
    return all[3:6], nil
}

func (r *Repository) taskMetrics(ctx context.Context, filters Filters) ([]Metric, error) {
    all, err := r.metrics(ctx, filters)
    if err != nil {
        return nil, err
    }
    return []Metric{all[7]}, nil
}

func (r *Repository) gamificationMetrics(ctx context.Context, filters Filters) ([]Metric, error) {
    var totalPoints, transactions, recipients, reasons float64
    err := r.db.QueryRow(ctx, `

```

```

        SELECT
            COALESCE(SUM(points), 0)::float8,
            COUNT(*)::float8,
            COUNT(DISTINCT user_id)::float8,
            COUNT(DISTINCT COALESCE(achievement_id::text, reason, source_type))::float8
        FROM points_transactions
        WHERE created_at BETWEEN parse_period_from($1) AND parse_period_to($2)
    `, filters.From, filters.To).Scan(&totalPoints, &transactions, &recipients, &reasons)
    if err != nil {
        return nil, err
    }
    return []Metric{
        {Code: "totalPoints", Label: "Начислено баллов", Value: totalPoints},
        {Code: "transactions", Label: "Начислений", Value: transactions},
        {Code: "recipients", Label: "Получателей", Value: recipients},
        {Code: "reasons", Label: "Оснований", Value: reasons},
    }, nil
}

func (r *Repository) auditMetrics(ctx context.Context, filters Filters) ([]Metric, error) {
    var actions, users, failed, deletes float64
    err := r.db.QueryRow(ctx, `
        SELECT
            COUNT(*)::float8,
            COUNT(DISTINCT user_id)::float8,
            COUNT(*) FILTER (WHERE status_code >= 400)::float8,
            COUNT(*) FILTER (WHERE action = 'delete')::float8
        FROM audit_log
        WHERE created_at BETWEEN parse_period_from($1) AND parse_period_to($2)
    `, filters.From, filters.To).Scan(&actions, &users, &failed, &deletes)
    if err != nil {
        return nil, err
    }
    return []Metric{
        {Code: "actions", Label: "Действий", Value: actions},
        {Code: "users", Label: "Сотрудников", Value: users},
        {Code: "failed", Label: "С ошибкой", Value: failed},
        {Code: "deletes", Label: "Удалений", Value: deletes},
    }, nil
}

func (r *Repository) topVolunteers(ctx context.Context) ([]TopVolunteer, error) {
    rows, err := r.db.Query(ctx, `
        SELECT u.id::text, concat_ws(' ', u.last_name, u.first_name), u.email,
        vp.total_hours::float8, vp.points, vp.level
        FROM volunteer_profiles vp
        JOIN users u ON u.id = vp.user_id
        ORDER BY vp.total_hours DESC, vp.points DESC, u.last_name, u.first_name
        LIMIT 10
    `)
    if err != nil {
        return nil, err
    }
}

```

Листинг Е.5 - Frontend API: запросы к аналитическим отчетам и экспорту

```

import { API_BASE_URL, ApiError, apiRequest } from '../..shared/api'
import { authState } from '../auth/store'
import type { AuditReport, EventsReport, GamificationReport, ManagementReport, OverviewReport,
TasksReport, VolunteersReport } from './

const token = () => authState.token

function query(params: { from?: string; to?: string }) {
    const search = new URLSearchParams()
    if (params.from) search.set('from', params.from)
    if (params.to) search.set('to', params.to)
    return search.toString() ? `?${search}` : ''
}

export const fetchAnalyticsOverview = (params: { from?: string; to?: string } = {}) =>
    apiRequest<{ item: OverviewReport }>(`/analytics/overview${query(params)}`, { token: token() })

export const fetchAnalyticsVolunteers = (params: { from?: string; to?: string } = {}) =>

```

```

    apiRequest<{ item: VolunteersReport }>(`/analytics/volunteers${query(params)}`, { token:
token() })

export const fetchAnalyticsEvents = (params: { from?: string; to?: string } = {}) =>
    apiRequest<{ item: EventsReport }>(`/analytics/events${query(params)}`, { token: token() })

export const fetchAnalyticsTasks = (params: { from?: string; to?: string } = {}) =>
    apiRequest<{ item: TasksReport }>(`/analytics/tasks${query(params)}`, { token: token() })

export const fetchAnalyticsGamification = (params: { from?: string; to?: string } = {}) =>
    apiRequest<{ item: GamificationReport }>(`/analytics/gamification${query(params)}`, { token:
token() })

export const fetchAnalyticsAudit = (params: { from?: string; to?: string } = {}) =>
    apiRequest<{ item: AuditReport }>(`/analytics/audit${query(params)}`, { token: token() })

export const fetchManagementReport = (kind: string, params: { from?: string; to?: string } = {}) =>
    apiRequest<{ item: ManagementReport }>(`/analytics/management/${kind}${query(params)}`, { token:
token() })

export async function downloadManagementReport(kind: string, format: 'xlsx' | 'pdf', params:
{ from?: string; to?: string } = {}) {
    const response = await fetch(`${API_BASE_URL}/analytics/management/${kind}/export/${format}${
query(params)}`, {
        headers: {
            Authorization: `Bearer ${authState.token ?? ''}`
        }
    })
    if (!response.ok) {
        const payload = await response.json().catch(() => null)
        throw new ApiError(payload?.message ?? 'Не удалось выгрузить отчет', response.status)
    }
    const blob = await response.blob()
    const url = URL.createObjectURL(blob)
    const link = document.createElement('a')
    link.href = url
    link.download = `puls-${kind}-report.${format}`
    document.body.appendChild(link)
    link.click()
    link.remove()
    URL.revokeObjectURL(url)
}

```

Листинг Е.6 - Frontend: отображение итоговых показателей AnalyticsPage

```

<script setup lang="ts">
import { computed, onMounted, ref, watch } from 'vue'
import { useRoute } from 'vue-router'
import { Award, BarChart3, CalendarDays, CheckSquare, ClipboardList, Download, History, UsersRound }
from 'lucide-vue-next'
import {
    downloadManagementReport,
    fetchAnalyticsAudit,
    fetchAnalyticsEvents,
    fetchAnalyticsGamification,
    fetchAnalyticsOverview,
    fetchAnalyticsTasks,
    fetchAnalyticsVolunteers,
    fetchManagementReport
} from '../entities/analytics/api'
import { downloadExport, type ExportKind } from '../entities/exports/api'
import type {
    AnalyticsReport,
    AuditEntry,
    ChartPoint,
    GamificationTransaction,
    ManagementReportRow,
    Metric,
    TopVolunteer
} from '../entities/analytics/types'

const route = useRoute()

```

```

const report = ref<AnalyticsReport | null>(null)
const errorMessage = ref('')
const exportMessage = ref('')
const isLoading = ref(false)
const isExporting = ref('')
const from = ref('')
const to = ref('')
const savedFilterName = ref('')
const savedFilters = ref<Array<{ name: string; from: string; to: string }>>([])

const reports = [
  { code: 'overview', title: 'Обзор', description: 'Ключевые показатели, заявки, посещаемость и топ волонтеров.', icon: BarChart3, to:
    { code: 'volunteers', title: 'Волонтеры', description: 'Статусы, уровни, часы и лидеры волонтерской активности.', icon: UsersRound, t
      { code: 'events', title: 'Мероприятия', description: 'Статусы мероприятий, заявки и фактическая посещаемость.', icon: CalendarDays, t
        { code: 'tasks', title: 'Задачи', description: 'Выполнение задач, статусы и распределение по приоритетам.', icon: CheckSquare, to: '/'
          { code: 'gamification', title: 'Начисления', description: 'Все начисления баллов: кому, сколько и за какое действие.', icon: Award, t
            { code: 'audit', title: 'Действия сотрудников', description: 'Журнал создания, изменения, удаления и служебных операций.', icon: Hist
  ]

const managementReports = [
  { code: 'executive', title: 'Executive dashboard', description: 'Сводка для руководителя по ключевым показателям и рискам.', icon: Ba
    { code: 'management', title: 'Для руководства', description: 'Динамика заявок и операционные показатели периода.', icon: ClipboardLis
      { code: 'grant', title: 'Для грантодателя', description: 'Подтвержденные часы и вклад организаций.', icon: Award, to: '/analytics/gra
        { code: 'branches', title: 'Филиалы', description: 'Активность филиалов, мероприятий и задач.', icon: CalendarDays, to: '/analytics/b
          { code: 'coordinators', title: 'Координаторы', description: 'Активность сотрудников по аудиту действий.', icon: UsersRound, to: '/ana
            { code: 'risks', title: 'Проблемные зоны', description: 'Ожидающие заявки, часы и ошибки операций.', icon: History, to: '/analytics/r
  ]

const exports = [
  { kind: 'volunteers', title: 'Волонтеры', description: 'Профили, статусы, часы, баллы и контакты.' },
  { kind: 'events', title: 'Мероприятия', description: 'Список мероприятий, статусы, даты, форматы и лимиты.' },
  { kind: 'applications', title: 'Заявки', description: 'Заявки волонтеров, статусы, события и сообщения.' },
  { kind: 'time-entries', title: 'Часы', description: 'Подтвержденные и ожидающие проверки записи времени.' },
  { kind: 'tasks', title: 'Задачи', description: 'Задачи, статусы, приоритеты, сроки и подтверждение.' },
  { kind: 'certificates', title: 'Сертификаты', description: 'Выданные документы, часы, коды проверки и даты.' }
] satisfies Array<{ kind: ExportKind; title: string; description: string }>

const currentReportCode = computed(() => String(route.params.report || ''))
const currentReport = computed(() => [...reports, ...managementReports].find((item) => item.code === currentReportCode.value))
const isCatalog = computed(() => !currentReport.value)
const metrics = computed<Metric[]>(() => report.value?.metrics ?? [])
const topVolunteers = computed<TopVolunteer[]>(() => ('topVolunteers' in (report.value ?? {})) ? (report.value as any).topVolunteers : [
const transactions = computed<GamificationTransaction[]>(() => ('transactions' in (report.value ?? {})) ? (report.value as any).transact
const auditEntries = computed<AuditEntry[]>(() => ('entries' in (report.value ?? {})) ? (report.value as any).entries : [])
const managementRows = computed<ManagementReportRow[]>(() => ('rows' in (report.value ?? {})) ? (report.value as any).rows : [])
const riskRows = computed<ManagementReportRow[]>(() => ('risks' in (report.value ?? {})) ? (report.value as any).risks : [])
const isManagementReport = computed(() => managementReports.some((item) => item.code === currentReportCode.value))

const chartSections = computed(() => {
  if (!report.value) return []
  const item = report.value as any

```

```

    if (currentReportCode.value === 'overview') {
      return [
        { title: 'Заявки по дням', items: item.applications as ChartPoint[] },
        { title: 'Посещения по дням', items: item.attendance as ChartPoint[] }
      ]
    }
    if (currentReportCode.value === 'volunteers') {
      return [
        { title: 'Статусы волонтеров', items: item.status as ChartPoint[] },
        { title: 'Часы по уровням', items: item.hoursByLevel as ChartPoint[] }
      ]
    }
    if (currentReportCode.value === 'events') {
      return [
        { title: 'Статусы мероприятий', items: item.byStatus as ChartPoint[] },
        { title: 'Заявки по дням', items: item.applications as ChartPoint[] },
        { title: 'Посещения по дням', items: item.attendance as ChartPoint[] }
      ]
    }
    if (currentReportCode.value === 'tasks') {
      return [
        { title: 'Статусы задач', items: item.byStatus as ChartPoint[] },
        { title: 'Приоритеты задач', items: item.byPriority as ChartPoint[] },
        { title: 'Выполнено по дням', items: item.completed as ChartPoint[] }
      ]
    }
    if (currentReportCode.value === 'gamification') {
      return [
        { title: 'Баллы по основаниям', items: item.byReason as ChartPoint[] },
        { title: 'Начислено по дням', items: item.pointsByDay as ChartPoint[] }
      ]
    }
    if (isManagementReport.value) {
      return [
        { title: 'Строки отчета', items: managementRows.value.map((row) => ({ label: `${row.label}: ${row.group}`, value: row.value })) },
        { title: 'Риски', items: riskRows.value.map((row) => ({ label: `${row.label}: ${row.group}`, value: row.value })) }
      ]
    }
    return [
      { title: 'Действия', items: item.byAction as ChartPoint[] },
      { title: 'Разделы системы', items: item.byEntity as ChartPoint[] },
      { title: 'Активность по дням', items: item.activity as ChartPoint[] }
    ]
  })
}

async function load() {
  if (isCatalog.value) return
  isLoading.value = true
  errorMessage.value = ''
  const params = { from: from.value ? new Date(from.value).toISOString() : '', to: to.value ? new Date(to.value).toISOString() : '' }
  try {
    if (currentReportCode.value === 'overview') report.value = (await fetchAnalyticsOverview(params)).item
    if (currentReportCode.value === 'volunteers') report.value = (await fetchAnalyticsVolunteers(params)).item
    if (currentReportCode.value === 'events') report.value = (await fetchAnalyticsEvents(params)).item
    if (currentReportCode.value === 'tasks') report.value = (await fetchAnalyticsTasks(params)).item
    if (currentReportCode.value === 'gamification') report.value = (await fetchAnalyticsGamification(params)).item
    if (currentReportCode.value === 'audit') report.value = (await fetchAnalyticsAudit(params)).item
    if (isManagementReport.value) report.value = (await fetchManagementReport(currentReportCode.value, params)).item
  } catch (error) {
    errorMessage.value = error instanceof Error ? error.message : 'Не удалось загрузить отчет'
  } finally {
    isLoading.value = false
  }
}

function loadSavedFilters() {
  savedFilters.value = JSON.parse(localStorage.getItem('unityaid:report-filters') || '[]')
}

```



```

function saveCurrentFilter() {
  const name = savedFilterName.value.trim() || `Фильтр ${savedFilters.value.length + 1}`
  const next = [{ name, from: from.value, to: to.value }, ...savedFilters.value.filter((item) =>
item.name !== name)].slice(0, 8)
  savedFilters.value = next
  localStorage.setItem('unityaid:report-filters', JSON.stringify(next))
  savedFilterName.value = ''
}

function applySavedFilter(name: string) {
  const item = savedFilters.value.find((filter) => filter.name === name)
  if (!item) return
  from.value = item.from
  to.value = item.to
  load()
}

async function exportReport(format: 'xlsx' | 'pdf') {
  exportMessage.value = ''
  try {
    await downloadManagementReport(currentReportCode.value, format, {
      from: from.value ? new Date(from.value).toISOString() : '',
      to: to.value ? new Date(to.value).toISOString() : ''
    })
  } catch (error) {
    exportMessage.value = error instanceof Error ? error.message : 'Не удалось выгрузить отчет'
  }
}

function barWidth(items: ChartPoint[], value: number) {
  const max = Math.max(1, ...items.map((item) => item.value))
  return `${Math.max(4, (value / max) * 100)}%`
}

function formatDateTime(value: string) {
  if (!value) return ''
  return new Date(value).toLocaleString('ru-RU', { dateStyle: 'short', timeStyle: 'short' })
}

function formatAction(entry: AuditEntry) {
  const id = entry.entityId ? `#${entry.entityId}` : ''
  return `${entry.method} ${entry.action}: ${entry.entityType}${id} · ${entry.path}`
}

async function exportData(kind: ExportKind) {
  isExporting.value = kind
  exportMessage.value = ''
  try {
    await downloadExport(kind)
  } catch (error) {
    exportMessage.value = error instanceof Error ? error.message : 'Не удалось выгрузить данные'
  } finally {
    isExporting.value = ''
  }
}

watch(() => route.params.report, () => {
  report.value = null
  load()
})
onMounted(() => {
  loadSavedFilters()
  load()
})
</script>

<template>
  <section class="page-section">
    <div class="page-heading">
      <div>
        <p class="eyebrow">Аналитика</p>
        <h1>{{ currentReport?.title || 'Отчеты' }}</h1>

```

```

        <p>{{ currentReport?.description || 'Выберите отчет, чтобы посмотреть управленческие
показатели системы «Пульс».' }}</p>
    </div>
</div>

<div v-if="isCatalog" class="analytics-report-grid">
    <RouterLink v-for="item in [...reports, ...managementReports]" :key="item.code"
class="analytics-report-card" :to="item.to">
        <component :is="item.icon" :size="24" />
        <div>
            <strong>{{ item.title }}</strong>
            <p>{{ item.description }}</p>
        </div>
    </RouterLink>
</div>

<section v-if="isCatalog" class="detail-panel analytics-top-panel">
    <div class="section-heading">
        <div>
            <p class="eyebrow">Экспорт</p>
            <h2>Выгрузка ключевых списков</h2>
        </div>
    </div>
    <div class="export-grid">
        <article v-for="item in exports" :key="item.kind" class="export-card">
            <div>
                <strong>{{ item.title }}</strong>
                <p>{{ item.description }}</p>
            </div>
            <button class="secondary-action" type="button" :disabled="isExporting === item.kind"
@click="exportData(item.kind)">
                <Download :size="17" />
                <span>{{ isExporting === item.kind ? 'Готовим...' : 'CSV' }}</span>
            </button>
        </article>
    </div>
</div>

```

Листинг Е.7 - Регистрация REST-маршрутов аналитики и сертификатов

```

certificatesRepository := certificates.NewRepository(deps.DB)
certificatesService := certificates.NewService(certificatesRepository)
certificatesHandler := certificates.NewHandler(certificatesService, authorizer)

certificatesGroup := api.Group("/certificates", authMiddleware, auditMiddleware)
certificatesGroup.GET("", certificatesHandler.List)
certificatesGroup.POST("/generate", certificatesHandler.Generate)
certificatesGroup.GET("/download/:id", certificatesHandler.Download)
api.GET("/certificates/verify/:code", certificatesHandler.Verify)

analyticsRepository := analytics.NewRepository(deps.DB)
analyticsService := analytics.NewService(analyticsRepository)
analyticsHandler := analytics.NewHandler(analyticsService)

analyticsGroup := api.Group("/analytics", authMiddleware, auditMiddleware)
analyticsGroup.GET("/overview", analyticsHandler.Overview)
analyticsGroup.GET("/volunteers", analyticsHandler.Volunteers)
analyticsGroup.GET("/events", analyticsHandler.Events)
analyticsGroup.GET("/tasks", analyticsHandler.Tasks)
analyticsGroup.GET("/gamification", analyticsHandler.Gamification)
analyticsGroup.GET("/audit", analyticsHandler.Audit)
analyticsGroup.GET("/management/:kind", analyticsHandler.Management)
analyticsGroup.GET("/management/:kind/export/:format", analyticsHandler.ExportManagement)

exportsHandler := dataexports.NewHandler(deps.DB)
exportsGroup := api.Group("/exports", authMiddleware, auditMiddleware, canManageContent)
exportsGroup.GET("/:kind", exportsHandler.Download)

```